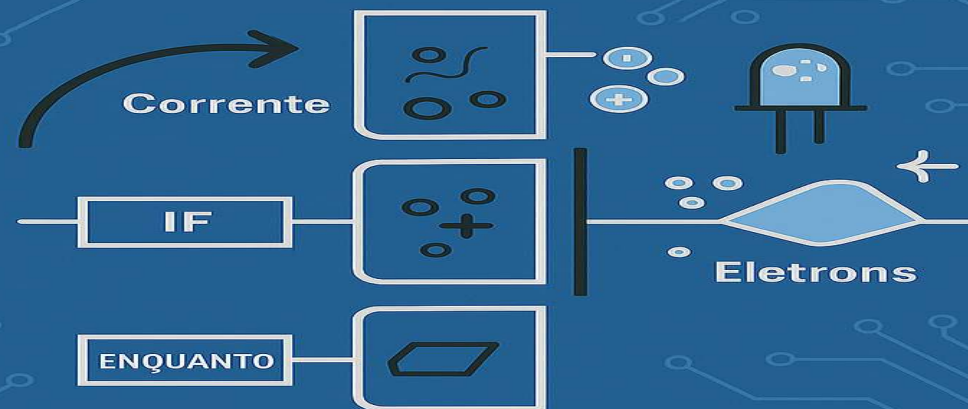


LÓGICA DE PROGRAMAÇÃO

TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS



HILTON ELIAS TOMASZEZISIIC DOS SANTOS
PEDRO GRAÇAS ALVES JUNIOR

SENAI

Desenvolvi esta Aula abrangente, repleta de exercícios para oferecer amplas oportunidades de prática. Quase todas as aulas incluem uma correspondente sessão de exercícios, proporcionando a chance de resolver problemas juntamente comigo.

Solicito sua dedicação e esforço para que, ao término desta unidade curricular, você esteja preparado para iniciar os estudos em uma linguagem de programação.

Este estudo abrangerá todos os fundamentos necessários para dar o primeiro passo como desenvolvedor.

Além de ser seu instrutor, pretendo também ser seu motivador, pois aprender programação é desafiador, especialmente nos primeiros estágios.

Nesta aula, iniciaremos o plano de estudos, com os conceitos fundamentais de algoritmos, lógica de programação e a linguagem que utilizaremos, será o Portugol.

Iremos iniciar através do cronograma da unidade curricular. Nesta aula em particular, abordaremos a introdução, variáveis, tipos primitivos e operações de entrada e saída de dados.

A seguir, abordaremos os operadores, incluindo os aritméticos, relacionais e lógicos. Em relação às estruturas condicionais, dedicaremos varias aulas para explorar a utilização da estrutura **SE simples e composta**.

Após isso, teremos sessões de exercícios.

Abordaremos a estrutura chamada "**Escolha Caso**" e apresentaremos alguns exercícios para prática, discutiremos as **estruturas de repetição**, que compreendem as **estruturas enquanto, repita e para**.

Exploraremos os **conceitos de procedimentos e funções**, destacando suas diferenças, os tópicos de **variável global** e **variável local**, explicando como são utilizados, discutiremos a passagem de valores por referência em procedimentos e funções.

Vetores e matrizes também serão temas abordados, acompanhados por uma série de exercícios práticos.

As aulas serão centradas na aplicação prática, proporcionando conceitos seguidos imediatamente por exercícios.

Finalmente, concluiremos com a criação de um algoritmo para algum jogo como o último exercício.

LÓGICA DE PROGRAMAÇÃO

A primeira coisa que nós temos que entender é o que é um algoritmo. Para começar a entender algoritmos, temos que pensar no dia a dia.

O que nós normalmente fazemos quando nós acordamos?

Nós Levantamos, depois tomamos banho, trocamos de roupa, tomamos o café da manhã, pegamos o ônibus ou de carro, desce do ônibus ou do carro.... Então, essa sequência de passos que eu estou mostrando aqui nada mais é que um algoritmo.

O algoritmo vai descrever como uma coisa acontece.

Vamos ver outro exemplo:

Eu estou em uma faixa de pedestre e quero atravessar a rua.

O que eu faço?

Primeiro vou até a faixa, olho para a direita, olho para a esquerda e aí sim eu atravesso. Se eu esquecer, por exemplo, de olhar para a direita ou olhar para a esquerda, isso pode ocorrer um acidente. Um carro pode vir correndo e me atropelar e eu nem vi isso acontecer.

Por que eu estou falando isso?

Algoritmos para programação é a mesma coisa.

Se você esquece de um passo, no seu código, no seu algoritmo, pode gerar um erro no seu programa. Então, temos que prestar bastante atenção, porque o computador não pensa. Embora nós temos hoje máquinas muito potentes, elas não pensam, elas já são programadas passo a passo para fazer o que elas fazem. Então, você precisa ensinar o computador ao que ele tem que fazer. O computador não pensa, então nós, que pensamos, vamos mostrar para ele passo a passo o que é para ele fazer.

Isso é algoritmo. Apesar de hoje em dia já termos a inteligência artificial, eu acredito que nunca chegaremos ao ponto em que um computador será capaz de pensar sozinho, sem que alguém o programe para isso.

O programe para fazer um cálculo ou qualquer coisa, qualquer situação, é passo a passo. Por isso, você precisa aprender algoritmos para você ser um futuro desenvolvedor.

O computador tem uma linguagem, tem uma maneira de entender o que a gente quer passar.

Então, nós temos que aprender a programar da maneira que ele entende. Hoje em dia, tudo envolve programação.

- ✓ O sistema para um avião envolve programação.
- ✓ O celular que você usa envolve programação.
- ✓ Até geladeiras hoje já temos com inteligência artificial, com uma programação pesada.
- ✓ Temos óculos 3D.
- ✓ O Google Play que você usa aí no celular tem programação.
- ✓ E não vai parar por aí, não.

No mundo que nós vivemos, tão moderno, aprender a programar é investir no futuro, porque é uma área não para de crescer. A área da tecnologia, a área de TI, é muito gigantesca e você tem aí diversas oportunidades ao redor do mundo que envolvem a programação.

Eu dei um exemplo do dia a dia, agora eu vou mostrar um exemplo de somar dois números.

Como nós faríamos para somar dois números?

Eu quero criar um programa, bem simples, que só vai somar dois números.

Vamos começar com o básico.

Como vou somar dois números, preciso desses números, preciso que a pessoa digite esses números.

Então, nós vamos pedir o número do teclado.

1. Primeiro passo é pedir o número do teclado.
2. Em seguida, nós pedimos mais um número, porque para somar, nós precisamos pelo menos de dois números.
3. E nós criaremos um algoritmo para calcular esses números.
4. Calculamos, próximo passo, mostrar o resultado na tela.

Então, algoritmos é mais ou menos assim.

Calcular uma idade.

Pense aí como você calcularia uma idade.

Como você faria um programa para calcular a idade?

1. Primeira coisa, posso pedir o ano de nascimento dessa pessoa via teclado. Peguei essa idade.

Como é calculado a idade de uma pessoa?

Pegamos o ano atual e subtraí com o ano de nascimento dessa pessoa.

2. O próximo passo é pegar o ano atual.

Como nós fazemos isso?

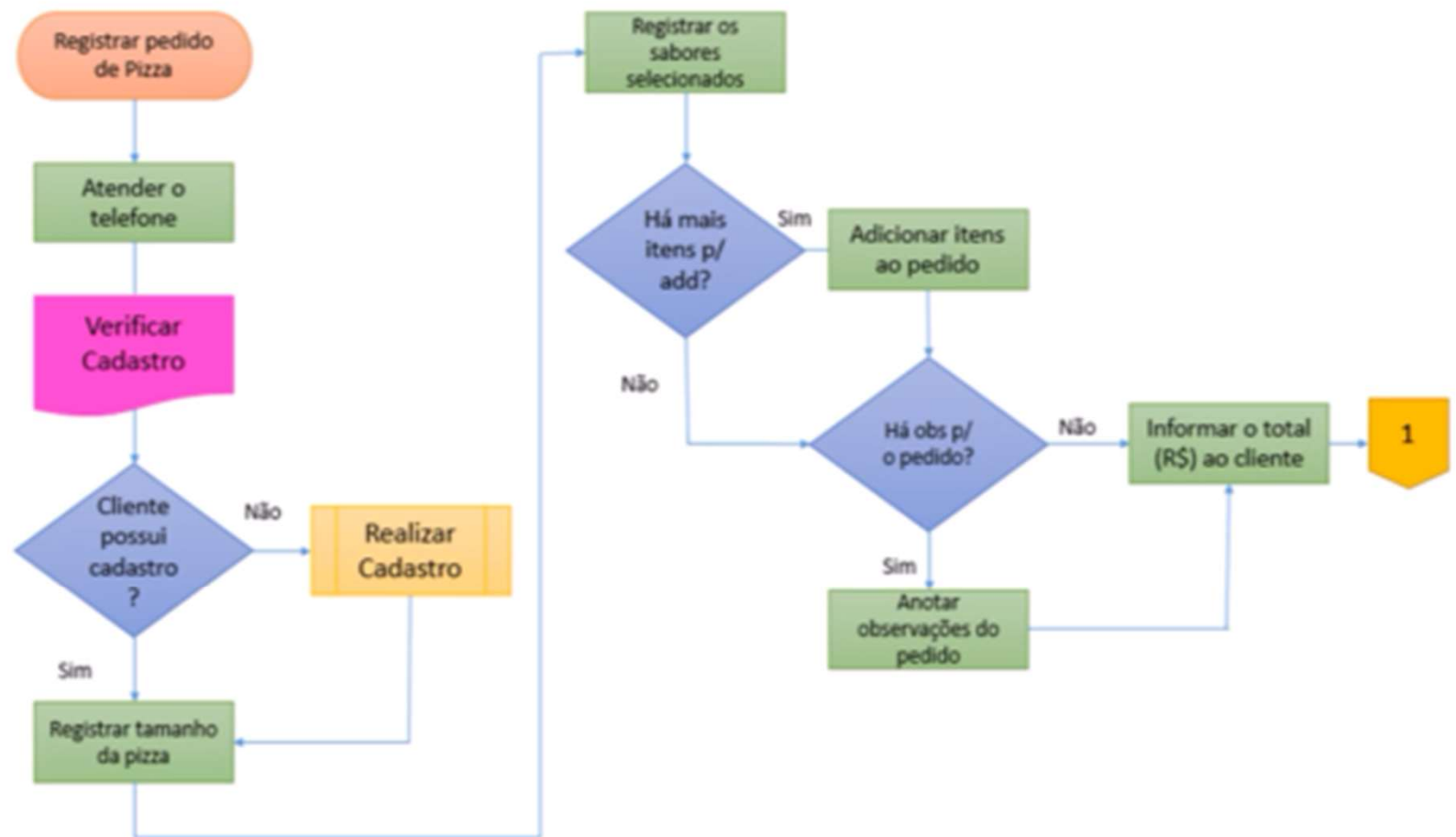
Podemos pegar no sistema.

2. O próximo passo é pegar o ano de nascimento da pessoa.
3. Pegamos o ano atual e a data de nascimento dessa pessoa, nós vamos fazer o cálculo.
4. Então, subtraí o ano atual com o ano de nascimento e mostra na tela.

Esse é mais um exemplo de algoritmo.

FERRAMENTAS

Fluxograma



FERRAMENTAS

Antigamente tínhamos o **fluxograma**, que é um passo a passo também, só que em forma de desenho. Isso já não é muito utilizado mais. Pode ser que planejem o programa antes de colocar a mão na massa, só que eu não vejo mais a utilização dele.

Já o **Portugol** é muito mais parecido com **linguagem de programação**, só que ele também é baseado nesse **fluxograma**. Por exemplo, aqui veremos algumas figuras, onde temos **entradas, saídas e comandos de decisão**.

Se isso for **verdade**, se alguma coisa for verdade, por exemplo:

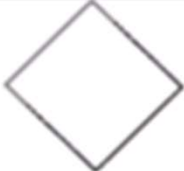
Cliente, possui cadastro?

Se sim, eu faço uma coisa;

Se não, eu vou para outro caminho e faço uma outra coisa.

Essa é uma decisão que o programa vai tomar. E tem vários símbolos.

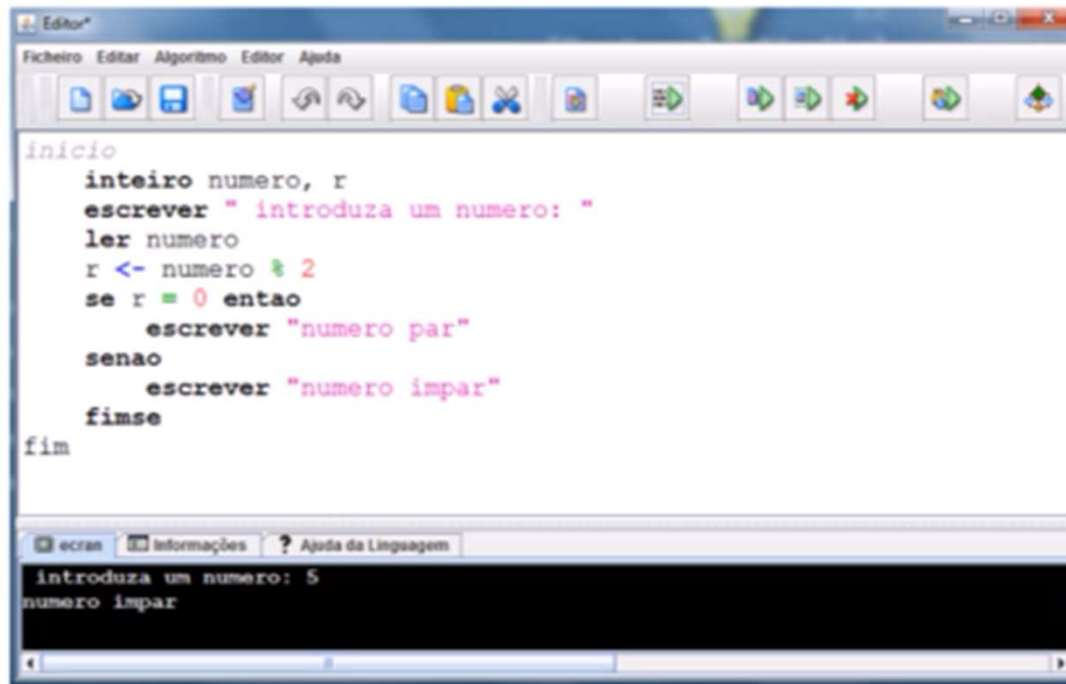
Temos o quadrado, o círculo inicial.... Cada coisa serve para alguma utilidade. Vejam um gráfico aqui com a figura e o seu significado:

FIGURA	SIGNIFICADO
	Figura para definir início e fim do algoritmo
	Figura usada no processamento de cálculo, atribuições e processamento de dados em geral
	Figura utilizada na representação de entrada de dados
	Figura utilizada para representação da saída de dados
	Figura que indica o processo seletivo ou condicional, possibilitando o desvio no caminho do processamento
	Símbolo geométrico usado como conector
	Símbolo que identifica o sentido do fluxo de dados, permitindo a conexão entre as outras figuras existentes

Nós iremos utilizar o **Portugol**, ou também chamado de **pseudocódigo**. Ele é o mais utilizado atualmente, e é mais próximo da **linguagem de programação**.

Então veja um exemplo, que faz a verificação se o número é par ou ímpar e mostrar na tela.

Tem várias ferramentas para você aprender **pseudocódigo**. A que nós utilizaremos é o **Visualg**, que é muito conhecido.



The image shows a screenshot of the Visualg IDE. The main editor window displays a Portugol program. The code is as follows:

```
inicio
    inteiro numero, r
    escrever " introduza um numero: "
    ler numero
    r <- numero % 2
    se r = 0 entao
        escrever "numero par"
    senao
        escrever "numero impar"
    fimse
fim
```

Below the editor, there is a console window titled "ecran" which shows the output of the program: "introduza um numero: 5" followed by "numero impar". The Visualg interface includes a menu bar with "Ficheiro", "Editar", "Algoritmo", "Editor", and "Ajuda", and a toolbar with various icons for file operations, editing, and execution.

Agora vou apresentar pra vocês a diferença de algo muito importante. Quando você aprende algoritmos, você aprende o funcionamento de uma **linguagem estrutural**. Como assim? Existe a **linguagem estrutural** e a **linguagem orientada a objetos**.

A **linguagem estrutural** é aquela que segue um padrão, digamos assim, um passo a passo. Terminou de executar a primeira linha, vai para a segunda, terminou de executar a segunda, vai para a terceira e assim sucessivamente.

Não que a **linguagem orientada a objetos** não seja assim, a **linguagem orientada a objetos** também é assim, só que ela é muito mais **dinâmica**. Ela tem **classes**, uma classe chama a outra, comunica com a outra. Mas é muito importante, você primeiro aprender a **linguagem estrutural** e só depois aprender a **linguagem orientada a objetos**.

Se você der um passo maior que a perna, digamos assim, tentar ir direto aprendendo uma **linguagem orientada a objetos**, vai chegar um ponto que você irá dizer: não estou entendendo nada, vou parar.

Então, é melhor você ir devagar. É por isso que o **Portugol** não é **orientado a objeto**, ele é **estrutural** e é isso que nós aprenderemos.

Para vocês conhecerem que **linguagens são estruturais**, temos, por exemplo:

A **linguagem C** é um tipo de **linguagem** que é **estrutural**.

Então, como falei, você tem, por exemplo, seis comandos, serão executados linha após linha de comando.

- Primeiro será o comando 1;
- Depois será o comando 2;
- Depois será o comando 3;
- Depois será o comando 4;
- Depois será o comando
- e assim por diante.

C

comando 1
comando 2
comando 3
comando 4
comando 5
comando 6

Diferente, não tão **diferente**, mas é **diferente** de uma **linguagem orientada a objetos** como, por exemplo, **C++**, **C#**, **Java** e etc.

Hoje em dia, o que é mais utilizado?

A linguagem orientada a objetos.

A linguagem orientada a objeto é o que domina o mercado.

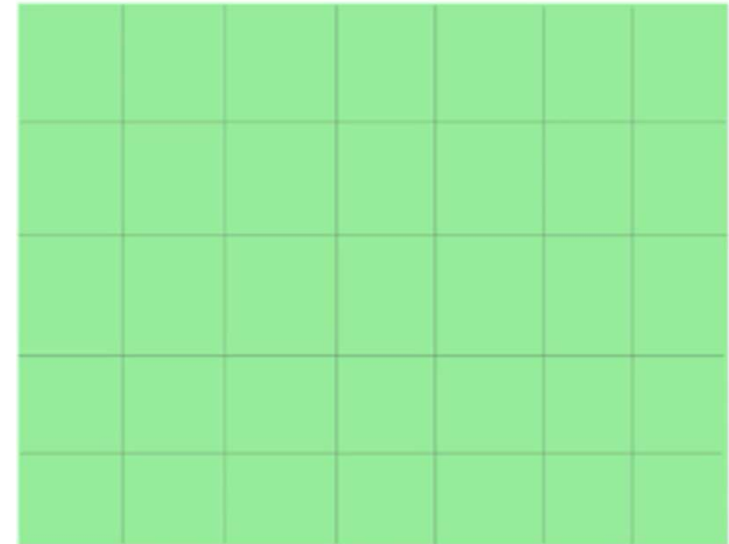
Algumas pessoas e empresas ainda utilizam a **linguagem C**, porém, isso logo vai acabar e todo mundo vai utilizar a **linguagem orientada a objeto**, que é muito mais fácil de utilizar, muito mais avançado e interessante.

VARIÁVEIS

É fundamental nós entendermos como é o funcionamento do computador em relação às variáveis.

Bom, para entendermos variáveis, temos que imaginar um armário, onde tem várias gavetas, ou seja, vários lugares para que você armazene alguma coisa.

Então, imagine que cada gaveta dessa vai armazenar um tipo de dado diferente.



Como assim?

O nosso computador tem memória, afinal de contas ele guarda informações, se não tiver memória, não é possível nem sequer você executar um programa, porque precisa da memória volátil.

E da mesma forma, para escrever algoritmos, nós precisaremos utilizar dessa memória do computador para armazenar algumas informações, que nós chamamos de variáveis.

Então, existem vários tipos de variáveis.

Para exemplificar, vamos imaginar assim:

Nesse primeiro quadrado, eu posso guardar apenas os alimentos.

Então, eu não posso guardar objetos ou outras coisas, apenas alimentos.

Lá no último quadrado, eu posso guardar apenas objetos, ou seja, apenas esse tipo de informação que é objetos, entre outros.

Então, se eu quero guardar alimentos, naquele primeiro quadrado, vamos guardar uma informação do tipo alimento, que será frutas.

Frutas é um alimento, então eu posso guardar o que mais?

Verduras. As verduras também são um alimento que eu posso guardar, ou seja, se eu tentar armazenar um objeto nesse quadrado que só armazena alimentos, então nós teremos um erro lá no nosso programa.

Da mesma forma, objetos, eu vou guardar um guarda-chuva, eu vou guardar um sofá. Cada tipo de informação precisa guardar apenas aquele tipo de informação.

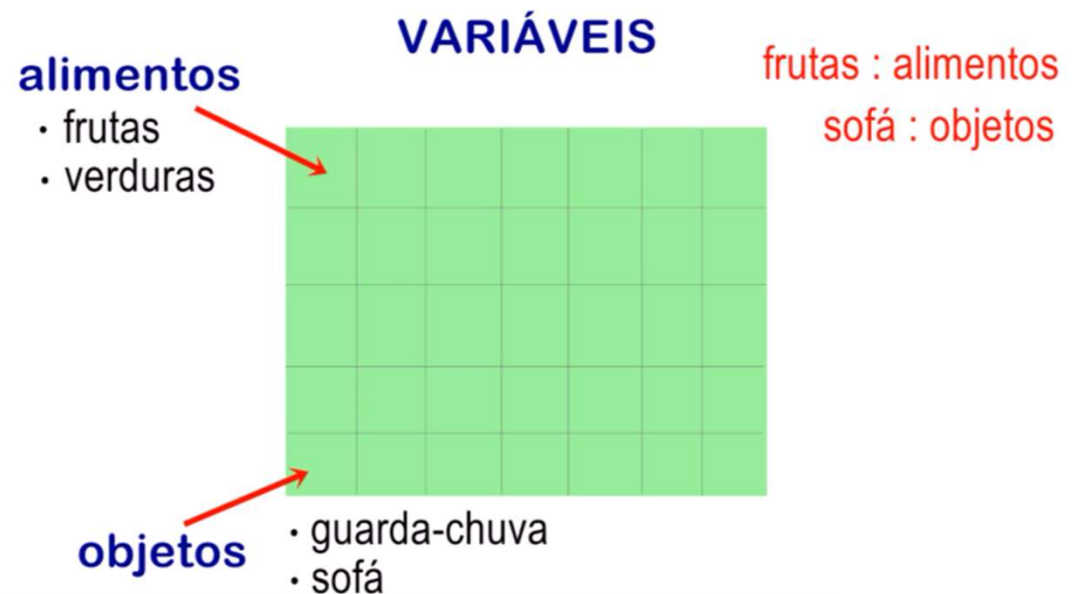
Como nós declaramos esses tipos de informações?

Vamos supor que esses alimentos, a palavra **alimentos** é o meu **tipo**, e **frutas** e **verduras** é o **nome do tipo** que eu estou dando.

Para declarar essa informação, o que eu vou falar?

Eu vou dar um nome, no meu caso eu dei **frutas**, dois **pontos**, e eu coloco o tipo de informação, que é **alimento**.

Então, por exemplo, **sofá**, dois **pontos**, **objetos**, ou seja, **sofá** é do **tipo objeto**, **fruta** é do **tipo alimento**.



Vamos ver agora quais são os tipos que nós temos em lógica de programação.
Nós temos o tipo:

- ✓ inteiro,
- ✓ Real
- ✓ Carácter
- ✓ lógico.

Quando você iniciar uma linguagem específica de programação, verá que existem mais tipos.

Por isso que o **Visualg**, **Portugol**, os algoritmos, quando nós aprendemos, temos que ter domínio sobre esses quatro, porque esses quatro são a base. Depois virão outros, mas que são baseados nesses quatro, que são de tamanhos menores ou maiores, você entenderá quando estiver estudando uma determinada **linguagem de programação**.

Os **inteiros**, como na matemática nós já aprendemos, eles são aqueles valores sem vírgula.

Por exemplo, o número **35**, o número **-1**, também é um **inteiro**, ou seja, um **inteiro negativo**, o número **0** também, tudo aquilo que não tem vírgula, aquilo que é **inteiro**, literalmente.

Os números **reais**, eles são aqueles números com vírgula.

12,566, **0,1** ou **-7,215**, tudo aquilo que envolve vírgula.

E você pode também considerar o número **inteiro** como **real**, afinal, na matemática, por exemplo, aquele número **35**, ele tem uma vírgula, ele está escondido, essa vírgula é depois do 5, sendo escrito assim **35,0**.

Mas, se você for realmente usar, por exemplo, valores de moeda **Reais**, de **Dólar**, que envolvem vírgula, então, pode utilizar **real**, que não vai ter problema durante a compilação do seu programa.

Nós também temos o tipo **carácter**, que guarda palavras, e sempre que for utilizar o tipo **carácter**, sempre colocar essa palavra em **aspas**. Se você colocar números entre **aspas**, ele não vai ser considerado como **inteiro** ou **real**, ele será considerado como **carácter**.

Veremos que teremos como converter esse número que está armazenado em **carácter** para **inteiro** ou para **real** também.

E o valor **lógico**, que vai receber **verdadeiro** ou **falso**.

Como declarar essas variáveis?

Já vai se acostumando com essas palavras, **declarar**, **atribuir**, que vai ser utilizado muito.

Declarar é definir uma variável que eu vou utilizar.

nome : caractere

Por exemplo, eu quero guardar o nome de uma pessoa, vou chamar essa variável de **nome**, eu coloco **dois pontos** e o **tipo**.

idade : inteiro

Nome são letras, palavras, então, colocarei o tipo **carácter**.

dolar : real

E outra variável, do tipo **inteiro**, é para armazenar a idade, que eu chamei de **idade**.

tem : logico

Dólar, do tipo **real**, afinal, tem vírgulas.

E uma variável do tipo **lógico**, chamado **tem**. Ela vai me dizer se **tem** ou **não**, **verdadeiro** ou **falso**.

O que você não pode fazer quando declarar uma variável?

Por exemplo, coloquei aqui 2nomes. Uma variável sempre começará com **carácter**, não **números**, essa forma está errada.

~~2nome : caractere~~

Outro erro é você colocar **espaço**. Se eu quero escrever, dar o nome da minha variável de **nome completo** e eu dou espaço ela estará errada.

~~nome completo : caractere~~

Variáveis sempre são palavras grudadas, então, mesmo que eu quisesse escrever **nome completo**, ali o completo, o **C**, começará com letra maiúscula para a gente poder separar isso. Então, essa forma também está errada. Aqui seria a maneira correta, eu quero escrever duas palavras, nome completo.

✓ nomeCompleto : caractere

Aqui também está errado, porque variáveis não podem ter acentuação. Outra forma também, símbolos, eu sempre tenho que começar com carácter.

~~dólar : real~~

~~\$dolar : real~~

Essa forma é um jeito que algumas pessoas utilizam. É mais utilizado você colocar o C em maiúscula, mas muitas pessoas utilizam essa, mas não está errado.

nome_completo : caractere

Número1, você pode, ou seja, a partir da primeira palavra, então, utilizar números no meio das frases, essa forma é uma forma correta.

✓ numero1 : inteiro

E as palavras reservadas, por exemplo, **algoritmo**.

Por que eu não posso utilizar **algoritmo**?

Você verá que lá no programa já existe essa palavra, **algoritmo**, que é utilizada, então, é uma palavra reservada. Tudo que já estiver lá como palavra, que eu consiga ver, que eu saiba que tem, eu não posso utilizar.

~~algoritmo : caractere~~

Por exemplo, **carácter**, é uma palavra que é reservada, não posso chamar **carácter**, **dois pontos**, **carácter**. Ou seja, tenho que utilizar apenas palavras não existentes.

Atribuição significa você preencher uma variável. Quando nós declaramos, o nome do tipo **carácter**, no **Visualg**, elas são inicializadas vazias.

Por exemplo, **carácter** vem vazio, sem nenhuma palavra, **Idade** vem com zero. Então, nós temos que preencher essas variáveis para a gente poder utilizar.

Em uma **linguagem de programação**, normalmente, na maioria delas, assim que eu declaro, por exemplo, **nome** do tipo **carácter**, eles **não vêm vazio**. Vêm com **memória**, com **lixo de memória**, e aí precisamos **inicializar** elas. Por exemplo, **idade**, que é do tipo **inteiro**, vem com **lixo de memória**.

Inicializamos com zero, fazemos essa **atribuição**, colocando zero. Só que aqui no **Visualg**, não precisamos, porque ele já vem com zero.

Como fazer uma **atribuição**?

É como se fosse uma setinha. Você coloca símbolo de menor < e um tracinho -, e assim estará **atribuindo** aquele valor.

O **nome** é do tipo **carácter**.

Irei **atribuir** a palavra **Joana** para esse **nome**, para essa variável.

Idade, a mesma coisa, só que sem as **aspas**. As **aspas** são apenas para os **caracteres**.

Dólar recebe o **número com vírgula**.

E **tem** recebe **falso** ou **verdadeiro**.

atribuição

nome <- "joana"

idade <- 32

dolar <- 30,2654

tem <- falso

O que é errado fazer?

Atribuir um tipo errado. Exemplo: **nome** recebe **10**.

nome ← 10 ~~nome ← 10~~

Isso irá gerar um erro, e não queremos erro durante a compilação do nosso programa. Então, se você realmente quer adicionar o número **10** ao **nome** por alguma razão, você tem que colocar esse **número** entre **aspas**, para ser considerado como **caractere**.

ENTRADA E SAÍDA DE DADOS

Como é uma **saída de dados**?

Tenho um **nome** do tipo **carácter**, onde **atribuí Maria** a esse **nome**.

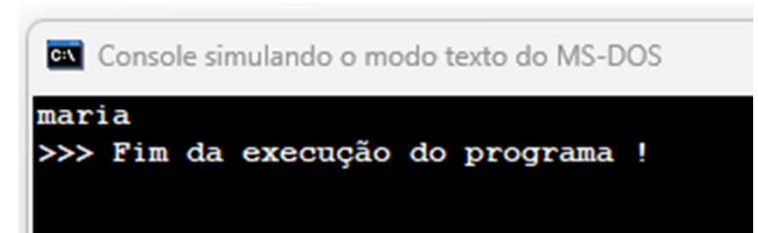
No **Visualg**, eu escrever a palavra, **escreva**, entre parênteses(), e colocar o nome dessa variável, que no caso é **nome**, e eu mandar executar o código, clicando no menu **Run(executar)**, ou pressionar **F9**, no teclado, o que acontecerá?

Aparecerá um console, uma Janela preta, e aparecerá a palavra **Maria**, que é o que está armazenado em **nome**.

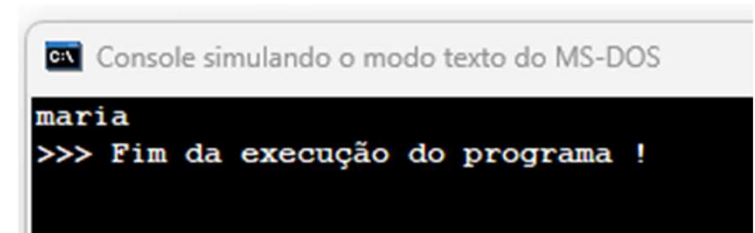
Então, não aparecerá **nome**, mas sim o que está armazenado em **nome**.

Posso colocar uma palavra qualquer entre **aspas**. temos duas formas de realizar uma saída. Você pode colocar variável ou colocar uma palavra.

```
1 Algoritmo "EscrevaNome"
2
3 Var
4 // Seção de Declarações d
5
6 nome:caractere
7
8 Inicio
9 // Seção de Comandos, pro
10
11 nome<-"maria"
12 escreva(nome)
13 Fimalgoritmo
```



```
1 Algoritmo "EscrevaNome"
2
3 Var
4 // Seção de Declarações d
5
6 nome:caractere
7
8 Inicio
9 // Seção de Comandos, pro
10
11 escreva("maria")
12 Fimalgoritmo
```



A mesma coisa para os outros tipos. Se eu tenho uma **idade** do tipo **inteiro**, onde eu atribui o valor **50**.

Então, escreva a **idade**, aparecerá o valor **50** na tela.

```
1 Algoritmo "EscrevaIdade"
2
3 Var
4 // Seção de Declarações da
5
6 idade: inteiro
7
8 Inicio
9 // Seção de Comandos, proce
10 idade <- 50
11 escreva(idade)
12 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

```
50
>>> Fim da execução do programa !
```

Ou posso também, escrever e colocar entre **aspas** o **número** que eu quero.

```
1 Algoritmo "EscrevaNumero"
2
3 Var
4 // Seção de Declarações das
5
6 Inicio
7 // Seção de Comandos, proce
8
9 escreva("123")
10 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

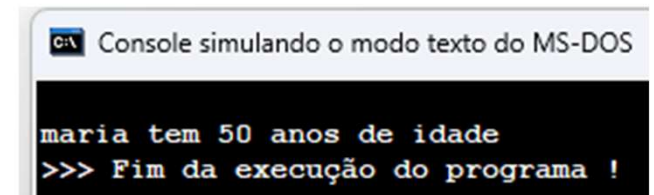
```
123
>>> Fim da execução do programa !
```

Como eu faço para escrever essa frase?

Maria tem 50 anos de idade. Ou seja, quero unir as duas variáveis em uma frase.

Escrevo abre parênteses, a variável **nome** vírgula - essa vírgula unirá todas as frases - vírgula **tem** entre **aspas** vírgula **idade** vírgula entre **aspas** **anos de idade** e fecha o parênteses.

```
1 Algoritmo "EscrevaFrase"
2
3 Var
4 // Seção de Declarações das variáveis
5
6 nome:caractere
7 idade:inteiro
8
9 Inicio
10 // Seção de Comandos, procedimento, funções,
11 nome<- "maria"
12 idade<- 50
13 Escreva(nome," tem",idade, " anos de idade")
14 Fimalgoritmo
```



Console simulando o modo texto do MS-DOS

```
maria tem 50 anos de idade
>>> Fim da execução do programa !
```

Algumas linguagens de programação, No Visual, também aceita o símbolo de + ao invés da vírgula, também vai fazer essa união de palavras.

Isso se chama **concatenação**. Quando você ouvir a palavra **concatene** ou **concatenar**, ou seja, é a mesma coisa que **juntar palavras**, perceba que **idade é do tipo inteiro**, e mesmo assim, ele se uniu a outras palavras do tipo **carácter**, porque ele é convertido automaticamente pelo visual, ele é considerado como **texto**.

ENTRADA E SAÍDA DE DADOS

Agora veremos a entrada de dados.

O que seria a entrada?

Seria pedir para uma pessoa digitar no teclado uma informação, que vai direto ser guardada dentro de uma variável.

Eu criei aqui, **nome** do tipo **carácter**, **idade** do tipo **inteiro**. E vou mostrar na tela. Escreva, irei mostrar na tela qual é o seu **nome**, então vai aparecer lá qual é o seu **nome**.

Na próxima linha escrevo, **leia**. A palavra **leia** é uma palavra **reservada**, onde vai guardar a informação que for digitar dentro da **variável** que eu colocar em **parênteses**, se eu coloco **leia, nome**, a palavra que a pessoa **digitar** e der Enter, essa **palavra vai ser guardada lá dentro**.

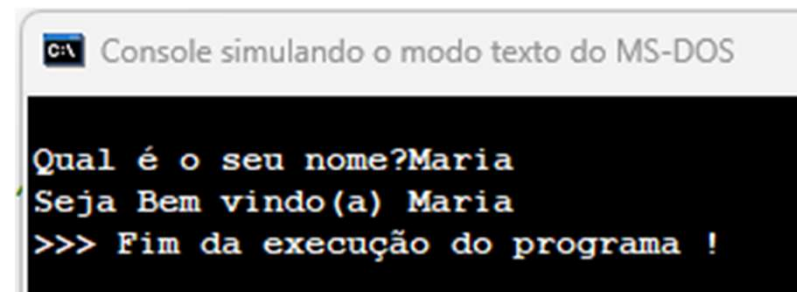
A seguir, posso digitar, **escreva**, seja bem-vindo, coloco uma **vírgula**, para se juntar com essa palavra que a pessoa digitou. Então, **seja bem-vindo, Maria**.

Isso é uma entrada de dados, a partir do momento que eu utilizo o **leia**.

Vamos para o Visualg, digite **escreva**, entre parênteses digite entre aspas duplas **"Seja bem-vindo(a)"**, coloque uma vírgula, para se juntar com essa palavra que a pessoa digitou: **Seja bem-vindo(a), Maria**. Isso seria uma entrada de dados, a partir do momento que eu utilizo o **leia**.

Onde temos **//** são comentários, eles servem para referenciar seu código, para que saiba o que está acontecendo, ele não interfere na programação.

```
1 Algoritmo "BemVindo"
2 // Descrição : Aqui você descreve o q
3
4 Var
5 // Seção de Declarações das variáveis
6
7 nome:caractere
8 idade:inteiro
9
10 Inicio
11 // Seção de Comandos, procedimento, fun
12
13 escreva("Qual é o seu nome?")
14 leia(nome)
15 escreva("Seja Bem vindo(a) ",nome)
16
17 Fimalgoritmo
```



C:\ Console simulando o modo texto do MS-DOS

```
Qual é o seu nome?Maria
Seja Bem vindo(a) Maria
>>> Fim da execução do programa !
```

OPERADORES

Veremos os operadores aritméticos, as **funções aritméticas**, a **ordem de precedência**, os **operadores relacionais** e os **operadores lógicos**.

Os **operadores aritméticos**, o nome até assusta, mas são aqueles **operadores matemáticos** que você conheceu no fundamental.

Vamos aprender como utiliza-los aqui.

Temos a **adição**, **subtração**, **multiplicação**, **divisão**, **divisão inteira**, **exponenciação**.

Os símbolos utilizados dentro do Visualg,

Operadores Aritméticos		
Adição	+	valor + valor
Subtração	-	valor – valor
Divisão	/	valor / valor
Multiplicação	*	valor * valor

A **divisão inteira**, nós colocamos uma barra invertida,

Operadores Aritméticos		
Divisão Inteira	\	valor \ valor

O que seria **divisão inteira**?

Quando realizamos a divisão, um número por outro, ele pode gerar um número com vírgula. Caso utilize a barra invertida, esse número com vírgula será desconsiderado, a parte da vírgula, ele retornará apenas a parte inteira. Pode ser que precise, em algum momento, em seu código.

Lembre-se, sempre quando você for realizar uma **divisão**, você precisa, **necessariamente**, utilizar uma variável do tipo **real**. Você não consegue **dividir** uma variável do tipo **inteira**, **duas** variáveis do tipo **inteira**, porque pode dar **erro** ou trazer um **resultado errado**. Ele pode pegar apenas o resultado inteiro e trazer para você como resposta. Então, fique de olho nisso.

A **potenciação** ou **exponenciação** é a operação matemática que representa a **multiplicação** de **fatores iguais**. Ou seja, usamos a **potenciação** quando um número é multiplicado por ele mesmo várias vezes.

Operadores Aritméticos		
Exponenciação	^	valor ^ valor

E o modulo retorna o resto da divisão inteira, do operando à esquerda pelo operando à direita.

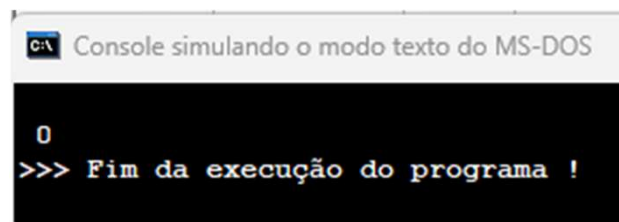
Operadores Aritméticos		
Modulo	%	valor % valor

Por exemplo, $10 / 2 = 5$. 5 é o número inteiro, então tem resto 0. Ele vai comparar o resto dessa divisão apenas. Então, $10 / 2$ gera um resto 0.

```

1 Algoritmo "Operadores%"
2 // Descrição : Aqui vc
3
4 Var
5 // Seção de Declarações
6
7 n1, n2, n3 : real
8 valor : inteiro
9
10 Inicio
11 // Seção de Comandos, pr
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 valor <- n1%2
17 Escreva(valor)
18 Fimalgoritmo

```



FUNÇÕES ARITMÉTICAS

Existem algumas funções que já vêm no Visualg, são elas: para **valor absoluto**, o **outro tipo de exponenciação**, o **valor inteiro**, **raiz quadrada**, π , **seno**, **cosseno**, **tangente** e **graus para radiano**.

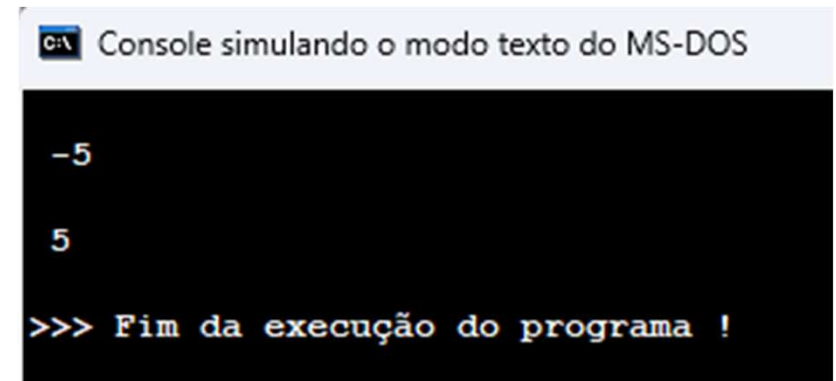
Ao Logo do curso nós veremos como fazer as nossas próprias funções.

Funções aritméticas já existentes

Valor absoluto	Abs	Abs(-10)	10
Exponenciação	Exp	Exp(3,2)	9
Valor inteiro	Int	Int(3.9)	3
Raiz quadrada	RaizQ	RaizQ(25)	5
π	PI	PI	3.14....
Seno	Sen	Sen(0.523)	0.5
Cosseno	Cos	Cos(0.523)	0.86
Tangente	Tan	Tan(0.523)	0.57
Graus para Radiano	GraupRad	GraupRad(30)	0.52

Veja agora alguns exemplos no Visualg, Valor Absoluto:

```
1 Algoritmo "valorabsoluto"
2 // Descrição : Aqui você descreve o que o prog
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor: inteiro
8
9 Inicio
10 // Seção de Comandos, procedimento, funções, ope
11
12     valor <- -5
13     escreval(valor)
14 //Aqui ele escreve o valor -5
15     escreval()
16     valor <- abs(valor)
17     escreval(valor)
18 //Aqui ele recebeu o valor absoluto escreve 5
19 Fimalgoritmo
```



C:\ Console simulando o modo texto do MS-DOS

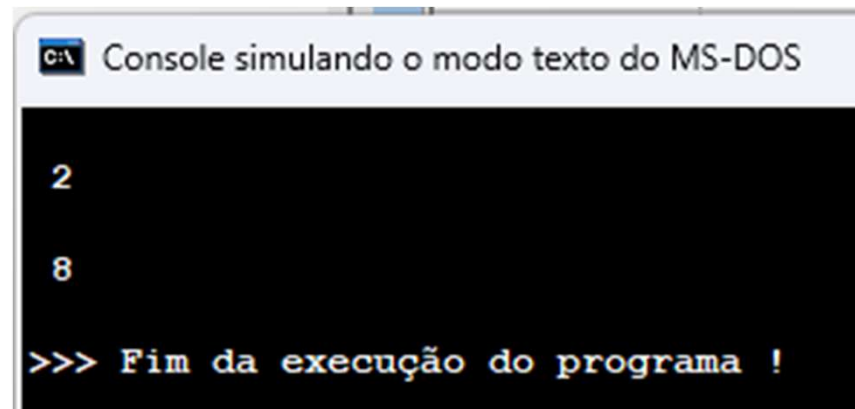
-5

5

>>> Fim da execução do programa !

Veja agora alguns exemplos no Visualg, Exponenciação:

```
1 Algoritmo "Exponenciação"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor: real
8
9 Inicio
10 // Seção de Comandos, procedimentos
11
12     valor <- 2
13     escreval(valor)
14 //Aqui ele escreve o valor 2
15     escreval()
16     valor <- exp(valor,3)
17     escreval(valor)
18 //Aqui ele recebeu a
19 //Exponenciação exibe 8
20 Fimalgoritmo
```



```
C:\> Console simulando o modo texto do MS-DOS

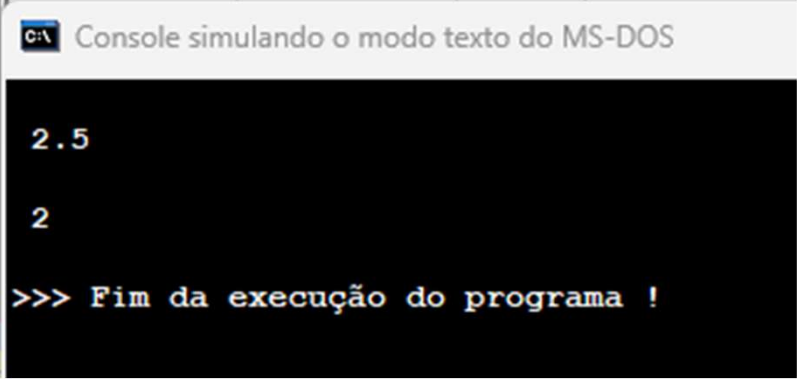
2

8

>>> Fim da execução do programa !
```

Veja agora alguns exemplos no Visualg, Numero Inteiro:

```
1 Algoritmo "Inteiro"
2 // Descrição : Aqui você descreve o
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor: real
8
9 Inicio
10 // Seção de Comandos, procedimento, fu
11
12 valor <- 2.5
13 escreval(valor)
14 //Aqui ele escreve o valor 2.5
15 escreval()
16 valor <- int(valor)
17 escreval(valor)
18 //Aqui ele escreve valor inteiro
19
20 Fimalgoritmo
```



```
C:\ Console simulando o modo texto do MS-DOS

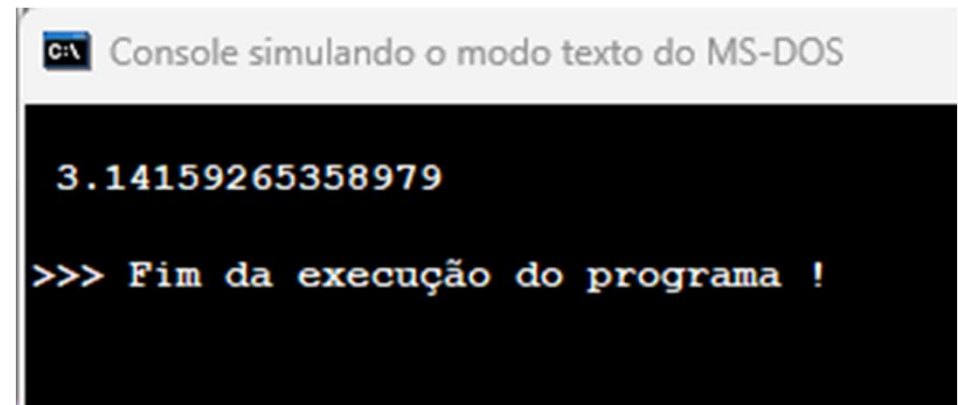
2.5

2

>>> Fim da execução do programa !
```

Veja agora alguns exemplos no Visualg, Valor de π (PI):

```
1 Algoritmo "PI"
2 // Descrição : Aqui você descreve c
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor:real
8
9 Inicio
10 // Seção de Comandos, procedimento, i
11
12     valor <- pi
13     escreval(valor)
14 //Aqui ele escreve
15 //o valor de pi
16 //3.14159265358979
17
18 Fimalgoritmo
```



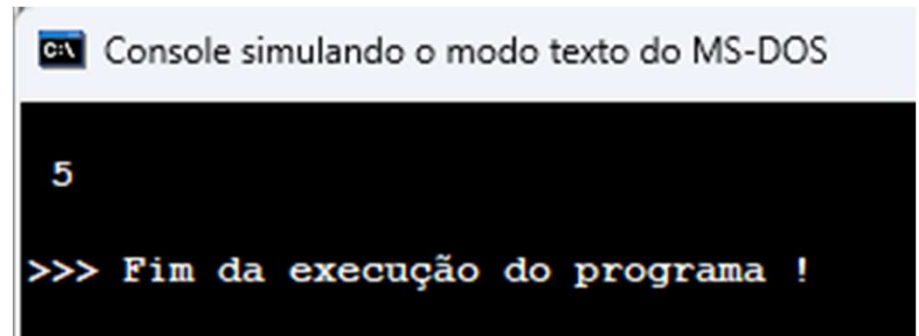
C:\> Console simulando o modo texto do MS-DOS

3.14159265358979

>>> Fim da execução do programa !

Veja agora alguns exemplos no Visualg, Raiz Quadrada:

```
1 Algoritmo "RaizQuadrada"  
2 // Descrição : Aqui você descrev  
3  
4 Var  
5 // Seção de Declarações das variáv  
6  
7 valor:real  
8  
9 Inicio  
10 // Seção de Comandos, procedimento  
11  
12     valor <- raizQ(25)  
13     escreval(valor)  
14 //Aqui ele escreve o valor  
15 //da Raiz Quadrada de 25  
16 // escreve 5  
17  
18 Fimalgoritmo
```



```
C:\> Console simulando o modo texto do MS-DOS  
  
5  
  
>>> Fim da execução do programa !
```

ORDEM DE PRECEDÊNCIA

O que seria a ordem de precedência?

Repare nessa tabela, temos no topo os parênteses, em seguida vem a exponenciação, depois a multiplicação e divisão, e por último, a adição e subtração.

Ordem de Precedência
()
$\wedge$
$*$ /
$+$ -

O que significa isso?

Se tenho uma operação grande, por exemplo, $2 + 5 \times 10$, multiplicado por...

Qual será a primeira coisa que será feita?

Isso é regra de matemática, que você já deveria saber. Primeiro, o que é feito são os parênteses, mesmo que seja uma adição, que é a última **ordem de precedência**, será realizado primeiro aquilo que está dentro do **parênteses**. A seguir, ou se não tiver outros **parênteses**, o que será realizado será a **exponenciação**. Depois, a **multiplicação e divisão**, que têm a mesma **ordem de precedência**.

E, por último, a **adição e multiplicação**. Então, veja um exemplo de **erro** que pode ser gerado caso não prestemos atenção nisso.

Iremos fazer esse valor, receber o $2 + 5 / 2$. Ele responderá $2 + 2.5 = 4.5$

O que eu quero que seja feito? $(5 + 2) = 7$ dividido por $2 = 3,5$.

Isso tem que me gerar um valor 3,5. Isso porque eu não prestei atenção, na **ordem de precedência**.

Vamos mostrar na tela e ver o resultado.

Demonstraremos na tela esse valor. Olha gerou 4,5.

Eu não esperava isso, né?

Então, o que acontece?

Você tem que **prestar atenção na ordem de precedência**.

```
1 Algoritmo "ErroPrecedencia"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor:real
8
9 Inicio
10 // Seção de Comandos, procedimentos
11
12 valor <- 2+5/2
13 escreval(valor)
14 //Aqui ele vai fazer 5/2
15 // depois 2+2.5 = 4.5 ERRADO
16
17 //A equação correta seria
18 //(2+5)/2 = 7/2 = 3.5
19
20 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

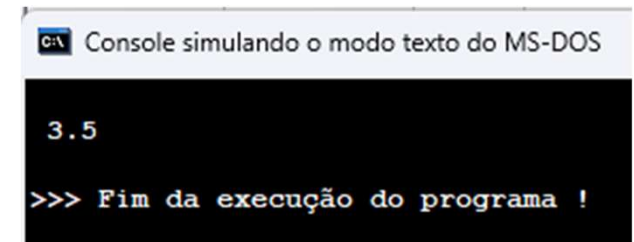
4.5

>>> Fim da execução do programa !

Agora veja na forma correta:
 $(5 + 2) = 7$ dividido por 2 = 3,5.

Isso tem que me gerar um
valor 3,5. Agora esta na **ordem
de precedência** correta.

```
1 Algoritmo "ErroPrecedencia"
2 // Descrição : Aqui você descre
3
4 Var
5 // Seção de Declarações das varia
6
7 valor:real
8
9 Inicio
10 // Seção de Comandos, procediment
11
12     valor <- (2+5)/2
13     escreval(valor)
14
15 //A equação correta seria
16 //(2+5)/2 = 7/2 = 3.5
17
18 Fimalgoritmo
```



Console simulando o modo texto do MS-DOS

3.5

>>> Fim da execução do programa !

Mas se eu colocar aqui $2 * 4 / 2$?
Eles têm a mesma ordem de precedência.

Tanto $2 * 4 / 2$, quanto $4 / 2 * 2$, tanto faz a ordem, sempre me trará o mesmo resultado pela questão matemática.

Mas e se eu colocar uma adição aqui depois desse $*$ 2? Então, vou colocar uma soma aqui, um valor 2.

Eu tenho que obrigar esse valor, se eu quiser que ele seja realizado primeiro, colocar ele em parênteses.

Ficando $2 * 4 / (2 + 2)$.

```
1 Algoritmo "Precedencia"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor:real
8
9 Inicio
10 // Seção de Comandos, procedimento,
11
12     valor <- 2 * 4 / 2
13     escreval(valor)
14
15 //Tanto faz a ordem,
16 //sempre me trará o mesmo
17 //resultado pela questão matemática
18
19 Fimalgoritmo
```

C:\> Console simulando o modo texto do MS-DOS

4

>>> Fim da execução do programa !

```
12     valor <- 2 * 4 / (2 + 2)
13     escreval(valor)
14
15 //Para obrigar a soma primeiro
16 //Tenho que colocar entre ( )
17
18 Fimalgoritmo
```

C:\> Console simulando o modo texto do MS-DOS

2

>>> Fim da execução do programa !

OPERADORES RELACIONAIS

Esses **operadores** são de **comparação**, posso comparar um número com outro, posso comparar uma variável com outra variável, ou uma variável com um número.

Por exemplo, temos aqui o símbolo de maior. Posso comparar **5 > 2, verdadeiro ou falso.**

É por isso que eles são de comparação, eles são do tipo **lógico**, onde já vimos que tem a variável do tipo **lógico** que traz um **verdadeiro** ou **falso**.

Aqui temos o símbolo **maior**, que é a seta para a direita, **menor**, seta para a esquerda, **maior ou igual**, ou seja, esse número é maior ou é igual a um outro número, **menor ou igual**, temos o **igual**, e o **diferente**.

Esses operadores serão muito utilizados, nas **estruturas condicionais**, nas **estruturas de repetição** entre outras. Precisa ter um domínio sobre eles e a sua utilização.

Ordem de Precedência	
>	Maior
<	Menor
>=	Maior ou igual
=	Menor ou igual
<>	Diferente

Vamos a um exemplo, eu tenho as variáveis **n1**, **n2** e **n3**.

Elas serão do tipo **inteiro**. Iremos criar uma variável **vf** do tipo lógico, **verdadeiro** ou **falso**. Agora adicionar o valor **8** para **n1**, o valor **5** para **n2** e o valor **5** também para **n3**. Precisamos fazer com que **vf** receba **verdadeiro** ou **falso** em uma condição.

Agora iremos perguntar, **n1** é **menor** que **n2**? E vamos mostrar isso na tela. Então, **n1** é 8, **n2** é 5. Então, isso tem que me trazer falso. Vamos **concatenar** com **vf**, que é o resultado, escrevendo: Isso é falso. Isso é maior? Então, 8 realmente é maior que 5. Então, isso é verdadeiro. E **n1 = n2**, **8 = 5** é Falso.

```
1 Algoritmo "OperadoresRelacionais"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimentos
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 vf <- n1 < n2
17 Escreva("Isso é", vf)
18 Fimalgoritmo
```

CA Console simulando o modo texto do MS-DOS

```
Isso é VERDADEIRO
>>> Fim da execução do programa !
```

```
1 Algoritmo "OperadoresRelacionais"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimentos
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 vf <- n1 = n2
17 Escreva("Isso é", vf)
18 Fimalgoritmo
```

CA Console simulando o modo texto do MS-DOS

```
Isso é FALSO
>>> Fim da execução do programa !
```

Vamos ver o funcionamento do operador diferente, vou perguntar, **n1 é diferente de n2?** n1 é 8, n2 é 5. Eles são diferentes, então isso é verdadeiro. Depois perguntaremos se ele é maior ou igual. **n1 é maior** do que **n2**, 8 é maior do que 5, ele se encaixa na primeira questão, que é maior.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você desc
3
4 Var
5 // Seção de Declarações das var
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedime
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 vf <- n1 <> n2
17 Escreva("Isso é", vf)
18 Fimalgoritmo
```

CA Console simulando o modo texto do MS-DOS

```
Isso é VERDADEIRO
>>> Fim da execução do programa !
```

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você de
3
4 Var
5 // Seção de Declarações das v
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedi
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 vf <- n1 >= n2
17 Escreva("Isso é", vf)
18 Fimalgoritmo
```

CA Console simulando o modo texto do MS-DOS

```
Isso é VERDADEIRO
>>> Fim da execução do programa !
```

OPERADORES LÓGICOS

Os **operadores lógicos** são utilizados exclusivamente para a realização de operações entre valores do tipo **lógico**, ou seja, **verdadeiro** ou **falso**.

Muitas vezes é necessário verificar se mais de uma expressão relacional é **verdadeira** para dar prosseguimento ao código.

É nesse ponto que os **operadores lógicos** entram em ação.

São quatro os tipos de **operadores lógicos**, denominados por:

1. **E (AND);**
2. **OU (OR);**
3. **XOU (XOR) e**
4. **NÃO (NOT).**

Iniciando pelo operador **E**, este é utilizado para verificar se um conjunto de valores do tipo lógico são todos **verdadeiros** ou não. Para tanto, é utilizada a seguinte tabela verdade, tendo como base a expressão “**a E b**”, que representa os resultados obtidos.

A E B		
A	B	RESULTADO
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	FALSO
FALSO	VERDADEIRO	FALSO
FALSO	FALSO	FALSO

Para que você consiga ler esta tabela, inicialmente, é preciso compreender que está utilizando como base a expressão “**a E b**”, ou seja, quer verificar se o valor correspondente à variável “**a**” e o valor correspondente à variável “**b**” são **verdadeiros**, caso contrário, deverá retornar **falso**.

Pela tabela, é possível observar que apenas caso os dois valores forem **verdadeiros** que o resultado será **verdadeiro**. Toda expressão que utilize o operador **E** segue essa tabela, ou seja, apenas será considerado **verdadeiro** se todos forem **verdadeiros**.

O **operador lógico OU** tem como objetivo verificar se **pelo menos uma das expressões é verdadeira**, ou seja, é preciso que **pelo menos a primeira OU a segunda expressão seja verdade**. Mantendo a expressão anterior como base, apenas alterando o **operador lógico**, você pode analisar a tabela verdade do operador **OU**.

A OU B		
A	B	RESULTADO
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

Diferentemente do operador **E**, no caso do operador **OU**, você possui apenas uma possibilidade de a expressão retornar **falsa**, que consiste em **quando todas as operações forem falsas**. Do contrário, o resultado será **verdadeiro**, por ser **preciso pelo menos uma das operações retornando verdade**.

Outro **operador lógico** existente é o operador **XOU**, também conhecido como **OU EXCLUSIVO**. Para esse caso, apenas uma das expressões pode ser **verdadeira**, de forma que ou a primeira ou a segunda expressão deve ser verdade, **caso ambas sejam falsas ou verdadeiras** o resultado é considerado **falso**. A seguir, é apresentada a tabela verdade para o operador **XOU**.

A XOU B		
A	B	RESULTADO
VERDADEIRO	VERDADEIRO	FALSO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

Por fim, existe o operador denominado **NÃO**, também conhecido como **negação**. Esse é o mais simples dos **operadores lógicos**, pois tudo o que faz é **negar um valor lógico**. Ou seja, ao **negar um valor verdadeiro**, você está dizendo que este **valor é falso**. Assim como, ao **negar um valor falso**, entende-se que esse **valor é verdadeiro**.

Pode-se compreender melhor esse operador ao identificar que ele inverte um valor lógico, conforme os exemplos a seguir.

```
a <- nao verdadeiro  
b <- nao falso
```

Nesses casos, as variáveis **a** e **b** são do tipo **lógico**, e no caso da variável **a**, está sendo atribuída a **negação do valor verdadeiro**, ou seja, essa variável receberá o **valor falso**. Referente à variável **b**, esta recebe a **negação do valor falso**, ou seja, o **valor verdadeiro**.

Assim como os operadores aritméticos, os operadores lógicos também possuem uma ordem de prioridade, que consiste no seguinte:

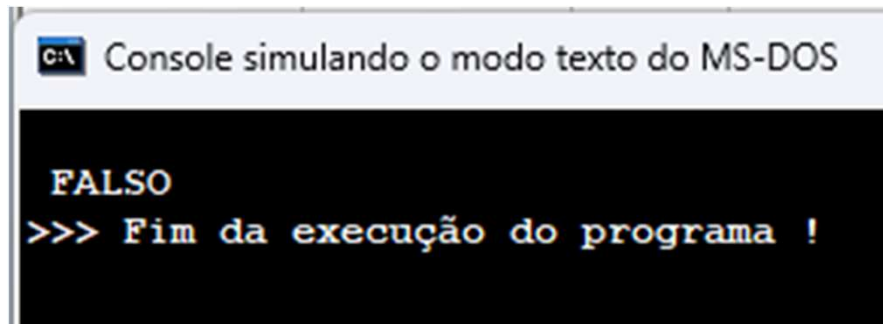
1. NÃO;
2. E;
3. OU.

Lembrando que, caso existam expressões entre parênteses, estas devem ser resolvidas primeiramente, e, caso existam apenas expressões de um único tipo, vamos supor, operador lógico **E**, deve-se iniciar a resolução da esquerda para a direita.

Vamos ver um exemplo agora, no Visualg. Vou digitar aqui, escreva falso e falso. O que dá Falso com Falso?

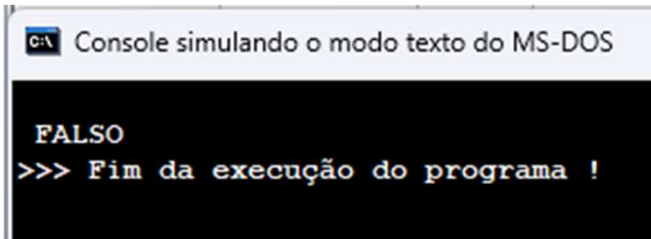
Eu **não tenho o bolo(Falso)** e **não tenho o refrigerante(Falso)**, então, eu **não levo**. Então, **Falso com Falso** é igual a **Falso**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui voce
3
4 Var
5 // Seção de Declarações de
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, proc
12
13 Escreva(falso e falso)
14 Fimalgoritmo
```



Verdadeiro e Falso, eu sou aquela pessoa fresca, eu quero os dois. Se **eu tenho o bolo(Verdadeiro)**, mas **não tenho o refrigerante(Falso)**, eu também **não vou levar(Falso)**. Isso vai me gerar **Falso**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você desc.
3
4 Var
5 // Seção de Declarações das var.
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedime.
12
13 Escreva(verdadeiro e falso)
14 Fimalgoritmo
```



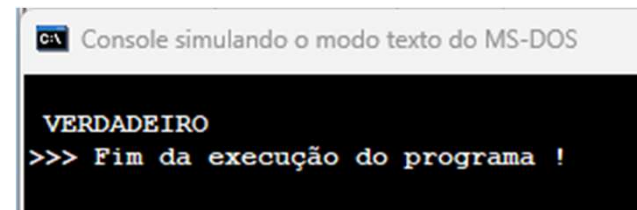
C:\ Console simulando o modo texto do MS-DOS

FALSO

>>> Fim da execução do programa !

Agora, se eu colocar **Verdadeiro** e **Verdadeiro**, ou seja, eu tenho o bolo(**Verdadeiro**) e tenho o refrigerante(**Verdadeiro**), então eu levo **verdadeiro**. Agora eu vou levar(**Verdadeiro**). Isso vai me gerar um **Verdadeiro**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 Escreva(verdadeiro e verdadeiro)
14 Fimalgoritmo
```

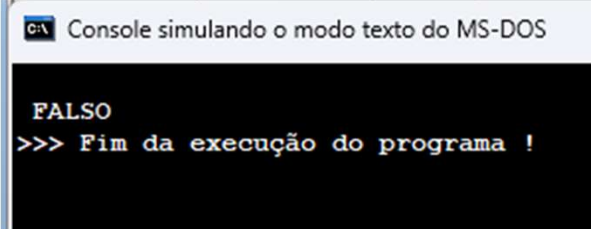


Console simulando o modo texto do MS-DOS

VERDADEIRO
>>> Fim da execução do programa !

Vamos ver a utilização do **OU Falso ou Falso**. Eu não tenho nenhum dos dois, então, **eu não vou levar**. Isso também vai me gerar um **Falso**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você desc
3
4 Var
5 // Seção de Declarações das var
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedime
12
13 Escreva(falso ou falso)
14 Fimalgoritmo
```



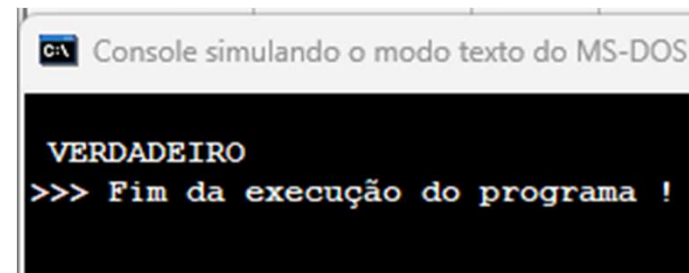
C:\ Console simulando o modo texto do MS-DOS

FALSO

>>> Fim da execução do programa !

Verdadeiro ou Verdadeiro. Então, eu tenho as duas coisas e eu vou levar também. Isso também vai me gerar um **Verdadeiro**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis:
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 Escreva(verdadeiro ou verdadeiro)
14 Fimalgoritmo
```

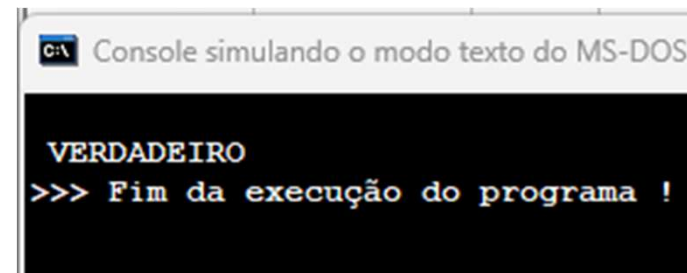


C:\ Console simulando o modo texto do MS-DOS

VERDADEIRO
>>> Fim da execução do programa !

Verdadeiro ou Falso. OU é uma coisa ou outra, eu posso levar também um dos dois. Então, se eu só tenho o bolo, eu vou levar o bolo. Então, isso é Verdadeiro.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimentos
12
13 Escreva(verdadeiro ou Falso)
14 Fimalgoritmo
```

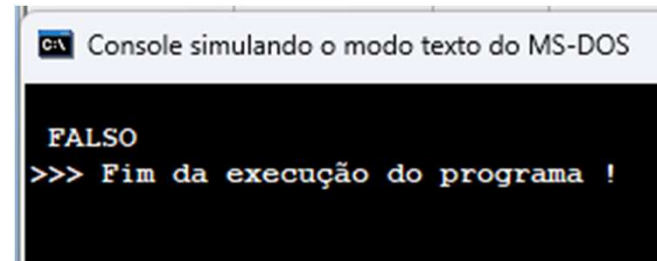


C:\ Console simulando o modo texto do MS-DOS

VERDADEIRO
>>> Fim da execução do programa !

Agora, vamos **negar** tudo isso. **Coloco em parênteses não**, ser negado. Se era **Verdadeiro**, agora é **Falso**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 Escreva(na|(verdadeiro ou Falso))
14 Fimalgoritmo
```



Console simulando o modo texto do MS-DOS

FALSO
>>> Fim da execução do programa !

Vou complicar um pouco agora. Vou fazer o **N1** receber **10**, **N2** receber **5**, e **N3** receber **75**. Escreva o seguinte. **N1** é maior do que **N2**? Então, isso é uma **Verdade**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você
3
4 Var
5 // Seção de Declarações da
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, proc
12
13 n1 <- 10
14 n2 <- 5
15 n3 <- 75
16
17 Escreva(n1 > n2)
18 Fimalgoritmo
```

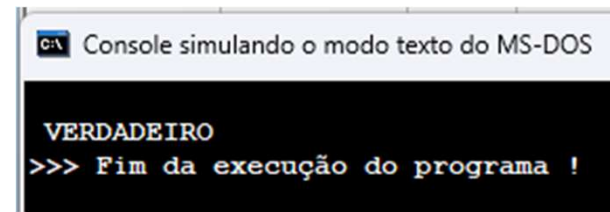
C:\ Console simulando o modo texto do MS-DOS

```
VERDADEIRO
>>> Fim da execução do programa !
```

Vamos colocar tudo isso em parênteses e vou fazer uma comparação do tipo **E**. E eu coloco o **Verdadeiro**. Perceba que estou fazendo a mesma coisa que antes.

Estou perguntando se **Verdadeiro** e **Verdadeiro** vai dar o quê? Se **tenho o bolo(Verdadeiro)** e **tenho o refrigerante(Verdadeiro)**, as duas coisas são **Verdades**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis.
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 n1 <- 10
14 n2 <- 5
15 n3 <- 75
16
17 Escreva((n1 > n2) e verdadeiro)
18 //Verdadeiro e Verdadeiro
19 Fimalgoritmo
```



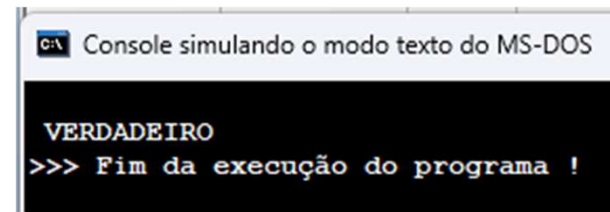
```
C:\> Console simulando o modo texto do MS-DOS

VERDADEIRO
>>> Fim da execução do programa !
```

Vamos colocar tudo isso em parênteses e vou fazer uma comparação do tipo **E**. E eu coloco o **Verdadeiro**. Perceba que estou fazendo a mesma coisa que antes.

Estou perguntando se **Verdadeiro** e **Verdadeiro** vai dar o quê? Se **tenho o bolo(Verdadeiro)** e **tenho o refrigerante(Verdadeiro)**, as duas coisas são **Verdades**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis.
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 n1 <- 10
14 n2 <- 5
15 n3 <- 75
16
17 Escreva((n1 > n2) e verdadeiro)
18 //Verdadeiro e Verdadeiro
19 Fimalgoritmo
```



ESTRUTURAS DE CONTROLE OU ESTRUTURAS DE DECISÃO

Conhecidos como **SE** ou o famoso **IF**.

Para que servem as estruturas de controle?

Estas estruturas são fundamentais, porque praticamente todo sistema que você vai desenvolver, irá utilizar elas.

É uma estrutura que você vai definir o que o seu programa vai fazer caso uma coisa seja verdade, caso uma coisa não corresponda àquilo que você quer.

Entenderemos isso melhor com um exemplo.

Vamos supor aqui:

**SE EU RECEBER HOJE ENTÃO
EU VOU AO MERCADO**

É uma decisão que eu quero tomar. Estou dizendo, **se eu receber hoje, se isso for uma verdade, se isso acontecer, então eu vou ao mercado**

SENÃO

VOU FICAR EM CASA

FIMSE

Caso contrário, ou seja, **se não, se eu não receber**, então eu vou ficar em casa. Veja que tenho duas opções onde o meu sistema pode realizar.

Por exemplo, **se** eu tenho um número que a pessoa digitou lá no teclado, e esse número for maior que 10, eu imprimo na tela: Seja bem-vindo. **Se não**, quando eu insiro **se não**, eu estou dizendo que qualquer outro número que não seja o 10, for enviado, **então** ele não vai entrar no sistema, não vai, por exemplo, imprimir uma mensagem de Seja bem-vindo.

Essa é a lógica, que é utilizaremos. Você precisa utilizar duas palavras. **Então**, é nessa linha aqui, **se eu receber hoje**, você vai escrever **então**, aí eu faço alguma coisa. E lá no final, você coloca um **fimse**, você está delimitando, que é aí que acaba a sua estrutura.

Outro exemplo, vamos imaginar uma situação fictícia, passará um filme de terror no cinema e a pessoa só pode entrar se ela tiver **mais de 18 anos ou 18 anos**. Isso é uma **condição** também, **então se a pessoa for maior ou igual a 18 anos, tiver mais de 18 anos, ela pode entrar para assistir esse filme.**

SE PESSOA $\geq$ 18 ANOS ENTÃO
PODE ENTRAR
SENÃO
ENTRADA PROIBIDA
FIMSE

Caso contrário, **ela está proibida**. Entenda que quando falo **se não**, eu digo que todas as outras opções que não sejam igual a essa, **não poderá realizar**, entrar nessa parte de cima do **se**, ele vai direto para o **se não**.

Perceba também que estou colocando de forma bem organizada, linha embaixo de linha, então o **se**, veja que ele corresponde ao **se não**, e for colocar o **fimse**, ele também vai seguir a mesma linha.

E sempre que eu colocar alguma coisa dentro dessa estrutura, eu dou um **tab**, um espaço, para ficar visível.

Isso você vai utilizar sempre, se chama **indentação**. Um bom programador sabe utilizar a **indentação** de forma que o sistema fique bem estruturado, não fica desorganizado.

Temos duas estruturas, a **simples** e a **composta**. A **simples**, ela não contém aquela palavra **se não**.

Então, o que acontece?

Se a pessoa for maior ou igual a 18, pode entrar. E o programa imprime **Pode Entrar**.

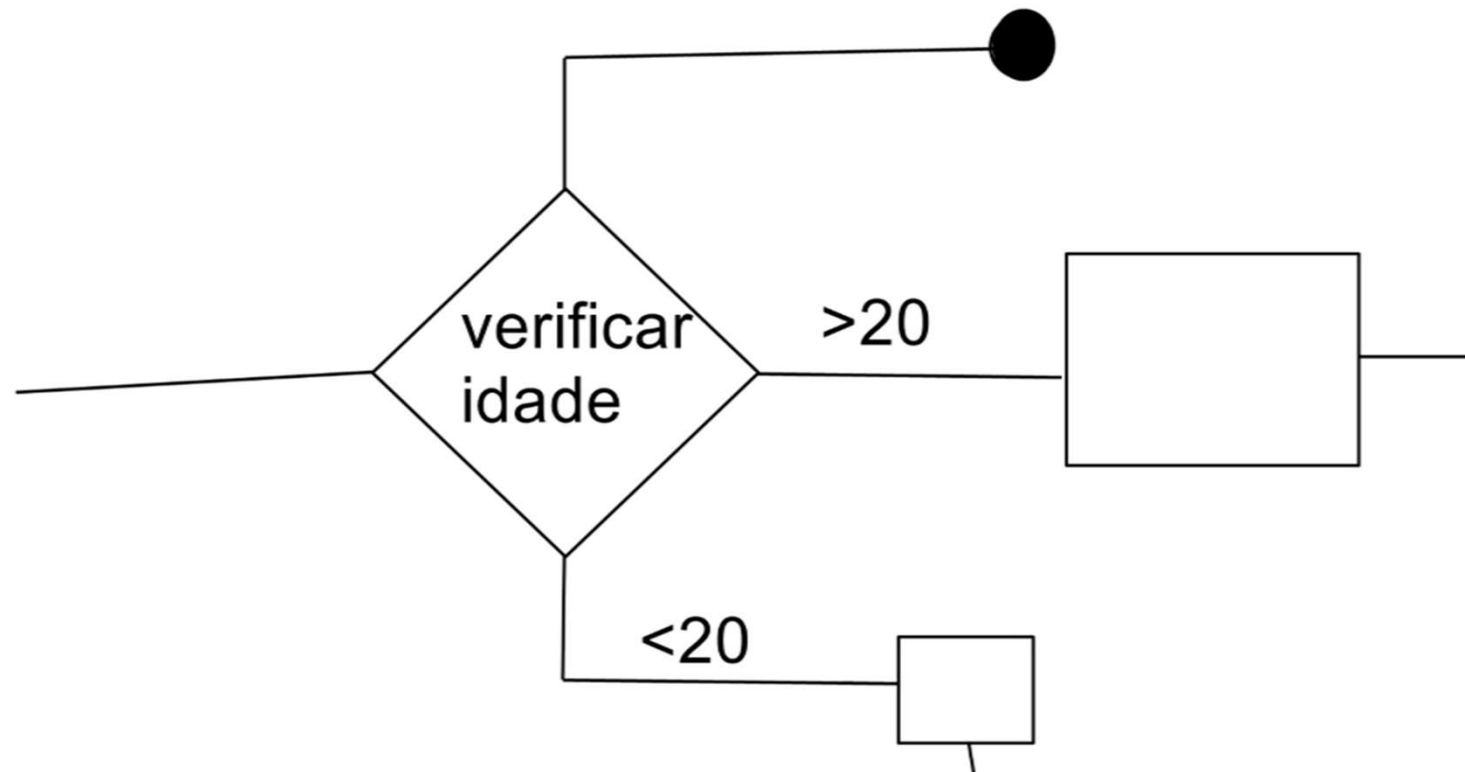
Se a pessoa tiver menos de 18, o programa não vai fazer nada, não vai escrever não pode entrar, não vai fazer nada. Só vai escrever pode entrar **caso tenha mais de 18 anos**.

Essa é uma **estrutura simples** que não tem o **se não**.

A **estrutura composta**, podemos colocar o **se não**, e também podemos colocar uma **estrutura condicional aninhada** (quando uma estrutura de decisão está localizada dentro do lado falso da outra), que é uma estrutura com vários **se não**.

Se a pessoa for menor que 5 anos, então ela é uma criança.

Se não, se uma pessoa tiver até 18 anos, adolescente. Se não, então eu posso colocar várias condições,
Veja aqui como seria uma estrutura no fluxograma.



Temos um caminho de código, e quando eu chego na estrutura condicional, eu coloco esse símbolo, e eu escrevo alguma coisa dentro desse símbolo.

Por exemplo, verificar idade. **Se a idade for maior que 20**, faço uma coisa. **Se for menor que 20**, faço outra, e tenho também o caminho que vai para o final desse fluxograma.

Sempre que tiver a bolinha, significa que chegou no final da estrutura, no final do programa.

Toda vez que for utilizar a **estrutura condicional** no fluxograma, será assim. Com esse símbolo, você coloca o que você quer fazer dentro, e coloca as setas que vão mostrar o caminho.

Agora Vamos Praticar alguns exercícios para fixar esse conteúdo.

1º Verificar qual numero é maior

Para verificar se um número é menor ou se é maior, vamos pedir para que o usuário entre com esta informação.

Então, vamos escrever, Escreval, digite um número.

Agora leia N1 para armazenar aquilo que for digitado em N1, e vamos criar essa variável N1, que será do tipo inteiro. A pessoa precisa digitar um número inteiro.

Vamos criar o N2 para armazenar o segundo número. Escreva Digite outro número.

Após digitar esses dois números, precisamos verificar qual é o número maior. Agora faremos o seguinte.

Precisamos colocar o fim se, se N1 for maior que N2, vamos dizer que N1 é maior que N2. Escreva o número, aí concatenamos com N1, é maior que N2. O número N1 é maior que o número 2. Caso contrário, se N1, se esse primeiro número não for maior, ele só pode ser menor, existe outra possibilidade, que é o número ser igual, mas não iremos abordar isso aqui.

Digite um número, vou colocar o número 10, digite outro número, eu vou colocar o número 25. Se N1 for maior que N2, então o número 1, N1 será maior que N2, mas caso contrário, ele é menor.

Veja o resultado: o número 25 é maior do que 10.

```

1 Algoritmo "semnome"
2 // Descrição : Aqui você descreve o que o programa faz
3
4 Var
5 // Seção de Declarações das variáveis
6 n1, n2 : inteiro
7
8 Inicio
9 // Seção de Comandos, procedimento, funções, operações
10 escreval("Digite um numero")
11 leia(n1)
12 escreval("Digite outro numero")
13 leia(n2)
14 se(n1 > n2) entao
15     escreval("O numero ",n1," é maior que ",n2)
16 senao
17     escreval("O numero ",n2," é maior que ",n1)
18 fimse
19
20 Fimalgoritmo

```

C:\ Console simulando o modo texto do MS-DOS

```

Digite um numero
10
Digite outro numero
25
O numero 25 é maior que 10

>>> Fim da execução do programa !

```

Vejam agora o próximo exercício:

Nesse exercício, queremos verificar se o número é par ou ímpar.

Para isso, nós temos que pensar, como nós sabemos se o número é par ou ímpar?

Sabemos que os números pares, eles são terminados em 0, 2, 4, 6, 8, ...

Na matemática, se você divide qualquer número pelo primeiro número par, o primeiro número par é o 2, você terá resto 0. Se você tiver algum resto que não seja 0, significa que esse número é ímpar.

Veja o exemplo, 30 dividido por 2 = 15, ou seja, 2 vezes 15 dá 30, que tirando, subtraindo 30 com 30 dá 0, que é o resto. Então, você percebe que o número 30 é um número par.

$$\begin{array}{r|l} 30 & 2 \\ - 30 & \\ \hline 0 & 15 \end{array}$$

Veja agora com o número ímpar.

Tenho 7, que é ímpar, se eu tentar dividir por 2, ele me retornará 3. faremos 2 vezes 3, vai dar 6, iremos subtrair, teremos resto 1, ou seja, sempre que retornar algum resto que não seja 0, será um número ímpar.

$$\begin{array}{r|l} 7 & 2 \\ - 6 & 3 \\ \hline 1 & \end{array}$$


E agora como representar isso dentro do algoritmo?

Nós utilizaremos o módulo.

Primeiro o usuário deve entrar com uma variável, com um valor, iremos chamar de **n**, será do tipo inteiro para a verificação se é **par** ou **ímpar**. **Se n, % “módulo” 2 for igual a 0**, ele vai pegar esse número, qualquer número que seja, dividir por 2 e verificar o resto.

```

1 Algoritmo "exercicios"
2
3 Var
4 // Seção de Declarações das variáveis
5 n : inteiro
6
7 Inicio
8 // Seção de Comandos, procedimento, fun
9 escreval("Digite um numero: ")
10 leia(n)
11     se (n%2 = 0) entao
12         escreva("O numero", n, " é par")
13
14         senao
15             escreva("O numero", n, " é impar")
16     fimse
17 Fimalgoritmo

```

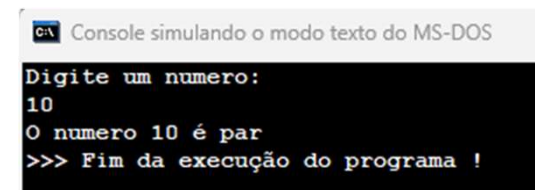
Digite o número 10 que é par.

Vamos o número 13 que é ímpar.

Se o resto for igual a 0, ele é um número par, então, escreva, o número n é par.

Caso contrário, se ele não é par, ele só pode ser ímpar, não existe outra possibilidade.

Então, escreva, caso contrário, o número n é ímpar, e fimse.

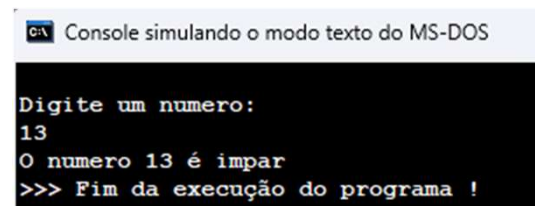


```

C:\> Console simulando o modo texto do MS-DOS

Digite um numero:
10
O numero 10 é par
>>> Fim da execução do programa !

```



```

C:\> Console simulando o modo texto do MS-DOS

Digite um numero:
13
O numero 13 é impar
>>> Fim da execução do programa !

```

Próximo exercício: Crie um sistema que verifique a senha e imprima uma mensagem de erro ou sucesso, acessível somente com a senha 1234.

Primeiro passo é criar a variável senha, é do tipo inteiro. Essa senha receberá o valor 1234. E iremos pedir ao usuário que digite a senha.

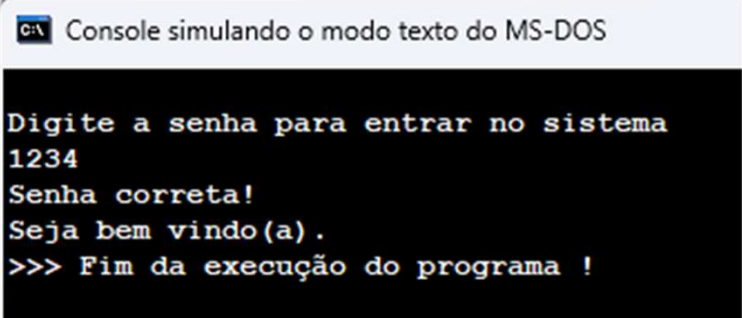
Assim, escreva, digite a senha para entrar no sistema. Agora preciso ler essa senha e armazenar em uma variável chamada de senhaDigitada, também do tipo inteiro.

```
1 Algoritmo "senha"  
2  
3 Var  
4 // Seção de Declarações das variáveis  
5 senha, senhaDigitada : inteiro  
6  
7 Inicio  
8 // Seção de Comandos, procedimento,  
9 senha <- 1234
```


Se a senhaDigitada for igual à senha, mostrar a mensagem que a pessoa está autorizada para acessar o sistema.

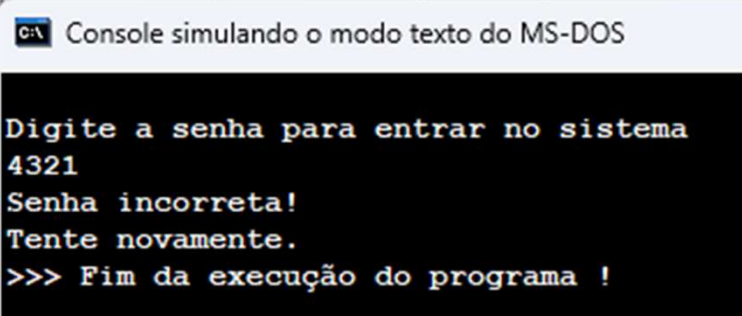
Então, escreval, **senha correta**, e mostre uma mensagem de boas-vindas, **Seja bem-vindo(a)!**. Se não, **Senha está incorreta!**, **Tente novamente**. E o fimse.

```
11 escreval("Digite a senha para entrar no sistema")
12 leia(senhaDigitada)
13     se(senhaDigitada = senha) entao
14         escreval("Senha correta!")
15         escreva("Seja bem vindo(a).")
16     senao
17         escreval("Senha incorreta!")
18         escreva("Tente novamente.")
19     fimse
20 Fimalgoritmo
```



Console simulando o modo texto do MS-DOS

```
Digite a senha para entrar no sistema
1234
Senha correta!
Seja bem vindo(a).
>>> Fim da execução do programa !
```



Console simulando o modo texto do MS-DOS

```
Digite a senha para entrar no sistema
4321
Senha incorreta!
Tente novamente.
>>> Fim da execução do programa !
```

EXERCÍCIO

Crie um algoritmo de uma calculadora simples:

Para iniciar precisamos fazer basicamente três coisas:

A primeira delas é pedir dois números, mesmo que a pessoa realize uma multiplicação, uma adição, uma divisão ou uma subtração, ela sempre precisará enviar dois números que serão armazenados em variáveis.

A segunda é pedir qual é a operação, a pessoa envia os dois números e a operação que ela deseja realizar.

A terceira é evitar erros, veremos que, como programadores, precisamos evitar o máximo de erros possível para que nosso software funcione.

Mesmo que seja apenas algoritmos e não uma linguagem de programação. Mas já é bom ver como é o funcionamento, como precisa pensar para evitar o máximo de erros possíveis.

EXERCÍCIOS

```
1 Algoritmo "calculadora"
2
3 Var
4 // Seção de Declarações das variáveis
5
6 n1, n2, res : real
7 operacao : inteiro
8 Inicio
9 // Seção de Comandos, procedimento, funções
10 escreva("Digite um número: ")
11 leia(n1)
12 escreva("Digite outro número: ")
13 leia(n2)
14 escreval()
15 escreval("Digite 1 para adição")
16 escreval("Digite 2 para subtração")
17 escreval("Digite 3 para multiplicação")
18 escreval("Digite 4 para divisão")
19 escreval
20 leia(operacao)
```

A primeira coisa será pedir dois números. Vamos escrever, digite um número e leia esse número, guardar em n1, digitar outro número e guardaremos em n2, criar as variáveis correspondentes do tipo real e pedir a operação.

Dar as opções, digite 1 para a adição, 2 para a subtração, 3 para a multiplicação e 4 a divisão, ler essa operação e criar a sua variável correspondente.

Como as opções são um, dois, três ou quatro, então nós criaremos do tipo inteiro, verificar qual número foi digitado, se a operação for igual a 1, criar uma variável para guardar a resposta, res do tipo real

```

20 leia(operacao)
21   se(operacao = 1) entao
22     res <- n1 + n2
23   senao
24     se(operacao = 2) entao
25       res <- n1 - n2
26     senao
27       se(operacao = 3) entao
28         res <- n1 * n2
29       senao
30         se(operacao = 4) entao
31           se(n2 = 0) entao //tratar divisao por 0
32             escreval("Não é possível dividir por 0")
33           senao
34             res <- n1 / n2
35           fimse
36         senao // aqui tem que responder caso a operação
37           // seja fora das 4 opções
38           escreval("Operação Invalida")
39         fimse
40       fimse
41     fimse
42   fimse
43 escreva ("O resultado é:", res)
44 Fimalgoritmo

```

Se a operação for igual a 1, res vai receber, a opção 1 é a adição, então $n1 + n2$, acrescentar um se se não se, a operação for igual a 2, será feita uma subtração. Se não, fim se, se a operação for igual a 3, se a operação for igual a 4, divisão.

E agora, nesse último se não, nós precisamos pensar na questão de a pessoa digitar um número que não seja de 1 a 4.

Ou seja, será uma operação inválida. Iremos mostrar na tela uma mensagem dizendo que a operação é inválida.

Agora nós evitamos o erro, a divisão com 0, é o segundo número que não pode ser 0, dentro aqui da opção 4, se $n2$, for igual a 0, não posso fazer essa operação, escreval Não é possível dividir por 0, se não, aí sim eu posso fazer o cálculo.

ESTRUTURA CONDICIONAL ESCOLHA, ESCOLHA CASO

Para que ela serve?

Nas linguagens de programação você encontrará como **switch**, **switch case**, que é o **escolha** e **escolha caso**.

Ela é uma estrutura também condicional, o funcionamento é muito parecido com o **SE**, mas é um pouco limitada, nós não podemos fazer comparação, por exemplo, que nem nós fazíamos no **SE**.

Se o número for menor que 2,5, se o número for igual a 2 e o outro for entre um número e outro, ela não consegue fazer isso, esta condicional é mais para números específicos.

Se o número for 1, 2 ou 3, ou se esse caractere for, por exemplo, de adição, de multiplicação, é pra isso que nós utilizamos pra um sistema mais básico, assim, menos complexo, e a estrutura não fica aninhada, igual o **SE**, que ficava um **SE** dentro do outro e formando uma montanha de código, aqui fica mais curto, porém ele também é um pouco mais limitado.

Veja um exemplo:

Crie uma variável valor do tipo inteiro, faça essa variável receber o número 2. Depois escreveremos **escolha**, o nome da variável que nós queremos utilizar, nesse caso a variável valor que contém o número 2, e colocar a palavra **caso**.

Caso 1, que significa caso 1? Caso o valor que está dentro dessa variável valor seja 1, eu vou escrever **OI**.

Caso 2, é o que esperamos, então escreva **TCHAU**.

Outro caso, ou seja, qualquer outro número, qualquer outra opção, iremos escrever **OI e TCHAU**, no fim, vem o **fim escolha**.

VALOR : INTEIRO

VALOR <- 2

ESCOLHA VALOR

CASO 1

ESCREVA("OI")

CASO 2

ESCREVA("TCHAU")

OUTROCASO

ESCREVA("OI E TCHAU")

FIMESCOLHA

EXERCÍCIO

Faremos uma calculadora também para ver como que é a diferença de uma calculadora usando o **se** e usando o **escolha caso**.

A ideia é a mesma, então vamos criar as variáveis n1 e n2 e a operação. Só que a operação não será do tipo inteiro, será do tipo caractere, para que a pessoa digite os mesmos símbolos das operações e o resultado do tipo real.

```
1 Algoritmo "calculadora"  
2  
3 Var  
4 // Seção de Declarações das variáveis  
5  
6 n1, n2, res : real  
7 operacao : caractere
```


EXERCÍCIOS

```
8 Inicio
9 // Seção de Comandos, procedimento, funções,
10 escreva("Digite um número: ")
11 leia(n1)
12 escreva("Digite outro número: ")
13 leia(n2)
14 escreval("Qual operação deseja realizar?")
15 escreval("+ - para somar")
16 escreval("- - para subtrair")
17 escreval("x - para multiplicar")
18 escreval("/ - para dividir")
19 leia(operacao)
20 escreval()
21   escolha operacao
22   caso "+"
23       res <- n1 + n2
24   caso "-"
25       res <- n1 - n2
26   caso "x"
27       res <- n1 * n2
28   caso "/"
29       res <- n1 / n2
30   outrocaso
31       escreval("Operação Invalida")
32   fimescolha
33 escreva ("O resultado é:", res)
34 Fimalgoritmo
```

Iremos iniciar pedindo os números que serão calculados. Digite o primeiro número. Leia n1, Digite o segundo número, leia n2, e escrever qual operação deseja realizar, exibir as opções com o símbolo de + para somar, - para subtrair, um x para multiplicar e uma / para dividir

Agora ler a operação, escrever escolha operação, caso o símbolo seja +, somar e guardar na variável res, caso seja o símbolo de - subtração.

Fazer o mesmo para a multiplicação e a divisão, outro caso, seja este um outro símbolo, mostrar uma mensagem, operação inválida e mostrar o resultado.

EXERCÍCIO

Agora, vamos simular um algoritmo para o Criança e Esperança. Nós iremos pedir a doação que a pessoa quer fazer.

```
1 Algoritmo "doação"  
2  
3 Var  
4 // Seção de Declarações das variáveis  
5 doacao : inteiro  
6 valor : real
```

Digite o número do valor que deseja doar e guardar em doação, criar essa variável do tipo inteiro, colocar as opções, 1 para doar 10 reais, 2 para doar R\$ 20,00, 3 para doar 50 reais ou quatro para doar mais. Na opção 4 ela digita a quantidade.

```

7 Inicio
8 // Seção de Comandos, procedimento, funções, operadores,
9 escreval("-----")
10 escreval("  CRIANÇA ESPERANÇA ")
11 escreval("-----")
12 escreval()
13 escreval("1 - para doar 10 reais")
14 escreval("2 - para doar 20 reais")
15 escreval("3 - para doar 50 reais")
16 escreval("4 - para doar mais")
17 escreval()
18 escreva("Digite o número do valor que deseja doar ")
19 leia(doacao)
20 escreval()
21     escolha doacao
22     caso 1
23         escreval("Você doou 10 reais!")
24         escreval("Obrigado por sua doação!")
25     caso 2
26         escreval("Você doou 20 reais!")
27         escreval("Obrigado por sua doação!")
28     caso 3
29         escreval("Você doou 50 reais!")
30         escreval("Obrigado por sua doação!")

```

Escrever no escolha doação, caso 1, Você doou 10 reais, Obrigado por sua doação!, Caso 2, você doou 20 reais, caso 3, você doou R\$50,00.

```
31 caso 4
32     escreva("Digite o valor que deseja doar R$: ")
33     leia(valor)
34     escreval()
35     escreval("Você doou", valor, " reais!")
36     escreval("Obrigado por sua doação!")
37 outrocaso
38     escreval("Operação Invalida")
39 fimescolha
40 Fimalgoritmo
```

Caso 4, pedir para informar o valor que deseja doar, digite o valor que deseja doar e precisamos ler essa informação, esse valor, criar a variável valor, do tipo real e vamos escrever você doou, aí colocamos o valor que ela informou e em reais. Obrigado por sua doação.

Por último, o outro caso, caso não digitou 1, 2, 3 ou 4, operação inválida e o fim escolha.

ESTRUTURA DE REPETIÇÃO, ENQUANTO, REPITA E PARA

Agora começaremos a ver os conceitos de estruturas de repetição.

Existem três tipos de estruturas iniciaremos com a estrutura **enquanto (WHILE)**, começando pela mais fácil e depois vamos complicar um pouquinho mais.

Para que é que serve as estruturas de repetição?

Como a própria palavra já diz, serve para repetir alguma coisa, haverá muitos e muitos momentos que nós vamos querer que um código seja repetido mais de uma vez, e ao invés de ficarmos escrevendo manualmente 500 linhas, nós podemos colocar uma linha dentro de uma estrutura, falar repita isso 10 vezes, 50 vezes, e aquilo será repetido.

Só que é mais do que isso, é uma repetição onde nós colocaremos regras complexas que poderemos utilizar em vários momentos. Se aplicarmos mesmo a lógica, vai ser muito útil.

Por exemplo, o algoritmo Criança Esperança. Digite 1 para doar R\$10, digite 2 para R\$20. Vamos colocar aqui 3 para doar mais. Lá digitávamos, por exemplo, 1 para doar R\$10, já falava obrigado por sua doação e já acabava o programa. Ou seja, para colocar uma outra opção, vamos supor que tenha outra pessoa, teria que executar o programa de novo.

Crianca esperança

Digite 1- para doar 10 reais
Digite 2 - para doar 20 reais
Digite 3 - para doar mais

Voce doou 20 reais!
Obrigado por sua doação!

Se colocarmos uma estrutura de repetição, por exemplo, para repetir 10 vezes, esse programa não fechará enquanto as 10 pessoas não digitarem a opção.

Então, com uma estrutura de repetição, abrem portas, oportunidades para fazermos muito mais coisas.

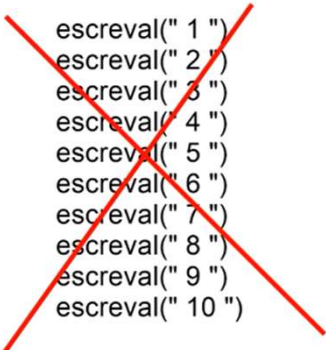
Por exemplo, eu posso pegar essas 10 vezes que foram executado esse código e posso guardar a votação de cada pessoa individualmente, fazer uma média de quantas pessoas doaram.

Posso fazer diversas coisas nesse código utilizando uma estrutura de repetição. 89

Mais um exemplo: Escrever, o número de 1 até o 10. Quero listar: 1, 2, 3, 4, 5...., para não escrever manualmente, escreva 1, escreva 2, podemos colocar uma estrutura de repetição, utilizar e fazer com que mostre isso apenas com 1 ou 2 linhas de código.

```
escreval(" 1 ")
escreval(" 2 ")
escreval(" 3 ")
escreval(" 4 ")
escreval(" 5 ")
escreval(" 6 ")
escreval(" 7 ")
escreval(" 8 ")
escreval(" 9 ")
escreval(" 10 ")
```

Veremos como fazer um contador, que é criar uma variável, fazer com que ela fique incrementando, ou seja, crescendo. Se a variável começa com 1... ela mostra na tela, depois nós incrementamos, vai para 2, aí mostra na tela 2, incrementamos, vai para 3, e assim nós conseguimos escrever do um ao dez na tela apenas com algumas linhas.



```
escreval(" 1 ")
escreval(" 2 ")
escreval(" 3 ")
escreval(" 4 ")
escreval(" 5 ")
escreval(" 6 ")
escreval(" 7 ")
escreval(" 8 ")
escreval(" 9 ")
escreval(" 10 ")
```

Vamos aprender a utilizar:
Ao colocar a palavra enquanto,
a expressão que nos queremos
utilizar, a palavra faça, os
comandos, o contador e o fim
enquanto.

Enquanto **expressão** faça

Comandos

Contador

Fimenquanto

Nós definimos o início, o fim, mas ainda preciso de um contador, precisamos que ele fique incrementando, ou seja, aumentando de 1 em 1, ou 2 em 2...

Definiremos o incremento. Vamos ao exemplo, contar até 10, temos o enquanto, o fim enquanto, uma variável **c** do tipo **inteiro**, que será nosso contador, que recebe 1, e ele contará de 1 até 10.

Agora vem os comandos. Faça o quê? Eu quero mostrar na tela esse número. E fazemos com que o contador seja incrementado.

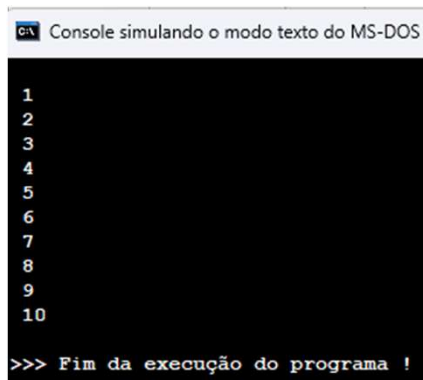
Nas linguagens de programação você verá esse símbolo aqui, **c++**. Não é a linguagem **c++**, é o símbolo de incremento.

Então, **c** recebe **c** mais **1**, é o mesmo que fazer **c++**, ou seja, o valor desse **c** sobe 1.

```

1 Algoritmo "contar10"
2
3 Var
4 // Seção de Declarações d
5 c : inteiro
6
7 Inicio
8 // Seção de Comandos, pro
9 c <- 1
10
11 enquanto c <= 10 faça
12     escreva(c)
13     c <- c + 1
14 fimenquanto
15
16 Fimalgoritmo

```



```

1
2
3
4
5
6
7
8
9
10
>>> Fim da execução do programa !

```

Agora escreva enquanto. Enquanto o quê? Enquanto esse *c* for menor ou igual a 10 faça e o fim enquanto.

Quero mostrar na tela, vou escrever e mostrar esse contador. Então vamos mostrar *c* e incrementar para que na segunda volta ele comece a valer 2.

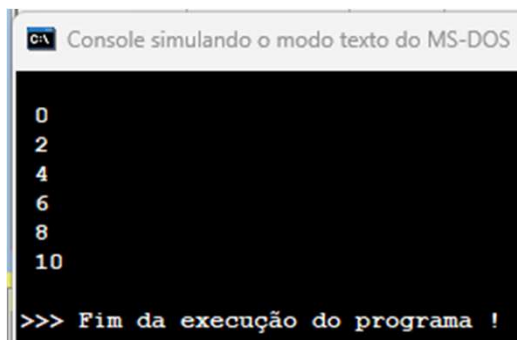
Então, *c* recebe o valor atual mais 1.

E isso é o suficiente para escrevermos na tela de 1 a 10, ou de 1 até quanto você quiser.


```

1 Algoritmo "contar10"
2
3 Var
4 // Seção de Declarações
5 c : inteiro
6
7 Inicio
8 // Seção de Comandos
9 c <- 0
10
11 enquanto c <= 10 faça
12     escreval(c)
13     c <- c + 2
14 fimenquanto
15
16 Fimalgoritmo

```



```

c:\ Console simulando o modo texto do MS-DOS
0
2
4
6
8
10
>>> Fim da execução do programa !

```

Agora vamos mostrar os números pares. Mas nós não precisamos colocar o resto da divisão de um número for igual a zero.

Vamos fazer aqui uma coisa bem simples. Apenas alterando o contador para 0 até 10, pulando de 2 em 2.

Afinal, o número par não é de dois em dois: 0, 2, 4, 6, 8...

Essa é uma forma bem fácil de mostrar os números pares, faço o contador ir pulando de dois em dois.

EXERCÍCIOS

1. Escreva um programa que pergunte um número ao usuário e mostra a sua tabuada completa (de 1 até 10);

```
1 Algoritmo "tabuada"
2
3 Var
4 // Seção de Declarações das variáveis
5 tab, c : inteiro
6
7 Inicio
8 // Seção de Comandos, procedimento, funções, ope
9 c <- 1
10 escreva("Qual tabuada você gostaria de ver? ")
11 leia(tab)
12
13 enquanto c<=10 faça
14     escreval(c, " x ", tab, " = ", c*tab)
15     // 1 x 1 =1
16     c <- c +1 //incrementa o contador
17     // 2 x 1 = 2
18 fimenquanto
19 Fimalgoritmo
```

EXERCÍCIOS

2. Ler um número inteiro n, escrever a soma de todos os número de 1 até n.

```
1 Algoritmo "soma_n_ate_n"
2
3 Var
4 // Seção de Declarações das variáveis
5 n, soma : real
6 c: inteiro
7 Início
8 // Seção de Comandos, procedimento, funções, op
9 c <- 1
10 soma <- 0 //importante
11 //na linguagem de programação
12 //a variável vem com lixo de memoria
13 escreva("Digite um número: ")
14 leia(n)
15
16 enquanto c<= n faça
17     soma <- soma + c // 0 + 1 do contador
18 // vai acrescer o contador
19 //ate o numero digitado
20     c<- c + 1
21 fimenquanto
22 escreval()
23 escreva("A soma de 1 até ", n, " é:", soma)
24 Fimalgoritmo
```

EXERCÍCIOS

3. Escreva um programa que apresente quatro opções: (a) consulta saldo, (b) saque (c), depósito e (d) sair. O saldo deve iniciar em R\$ 0,00. A cada saque ou depósito o valor do saldo deve ser atualizado. Exemplo:

Opções:
(a) consulta saldo
(b) saque
(c) depósito
> a
R\$ 0.00

Opções:
(a) consulta saldo
(b) saque
(c) depósito
> c
valor: 20.00

Opções:
(a) consulta saldo
(b) saque
(c) depósito
>a
R\$ 20.00

EXERCÍCIOS

```
1 Algoritmo "banco"
2
3 Var
4 // Seção de Declarações das variáveis
5 op : caractere
6 saque, deposito, saldo : real
7
8 Inicio
9 // Seção de Comandos, procedimento, funções, operadores,
10 escreval("Opções:")
11 escreval("(a) consultar saldo")
12 escreval("(b) sacar")
13 escreval("(c) depositar")
14 escreval("(d) sair")
15 escreval()
16 escreval("O que vc gostaria de fazer?")
17 leia(op)
18 enquanto op<>"d" faca
19     se op = "a" entao
20         limpatela
21         escreval("seu saldo atual é de R$: ",saldo)
22     senao
23         se op = "b" entao
24             escreval("Quanto vc quer sacar?")
25             leia(saque)
26             se saldo >= saque entao //verifica se tem saldo
27                 saldo <- saldo - saque
28                 limpatela
29                 escreval("Você sacou R$ ",saque)
30             senao
```

```
31         escreval("Você não tem saldo suficiente")
32         fimse
33     senao
34         se op = "c" entao
35             escreval("Qual é o valor do depósito?")
36             leia(deposito)
37             saldo <- saldo + deposito
38             limpatela
39             escreva("Você depositou R$ ",deposito)
40         senao
41             limpatela
42             escreval("Opção Invalida!")
43             escreva("Tente novamente")
44         fimse
45     fimse
46 fimse
47 escreval()
48 escreval()
49 // copiar as opções pois
50 //nao estao dentro do loop
51 escreval("Opções:")
52 escreval("(a) consultar saldo")
53 escreval("(b) sacar")
54 escreval("(c) depositar")
55 escreval("(d) sair")
56 escreval()
57 escreval("O que vc gostaria de fazer?")
58 leia(op)
59 fimenquanto
60 escreval("-----saindo...-----")
61 Fimalgoritmo
```

ESTRUTURA DE REPETIÇÃO, ENQUANTO, REPITA E PARA

Veremos mais uma estrutura de repetição, que é a estrutura repita. Essa estrutura é basicamente a estrutura enquanto invertida.

Como eu escrevo a estrutura repita? Nós escreveremos a palavra repita, colocamos os comandos igual no enquanto, qualquer comando que eu quiser fazer, e também um contador. No final que eu vou fazer a condição, eu vou fazer a verificação.

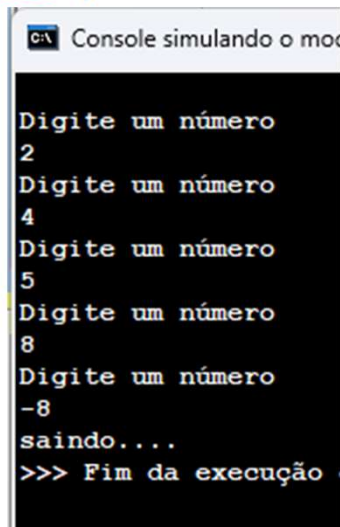
Então, no enquanto a condição, a verificação lá no começo, eu coloco repita, é executado pelo menos uma vez e eu coloco a palavra até a condição. Perceba que lá no enquanto, se o meu enquanto for falso, a condição for falso, ele nem entra no loop, não vai nem acontecer nenhuma vez o loop.

Aqui mesmo que a condição seja falsa, esse código vai acontecer pelo menos uma vez, vamos começar colocando o contador, como nós sempre fazemos, do tipo inteiro.

EXEMPLOS

Sair do loop quando o número for negativo

```
1 Algoritmo "repita"
2
3 Var
4 // Seção de Declarações das variáveis
5 n : real
6
7 Início
8 // Seção de Comandos, procedimento
9 repita
10     escreval("Digite um número")
11     leia(n)
12 ate (n<0)
13
14 escreva("saindo....")
15
16 Fimalgoritmo
```



```
C:\> Console simulando o mo...
Digite um número
2
Digite um número
4
Digite um número
5
Digite um número
8
Digite um número
-8
saindo....
>>> Fim da execução
```

Então ele não vai sair do loop se ele for **positivo** apenas quando eu digitar o número **negativo**, vou escrever, **repita...** Repita o que? Vamos pedir esse número então. Primeiro, digite um número e vamos armazenar esse número numa variável **n**, poderá ser do tipo **real**. Vou escrever até. Até que o quê? Até que esse **c** seja **negativo**. Como eu posso dizer isso? Até que o **c** seja menor do que **0**, porque um número **negativo** está abaixo de **0**. Então, - 1, - 2, - 3..., já começam os números negativos. E por fim, vamos escrever saindo.

EXERCÍCIO

```
1 Algoritmo "eleicoes"
2
3 Var
4 // Seção de Declarações das variáveis
5 voto, z, k, j : inteiro
6 resp : caractere
7 Inicio
8 // Seção de Comandos, procedimento, funções, ope
9 z <- 0
10 k <- 0
11 j <- 0
12 repita
13   escreval("----- Eleições 2024 -----")
14   escreval()
15   escreval("Digite 1 para Zé da Manga")
16   escreval("Digite 2 para Kleitinho do Grau")
17   escreval("Digite 3 para João Plim Plim")
18   escreval()
19   escreva("Entre com seu Voto: ")
20   leia(voto)
21   se voto = 1 entao
22     z <- z + 1
23   senao
24     se voto = 2 entao
25       k <- k + 1
26     senao
27       se voto = 3 entao
28         j <- j + 1
29       senao
30         escreval("Opção Invalida")
```

1. Desenvolva um programa onde exista uma eleição para eleger dentre três candidatos;

```
31         fimse
32     fimse
33     fimse
34     escreval("Deseja Continuar? S/N")
35     leia(resp)
36     escreval("-----")
37     limpatela
38     ate(resp = "n") ou (resp = "N")
39
40     escreval("----- Resultado Final -----")
41     escreval()
42     escreval("Zé da Manga - Obteve -",z," votos")
43     escreval()
44     escreval("Kleitinho do Grau - Obteve -",k," votos")
45     escreval()
46     escreval("João Plim Plim - Obteve -",j," votos")
47     escreval()
48     escreva("-----")
49 Fimalgoritmo
```


EXERCÍCIO

2. Crie um algoritmo que imprima os primeiros 400 números que são divisíveis por 3;

```
1 Algoritmo "400por3"  
2  
3 Var  
4 // Seção de Declarações das variáveis  
5 c: inteiro  
6  
7 Inicio  
8 // Seção de Comandos, procedimento,  
9 c <- 1  
10 repita  
11     se c % 3 = 0 então  
12         escreva(c, ", ")  
13     fimse  
14 c <- c + 1  
15 ate(c > 400)  
16 Fimalgoritmo
```

EXERCÍCIO

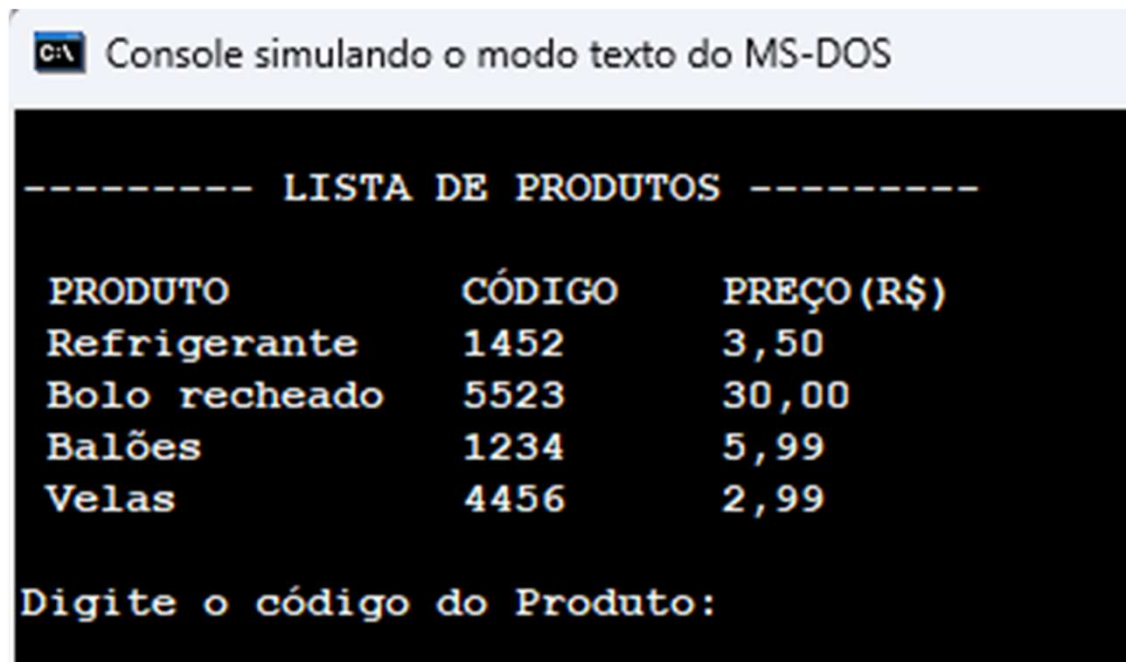
3. Crie um algoritmo que solicite 20 números, se o número for ímpar, imprima na tela e ao final mostre a soma de todos os números pares.

```
1 Algoritmo "imparSomaPar"
2
3 Var
4 // Seção de Declarações das variáveis
5 c, n, totalPar: inteiro
6
7 Inicio
8 // Seção de Comandos, procedimento, funções, operadores, etc...
9 c <- 1
10 totalPar <- 0
11 repita
12     escreva("Entre com um número: ")
13     leia (n)
14     se n % 2 = 0 entao
15         totalPar <- totalPar + n
16     senao
17         escreval()
18         escreval(n, " é ímpar")
19         escreval()
20     fimse
21
22 c <- c + 1
23 ate (c > 20)
24     limpatela
25     escreval("A soma de todos os números pares é ", totalPar)
26 Fimalgoritmo
```

EXERCÍCIO

Crie um algoritmo para um sistema de supermercado com uma lista de produtos com código e preço.

O usuário poderá inserir o código do produto e a quantidade que deseja comprar e ao final mostrar o total da compra



```
C:\ Console simulando o modo texto do MS-DOS

----- LISTA DE PRODUTOS -----

PRODUTO          CÓDIGO          PREÇO (R$)
Refrigerante     1452            3,50
Bolo recheado    5523            30,00
Balões           1234            5,99
Velas           4456            2,99

Digite o código do Produto:
```

EXERCÍCIO

Crie um algoritmo para um sistema de supermercado com uma lista de produtos com código e preço.

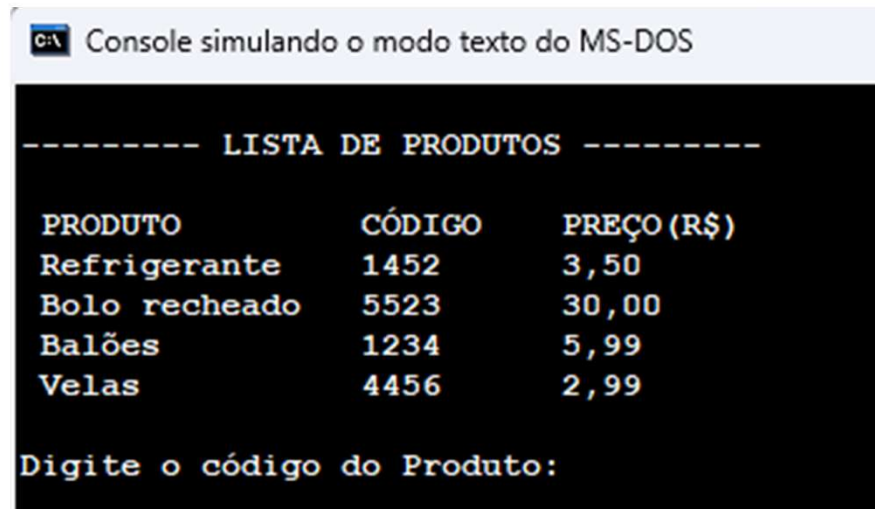
O usuário poderá inserir o código do produto e a quantidade que deseja comprar e ao final mostrar o total da compra

```
1 Algoritmo "supermercado"
2
3 Var
4 // Seção de Declarações das variáveis
5 cod, qt, op : inteiro
6 preco, total : real
7
8 Inicio
9 // Seção de Comandos, procedimento, funções, operadores, etc..
10 total <- 0
11 repita
12     escreval("----- LISTA DE PRODUTOS -----")
13     escreval()
14     escreval(" PRODUTO          CÓDIGO      PREÇO(R$) ")
15     escreval(" Refrigerante      1452        3,50")
16     escreval(" Bolo recheado      5523        30,00")
17     escreval(" Balões              1234         5,99")
18     escreval(" Velas               4456         2,99")
19     escreval()
20
```

```
21     escreval("Digite o código do Produto: ")
22     leia(cod)
23     escreval("Digite a quantidade deste Produto: ")
24     leia(qt)
25     se cod = 1452 entao
26         preco <- 3.50
27     senao
28         se cod = 5523 entao
29             preco <- 30.00
30         senao
31             se cod = 1234 entao
32                 preco <- 5.99
33             senao
34                 se cod = 4456 entao
35                     preco <- 2.99
36                 senao
37                     escreval("Código Invalido")
38                 fimse
39             fimse
40         fimse
41     fimse
42     total <- total + (preco * qt)
43     escreval("1 - para continuar comprando")
44     escreval("2 - para encerrar a compra")
45     leia(op)
```

EXERCÍCIO

```
46 limpatela
47 ate(op = 2)
48
49 escreva("O total de sua compra é de R$",total)
50 Fimalgoritmo
```

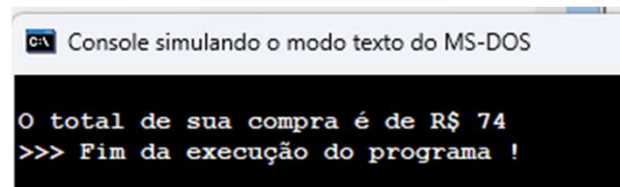


C:\> Console simulando o modo texto do MS-DOS

```
----- LISTA DE PRODUTOS -----

PRODUTO      CÓDIGO      PREÇO (R$)
Refrigerante  1452        3,50
Bolo recheado 5523        30,00
Balões        1234        5,99
Velas        4456        2,99

Digite o código do Produto:
```



C:\> Console simulando o modo texto do MS-DOS

```
O total de sua compra é de R$ 74
>>> Fim da execução do programa !
```

ESTRUTURA DE REPETIÇÃO, ENQUANTO, REPITA E PARA

Veremos a última estrutura de repetição, essa é a estrutura mais utilizada, que é a estrutura **PARA**.

Qual a diferença da estrutura **PARA**, com a estrutura **ENQUANTO** e a estrutura **REPITA**?

Como vimos, que a estrutura **ENQUANTO**, tem uma condição definida no início, **ENQUANTO** isso for verdade entre no loop, se o loop for falso, ele nem sequer entra.

Já o **REPITA**, ele vai acontecer pelo menos uma vez, porque a verificação acontece na última linha que é o até. Assim **REPITA**, aí vai fazer tudo isso e até algo acontecer, irá fazer a verificação. Essa é a diferença entre o **ENQUANTO** e o **REPITA**.

Agora a diferença dos dois em relação ao **PARA** é que o **PARA** ele é limitado, ele tem um limite definido.

Por exemplo, escreval, digite um valor, leia esse valor.

Eu digitei o número 10 e vai entrar nesse loop, para i de 1 até n, eu estou dizendo que ele vai de um número 1 até um número que eu vou definir.

Aqui eu defino o limite.

A estrutura PARA será sempre assim, de 1 até um número que eu digitar ou um número que eu definir.

```
escreval("Digite valor:")  
leia(n)
```

```
para i de 1 ate n faca
```

```
    escreval(n)
```

```
fimpara
```

E mais um detalhe é que ele não utiliza **caracteres**. Não podemos utilizar como fizemos no **REPITA**, pois essa instrução até que a variável operação **op** for igual a **string n** que digitou, pois quando ela digita a **n**, ela sai do loop.

Na estrutura **PARA**, você não consegue fazer isso, utiliza apenas um contador que é do tipo **inteiro**, na estrutura **PARA** será sempre assim.

Um valor **inicial** que vai até um valor final **n**, mesmo ele sendo limitado em relação ao final da sua estrutura, em relação a não poder utilizar caracteres, ainda assim ele é o mais utilizado.

Poderemos utilizar em vetores, que é muito mais fácil com essa estrutura do que utilizar com a estrutura **ENQUANTO**, e **REPITA**, mas também dá para utiliza-las.

REPITA

....

ATE(OP = "N")

- LIMITE DEFINIDO
- NÃO SE UTILIZA CARACTERES , SOMENTE INTEIROS

VALOR INTEIRO.....N

Como que é, então, essa estrutura?

Nós escrevemos **PARA** e colocamos uma variável, que será a variável **contadora**, faremos essa variável contadora receber um **valor inicial**.

Lembra que como fazíamos?

Nós instanciávamos a variável **c** do tipo **inteiro** e depois iniciávamos ela com o valor **1**, aqui já podemos fazer isso nesta linha. Colocamos a variável **c**, irá receber o valor **1**, já que no **PARA**, depois a palavra **até**, na mesma linha, e definimos o **fim**.

Para **variável** <- **inicio** ate **fim**

Por exemplo, posso criar uma variável **c** que inicia com **1**, que é o meu início, e vai **até** o meu fim, que é **10**, **do 1 até o 10**. Em seguida, pode adicionar o seu **passo**, que é o salto que quer dar.

Para **variável** <- **inicio** ate **fim** **passo** salto

Por exemplo, quero de **1 em 1**, será 1, 2, 3, 4, 5, 6...., que é o incremento do contador, no para é apontado nessa linha. Caso queira de 2 em 2, bastar escrever passo e dá o **salto** que desejar.

Para **variável** <- **inicio** ate **fim** **passo** **salto**

Aqui, você substitui essa palavra **salto** pelo valor que deseja saltar. Se for 2, então irá pular de 2 em 2, se for 3, vai pular de 3 em 3, também pode ser "-1", "-2" só em caso de 1 em 1 será opcional, por padrão, se não colocar a palavra **passo e o salto**, o salto automaticamente terá o incremento de 1 em 1.

PARA opcional
↓

Para **variável** <- **inicio** ate **fim** **passo** **salto**

E por fim escreve a palavra faça.

Para **variável** <- **inicio** ate **fim** **passo** **salto** **faça**

Dentro do para, colocaremos os comandos que quiser e o fim para.

Para demonstrar de 1 a 10, criaremos a variável contadora, e substituímos ali em variável, que será a **c**, e definir para contar de 1 até 10, o início é 1. Trocamos esse início para 1. Lembra que nós colocávamos c recebe 1? Não vamos precisar mais disso, definimos isso aqui na linha, já no início, **c** recebe 1. Perceba que é a mesma coisa até o fim, que é 10.

Vamos substituir a palavra fim por 10. Como isso é de 1 em 1... precisa ficar incrementando. Aquilo que fazíamos que era **c** recebe **c** mais 1, não será feito mais, porque isso será feito na linha do **PARA**, no salto, substitui pela quantidade de salto que você quer dar, 1, 2, -1, depende do salto. Se eu quero pular de 1 em 1, posso colocar passo 1 ou deixar vazio, porque automaticamente o passo será 1, vai ser incrementado de 1 em 1.

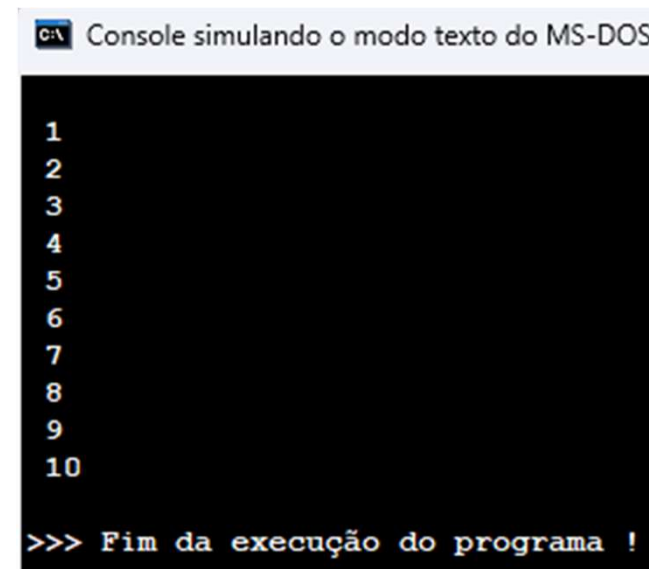
Para c <- 1 ate 10 passo salto faca

comandos

fimpara

Agora vamos ver um exemplo: Iniciaremos com o **PARA** c recebe 1, criaremos o contador c do tipo inteiro e limitar até 10, o faça e o fim para. Agora o escreva demonstrando o valor da variável. Assim já conseguimos escrever do número 1 ao 10 na tela.

```
1 Algoritmo "para1a10"  
2  
3 Var  
4 // Seção de Declaração  
5 C : inteiro  
6  
7 Inicio  
8 // Seção de Comandos,  
9  
10 para c <- 1 ate 10 fac  
11     escreval(c)  
12 fimpara  
13  
14 Fimalgoritmo
```

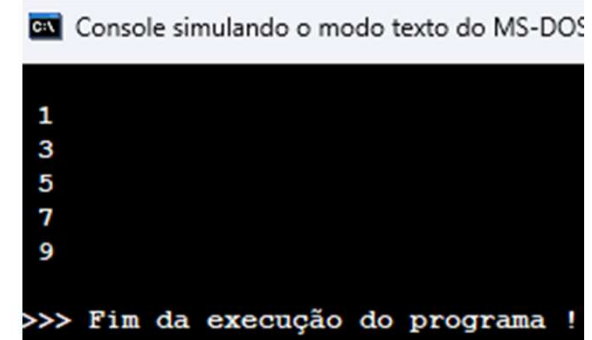


Console simulando o modo texto do MS-DOS

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
  
>>> Fim da execução do programa !
```

Se colocar passo 2, ele vai incrementar de dois em dois, ele começou no 1, pulou 2, 3, pulou 2, 5, 7 e 9.

```
10 para c <- 1 ate 10 passo 2 faca
11     escreval(c)
12 fimpara
```

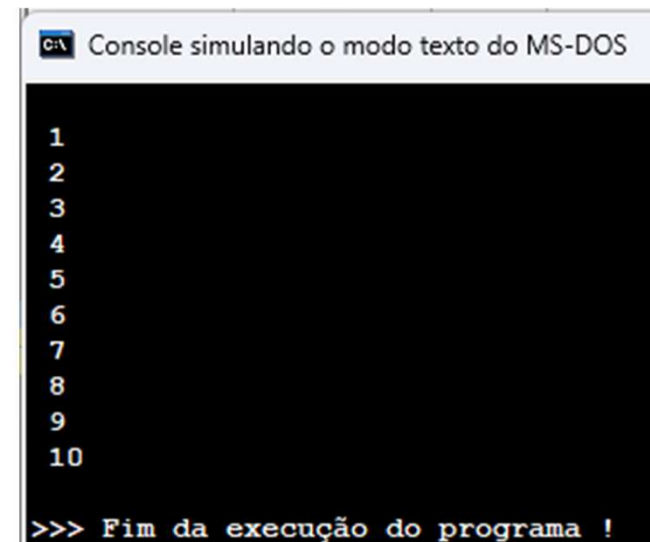


Console simulando o modo texto do MS-DOS

```
1
3
5
7
9
>>> Fim da execução do programa !
```

Além da seta, também podemos utilizar a palavra **de**.
O que eu estou dizendo?
Que essa variável **c** vai de 1 até 10.

```
10 para c de 1 ate 10 faca
11     escreval(c)
12 fimpara
```



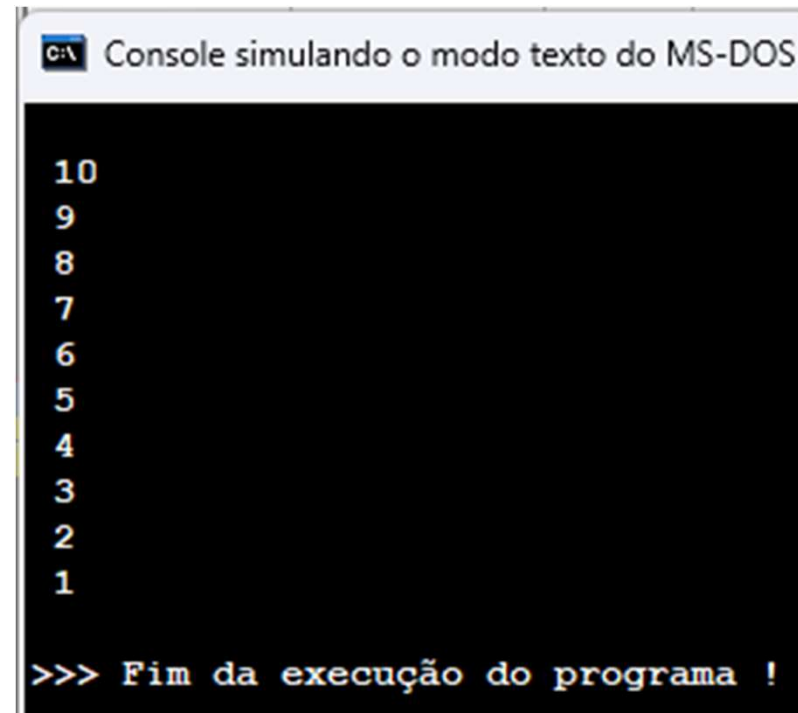
Console simulando o modo texto do MS-DOS

```
1
2
3
4
5
6
7
8
9
10
>>> Fim da execução do programa !
```

Nós podemos fazer o contrário, que é decrementar, vamos colocar a variável inicial com o valor 10 e vai até 1, isso irá fazer com que ela venha sendo decrementando de 1 em 1.

De menos 1 em menos 1, será 10, 9, 8, 7, 6..., da mesma forma que as outras estruturas, tudo o que tiver dentro dessa estrutura ficará sendo repetida.

```
1 Algoritmo "para1a10"  
2  
3 Var  
4 // Seção de Declarações das variáveis  
5 C : inteiro  
6  
7 Inicio  
8 // Seção de Comandos, procedimento,  
9  
10 para c de 10 ate 1 passo -1 faça  
11     escreval(c)  
12 fimpara  
13  
14 Fimalgoritmo
```



C:\ Console simulando o modo texto do MS-DOS

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
  
>>> Fim da execução do programa !
```

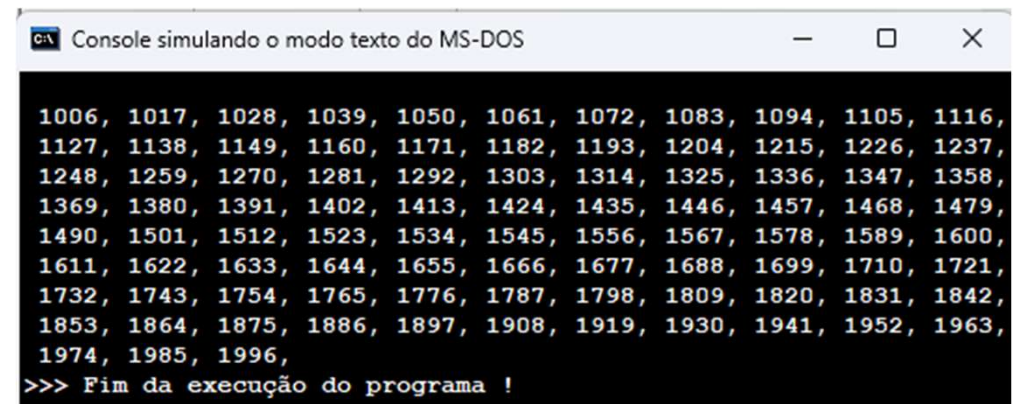
EXERCÍCIO

1. Faça um programa que verifique e mostre os números entre 1000 e 2000, que quando divididos por 11, produzam resto igual a 5.
2. Faça um programa que receba a idade e o sexo de 7 pessoas e que calcule e mostre:
 - a idade média do grupo
 - a idade média das mulheres
 - a idade média dos homens
3. Faça um programa que apresente uma tabuada completa do 1 ao 10.

EXERCÍCIO

1. Faça um programa que verifique e mostre os números entre 1000 e 2000, que quando divididos por 11, produzam resto igual a 5.

```
1 Algoritmo "dividido11resto5"
2
3 Var
4 // Seção de Declarações das vari
5 C : inteiro
6
7 Inicio
8 // Seção de Comandos, procedimen
9
10 para c <- 1000 ate 2000 faca
11     se c % 11 = 5 entao
12         escreva(c, ", ")
13     fimse
14 fimpara
15
16 Fimalgoritmo
```



```
CA\ Console simulando o modo texto do MS-DOS
1006, 1017, 1028, 1039, 1050, 1061, 1072, 1083, 1094, 1105, 1116,
1127, 1138, 1149, 1160, 1171, 1182, 1193, 1204, 1215, 1226, 1237,
1248, 1259, 1270, 1281, 1292, 1303, 1314, 1325, 1336, 1347, 1358,
1369, 1380, 1391, 1402, 1413, 1424, 1435, 1446, 1457, 1468, 1479,
1490, 1501, 1512, 1523, 1534, 1545, 1556, 1567, 1578, 1589, 1600,
1611, 1622, 1633, 1644, 1655, 1666, 1677, 1688, 1699, 1710, 1721,
1732, 1743, 1754, 1765, 1776, 1787, 1798, 1809, 1820, 1831, 1842,
1853, 1864, 1875, 1886, 1897, 1908, 1919, 1930, 1941, 1952, 1963,
1974, 1985, 1996,
>>> Fim da execução do programa !
```


EXERCÍCIO

2. Faça um programa que receba a idade e o sexo de 7 pessoas e que calcule e mostre:

- a idade média do grupo
- a idade média das mulheres
- a idade média dos homens

```
1 Algoritmo "idademediaGrupo-H-M"
2
3 Var
4 // Seção de Declarações das variáveis
5 c, idade, totalF, totalM, qntF, qntM : inteiro
6 sx : caractere
7 mediaGrupo : real
8
9 Inicio
10 // Seção de Comandos, procedimento, funções, op
11
12 para c <- 1 ate 7 faca
13     escreval("Qual a sua idade?")
14     leia(idade)
15     escreval("Qual o seu sexo [F] / [M]?")
16     leia(sx)
17     se (sx = "f") ou (sx = "F") entao
18         totalF <- totalF + idade
19         //soma das idades das mulheres
20         qntF <- qntF + 1
```

EXERCÍCIO

```
21     senao
22         se (sx = "m") ou (sx = "M") entao
23             totalM <- totalM + idade
24             //soma das idades dos homens
25             qntM <- qntM + 1
26         senao
27             escreval("Digite m ou f")
28         fimse
29     fimse
30 fimpara
31 // media
32 mediaGrupo <- (totalF + totalM) / 7
33 limpatela
34 escreval("A média de idade do grupo é de:", mediaGrupo:8:2)
35 escreval("A média de idade das mulheres é de:", (totalF / qntF))
36 escreval("A média de idade dos homens é de:", (totalM / qntM))
37
38 Fimalgoritmo
```

C:\> Console simulando o modo texto do MS-DOS

```
A média de idade do grupo é de:  12.57
A média de idade das mulheres é de:  11.00
A média de idade dos homens é de:  14.67

>>> Fim da execução do programa !
```

EXERCÍCIO

3. Faça um programa que apresente uma tabuada completa do 1 ao 10

```
1 Algoritmo "tabuadaPARA"
2 // tabuada completa
3 Var
4 // Seção de Declarações das variáveis
5 esq, dir : inteiro
6
7 Inicio
8 // Seção de Comandos, procedimento, funções, operadores, etc.
9   para esq <- 1 ate 10 faca
10     para dir <- 1 ate 10 faca
11       escreval(esq, " X", dir, " = ", esq * dir)
12     fimpara
13   escreval()
14 fimpara
15
16 Fimalgoritmo
```

CA Console simulando

1 X 1 = 1	3 X 1 = 3	5 X 1 = 5	7 X 1 = 7	9 X 1 = 9
1 X 2 = 2	3 X 2 = 6	5 X 2 = 10	7 X 2 = 14	9 X 2 = 18
1 X 3 = 3	3 X 3 = 9	5 X 3 = 15	7 X 3 = 21	9 X 3 = 27
1 X 4 = 4	3 X 4 = 12	5 X 4 = 20	7 X 4 = 28	9 X 4 = 36
1 X 5 = 5	3 X 5 = 15	5 X 5 = 25	7 X 5 = 35	9 X 5 = 45
1 X 6 = 6	3 X 6 = 18	5 X 6 = 30	7 X 6 = 42	9 X 6 = 54
1 X 7 = 7	3 X 7 = 21	5 X 7 = 35	7 X 7 = 49	9 X 7 = 63
1 X 8 = 8	3 X 8 = 24	5 X 8 = 40	7 X 8 = 56	9 X 8 = 72
1 X 9 = 9	3 X 9 = 27	5 X 9 = 45	7 X 9 = 63	9 X 9 = 81
1 X 10 = 10	3 X 10 = 30	5 X 10 = 50	7 X 10 = 70	9 X 10 = 90
2 X 1 = 2	4 X 1 = 4	6 X 1 = 6	8 X 1 = 8	10 X 1 = 10
2 X 2 = 4	4 X 2 = 8	6 X 2 = 12	8 X 2 = 16	10 X 2 = 20
2 X 3 = 6	4 X 3 = 12	6 X 3 = 18	8 X 3 = 24	10 X 3 = 30
2 X 4 = 8	4 X 4 = 16	6 X 4 = 24	8 X 4 = 32	10 X 4 = 40
2 X 5 = 10	4 X 5 = 20	6 X 5 = 30	8 X 5 = 40	10 X 5 = 50
2 X 6 = 12	4 X 6 = 24	6 X 6 = 36	8 X 6 = 48	10 X 6 = 60
2 X 7 = 14	4 X 7 = 28	6 X 7 = 42	8 X 7 = 56	10 X 7 = 70
2 X 8 = 16	4 X 8 = 32	6 X 8 = 48	8 X 8 = 64	10 X 8 = 80
2 X 9 = 18	4 X 9 = 36	6 X 9 = 54	8 X 9 = 72	10 X 9 = 90
2 X 10 = 20	4 X 10 = 40	6 X 10 = 60	8 X 10 = 80	10 X 10 = 100

>>> Fim da execução do programa !

PROCEDIMENTOS, VARIÁVEL LOCAL, VARIÁVEL GLOBAL E REFERENCIA

A partir de agora começaremos a falar sobre os **procedimentos** e **funções**, também falaremos sobre **variável local** e **variável global**.

Saber qual a diferença dessas variáveis.

E por último, veremos um pouco sobre **referência**, que também é em relação às variáveis, sendo do tipo **referência**.

Vamos dar início ao **procedimento**, para que serve um **procedimento**?

Um **procedimento**, ele também é uma **função**, só que o **procedimento**, ele não tem retorno e a **função** tem um retorno, veremos mais sobre retornos na aula de função.

O procedimento, ele é utilizado quando precisamos fazer o mesmo código várias vezes, pegamos esses códigos que são repetidos, colocamos dentro de um procedimento e a onde seria executado o código, nós chamamos o procedimento.

O procedimento serve para isso, mas ele também serve para organizar o seu código e o seu sistema também.

Vocês verão muito isso quando estiverem aprendendo uma linguagem orientada a objetos.

Algoritmo é uma linguagem estrutural, ou seja, linha embaixo de linha.

Executo a primeira linha, passo para a segunda, depois para a terceira...

A linguagem orientada a objetos também é assim, porém, ela tem uma dinâmica maior em relação às classes e também aos métodos.

Na linguagem de programação, as funções são chamadas de métodos.

Vocês verão aqui, como o código ficará muito mais organizado, utilizando procedimento ao invés de colocar apenas o código dentro do seu programa.

Vamos a um exemplo.

Tenho aqui um cabeçario e alguns tracejados só para destacar.

Escrevi aqui eleições 2018 e mais um tracejado.

Embaixo temos alguns comandos. Comando 1, comando 2, e limpar a tela. Aqui quero que aconteça o que?

Limpe a tela e mostre novamente esse cabeçario e um terceiro comando.

Veja que escrevi duas vezes a mesma informação e não é correto fazer isso.



```
Escreval("-----")  
Escreval("Eleicoes 2018")  
Escreval("-----")
```

comando 1

comando 2

limpatela



```
Escreval("-----")  
Escreval("Eleicoes 2018")  
Escreval("-----")
```

comando 3

Então o que faremos?

Pegaremos essas três linhas, tiraremos elas dali e colocaremos dentro de um procedimento e daremos um nome ao procedimento.

Chamei meu procedimento de mostrar cabeçario, em vez de escrever o código, escrevemos mostrar cabeçario e quando essa linha for executada, vai chamar aquelas três linhas e será mostrado na tela.

```
Escreval("-----")  
Escreval("Eleicoes 2018")  
Escreval("-----")
```

mostrarCabecario()

comando 1

comando 2

limpatela

mostrarCabecario()

comando 3

Agora vamos criar o procedimento:

Ele é realizado dentro das variáveis, colocamos procedimento e o nome do procedimento que deseje, abra e fecha parênteses, em seguida, escrever var, essa área será a de variáveis deste procedimento.

O início e o fim procedimento.

Para teste, vamos fazer com que esse procedimento apenas mostre uma frase, escreverá "Bom Dia!".

Depois na área de início, do nosso algoritmo, iremos chamar esse procedimento.

Como é que nós faremos isso?

Apenas escrever o nome do procedimento, que eu chamei de nomeDoProcedimento, com os parênteses. Ele irá executar o que está dentro do procedimento.

```
1 Algoritmo "procedimento"
2
3 Var
4 // Seção de Declarações das variáveis
5     procedimento nomeDoProcedimento()
6     Var //variaveis do procedimento
7
8     inicio//início do procedimento
9         Escreval("Bom Dia!")
10    fimprocedimento
11
12 Inicio
13 // Seção de Comandos, procedimento, fun
14     nomeDoProcedimento()
15
16 Fimalgoritmo
```


Veja um exemplo de Soma e Subtração utilizando procedimento:

```
1 Algoritmo "procedSomaSub"
2
3 Var
4 // Seção de Declarações das variáveis |
5 n1, n2 : real
6 resp : inteiro
7
8 procedimento Somar()
9   Var //variaveis do procedimento
10
11   inicio//inicio do procedimento
12     Escreval("A soma é: ", n1+n2)
13   fimprocedimento
14
15 procedimento Sub()
16   Var //variaveis do procedimento
17
18   inicio//inicio do procedimento
19     Escreval("A subtração é: ", n1-n2)
20   fimprocedimento
21
```

```
22 Inicio
23 // Seção de Comandos, procedimento, função.
24 escreva("Digite um numero: ")
25 leia(n1)
26 escreva("Digite outro numero: ")
27 leia(n2)
28 escreval("O que deseja fazer?")
29 escreval("1 - para somar")
30 escreval("2 - para somar")
31 leia(resp)
32   se resp = 1 entao
33     somar()
34   senao
35     sub()
36   fimse
37
38 Fimalgoritmo
```

- ✓ **Variáveis Globais** são aquelas declaradas no início de um algoritmo, são visíveis, ou seja, podem ser utilizadas no algoritmo principal e por todos os outros sub-algoritmos (Procedimentos, Funções e etc).
- ✓ **Variáveis Locais** são aquelas declaradas no início de um sub-algoritmos, podendo somente ser utilizados por estes.
- ✓ **Parâmetros** são valores que, na linha de chamada, ficam entre os parênteses e que estão separados por vírgulas, ao se chamar um módulo, também é possível passar-lhe determinadas informações.

```
1 Algoritmo "procedSomaSub"
2
3 Var //(Variáveis Globais)
4 // Seção de Declarações das variáveis
5 n1, n2 : real
6 resp : inteiro
7
8 procedimento Somar()
9 Var //variaveis do procedimento
10 //(Variáveis Locais)
11
12 inicio//inicio do procedimento
13     Escreval("A soma é: ", n1+n2)
14 fimprocedimento
15
```

PASSAGEM DE PARÂMETROS

- **Passagem por valor** – Ocorre quando o valor dos parâmetros formais recebem os valores dos parâmetros reais. Caso os valores dos parâmetros formais sejam alterados, este não reflete para os parâmetros reais.

```
algoritmo Exemplo
```

```
var nome:literal
```

```
procedimento imprimir(novoNome:literal);
```

```
inicio
```

```
    escreval("Este e o valor da variavel que foi passado como  
    parametro: ", novoNome)
```

```
fimprocedimento
```

```
inicio
```

```
    nome <- "PASCAL"
```

```
    escreva("Esta é a variavel Global: ", nome)
```

```
    imprimir("Algoritmo")
```

```
finalgoritmo
```

Declaração de
procedimento
recebendo
parâmetro formais

Chamada de
procedimento
passando
parâmetro reais

PASSAGEM DE PARÂMETROS

- **Passagem por Referência** – Ocorre quando qualquer alteração no valor dos parâmetros formais durante a execução do procedimento será refletido no valor de seus parâmetros reais correspondentes.

```
Algoritmo exemplo
var nome:literal
procedimento imprimir(var novoNome:literal);
inicio
    novoNome <- "DELPHI:"
    escreval('Este é o valor da variável dentro do
              procedimento:', novoNome)
fimprocedimento

inicio
    nome <- "PASCAL"
    escreval("Esta é a variável antes de chamar o
              procedimento", nome)
    imprimir(nome)
    escreval("Este é a variável depois de chamar o
              procedimento", nome)
finalgoritmo
```

Declaração de procedimento recebendo parâmetro por referência

Chamada de procedimento passando parâmetro

FUNÇÕES

Agora daremos início às **funções**. A primeira coisa que você precisa saber sobre **função** é que o **procedimento** é uma **função**, só que a diferença do **procedimento** para a **função** é que o **procedimento** não tem **retorno** e uma **função** contém um **retorno**. Como fazer uma função? Vejamos um procedimento chamado **Proc**, que contém um parâmetro **a** do tipo **real**, vamos criar aqui uma variável **b** do **tipo real** dentro desse **procedimento**, e fazer com que esse **b** receba **a + a**, que é o valor que foi enviado através dos parâmetros, em seguida, mostrar na tela **b**, se enviei o valor 10 no **a** esse **b** vai receber **10 + 10**, ou seja, **20**, e vai mostrar na tela o **valor 20**.

```
PROCEDIMENTO PROC (A : REAL)
VAR
    B : REAL

INICIO
    B <- A + A
    ESCREVA (B)

FIMPROCEDIMENTO
```

Como seria uma função?

Você escreve **função** irei chamar ela de **FC**, também possui parâmetros, também pode colocar os seus **parâmetros**, temos também a área de **variável, início e fim função**.

Após a linha dos **parâmetros**, após os **parênteses**, colocaremos o **tipo de retorno**, o que não existia no **procedimento**. Aqui devemos colocar, quando tratamos de funções, dois pontos e o **tipo**, neste exemplo utilizei o retorno do tipo **real**.

```
FUNCAO FC (A : REAL) : REAL
```

```
VAR
```

```
B : REAL
```

```
INICIO
```

```
B <- A + A
```

```
FIMFUNCAO
```

Aqui, posso criar uma variável, também chamada **B**, do tipo **REAL**, adicionar o valor de **A** + **A**, essa variável **B** também está valendo no caso se **A** fosse **10**, esse **B** também estaria valendo 20.

Veja agora a diferença, ao invés de escrever **B** na tela, o que eu faço?

Pego **B** e retorno. Retorno pra quem? Retorno pra quem chamou essa função.

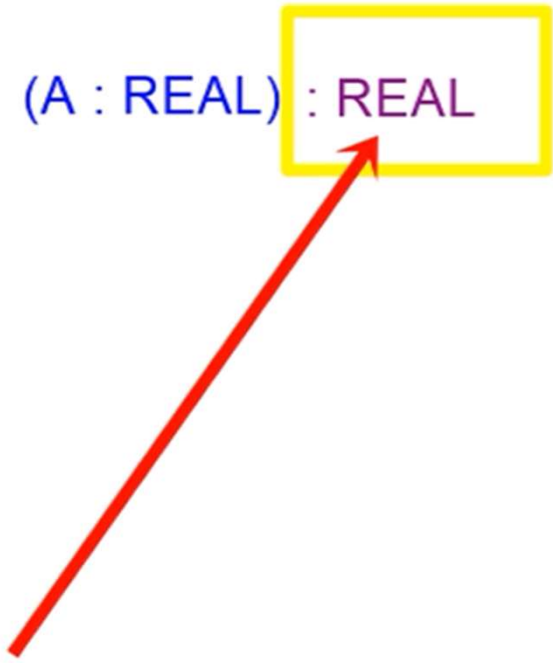
Toda função tem que ter essa palavra **retorne**, e ela tem que corresponder com aquele meu **tipo**, se eu coloquei o tipo **real** como **retorno**, esse **B** que eu estou retornando tem que ser do **tipo real**.

Se eu coloco do **tipo inteiro**, esse **B** tem que ser do **tipo inteiro**, e você pode retornar de qualquer tipo que quiser, você pode colocar o **tipo lógico**, o **tipo caractere**, o **tipo real** ou o **tipo inteiro**.

```
FUNCAO FC (A : REAL) : REAL
VAR
B : REAL

INICIO
B <- A + A
RETORNE B

FIMFUNCAO
```



Agora vamos a parte do algoritmo, temos no algoritmo, o nome do algoritmo, var, início e fim algoritmo, vamos criar nesse algoritmo uma variável **resposta** do **tipo real**.

Essa variável será a variável que vai receber a chamada do método **FC**.

Então como fazemos isso?

Colocamos o símbolo de atribuição, aquele mesmo símbolo que nós colocamos na variável, a resposta, ela vai receber aquilo que for retornado pela função.

Perceba, chamei **FC** e passei o valor **13**.

ALGORITMO

VAR

RESPOSTA : REAL

INICIO

RESPOSTA <- FC(13)

FIMALGORITMO

A função **FC** pegou o valor **13** e guardou no **A**, e esse **A** foi somado ali e guardado no **B**. Em seguida, eu **retornei** esse **B**.

Quando eu **retorno** esse **B**, que é do mesmo tipo de retorno, temos que prestar atenção no tipo do retorno, quando coloco **retorne B**, o meu **resposta** recebe **B**.

Agora, o que está guardado na variável **resposta do tipo real**?

Aquilo que foi **retornado em B**, que nesse caso seria o **26**, já que enviei o valor **13**.

```
ALGORITMO
VAR
  RESPOSTA : REAL
INICIO

  RESPOSTA <- FC(13)
  ESCREVA(RESPOSTA)

FINALGORITMO
```



26

```
FUNCAO FC (A : REAL) : REAL
VAR
  B : REAL
INICIO
  B <- A + A
  RETORNE B
FIMPROCEDIMENTO
```

E assim, com essa variável resposta que já está preenchida, eu posso fazer o que eu quiser.

Eu posso, por exemplo, escrever a resposta na tela, por fim, você escolhe no seu código se você quer usar um procedimento, se você prefere que tudo seja feito dentro da função.

Vejamos tudo na prática, para isso, vamos criar uma **função** chamada **adição**.

E essa função terá dois **parâmetros**, **n1** e **n2** do **tipo inteiro**, e vai **retornar** também um **tipo inteiro**.

Coloco dois pontos, **inteiro**.

Vamos criar a **área de variáveis**, **início** e **fim função**.

```
1 Algoritmo "Função"
2
3 Var
4 // Seção de Declarações das variáveis
5     funcao adicao(n1, n2 : Inteiro) : inteiro
6     var
7
8     inicio
9
10    fimfuncao
11
12 Inicio
13 // Seção de Comandos, procedimento, funções, op
14
15
16 Fimalgoritmo
```

Vamos criar uma segunda função, que vai fazer o quadrado de um número, vamos chamar ela de **quadrado**, recebendo um **parâmetro** chamado **n1** do **tipo inteiro** e vai **retornar** um **tipo inteiro** também, a **área de variáveis**, **início** e **fim função**.

E ainda uma terceira função para escrever na tela, que vai receber um **parâmetro** do tipo **caractere** chamado **nome**, e vai **retornar** um **caractere**.

Estou colocando aqui **inteiro**, **retorna inteiro**, **caractere** **retorna caractere**, mas não precisa ser assim, pode ser um **inteiro** que **retorne** um **outro tipo**.

```
funcao quadrado(n1: inteiro) : inteiro  
var
```

```
inicio
```

```
fimfuncao
```

```
funcao escreverTela(nome : caractere) : caractere  
var
```

```
inicio
```

```
fimfuncao|
```

Agora na primeira função, vamos criar uma variável dessa função chamada **S** do **tipo inteiro**, como estamos tratando de uma **função** que vai fazer a **adição de dois números**, vamos simplesmente fazer com que **S receba N1 mais N2 e retornar esse S**, que corresponde ao mesmo **tipo de retorno**.

```
funcao adicao(n1, n2 : Inteiro) : inteiro
var
    s: inteiro
inicio
    s <- n1 + n2
    retorne s
fimfuncao
```

Na segunda função, a mesma coisa, vamos criar um **S do tipo inteiro**, que vai receber, como estamos tratando, de um número elevado ao quadrado, **será n1 vezes n1** e retorne esse **S**, que é do mesmo tipo de retorno.

```
funcao quadrado(n1: inteiro) : inteiro
var
    S : inteiro
inicio
    S : n1 * n1
    retorne s
fimfuncao
```

Na terceira função, vamos criar uma **variável de boasVindas**, vamos criar **boasVindas** do tipo **caractere**.

E esse Boas-vindas vai receber uma frase. Seja bem-vindo(a)! concatenando com a variável **nome** que foi enviada.

Então ficará **seja bem-vindo mais o nome** dessa pessoa.

E vamos **retornar** essa frase completa, que está guardado dentro de **boasVindas**.

```
funcao escreverTela(nome : caractere) : caractere
var
    boasVindas : caractere
inicio
    boasVindas <- "Seja bem Vindo(a?) " + nome
    retorne boasVindas
fimfuncao
```

Agora temos três funções, uma pra adição, uma segunda pra elevar ao quadrado e uma terceira pra dar umas boas vindas.

Agora, na área do algoritmo, vamos perguntar o que essa pessoa quer fazer, escreval o que deseja fazer e exibir as opções.

Vamos colocar:

1 pra somar dois números;

2 para elevar um número ao quadrado e

3 para ver uma mensagem de boas-vindas.

Vamos ler essa informação, que é 1, 2 ou 3, então é **do tipo inteiro** e criar essa variável **op do tipo inteiro**.

Início

```
// Seção de Comandos, procedimento, funções, operadores, e  
escreval("O que deseja fazer?")  
escreval("1 - somar dois numeros")  
escreval("2 - elevar um numero ao quadrado")  
escreval("3 - ver uma mensagem de boas vindas")  
leia(op)|
```

E utilizar a estrutura **escolha**, colocaremos **escolha** e o nome da variável **op**, depois os casos.

Caso 1, somar dois números, para somar dois números nós precisamos de dois números.

Então vamos pedir essas duas informações, escreval, digite um número, e armazenarem em **n**, digite outro número. e armazenar em **n2**, vamos criar essas variáveis e chamar o método de adição, nós escrevemos **adição enviando esses dois valores**, como tenho um **retorno do tipo inteiro**, ele vai me devolver alguma coisa e eu preciso guardar em algum lugar, vamos criar uma variável chamada **r** do **tipo inteiro** e vamos fazer com que **r receba a chamada do método adição enviando dois valores** e por fim mostrar a soma, escreval, a soma é... vamos mostrar **r**.

```
caso 1
  escreval("Digite um numero: ")
  leia(n)
  escreval("Digite outro numero: ")
  leia(n2)
  r <- adicao(n, n2)
  escreval("A soma é ", r)
```

Caso 2, vou elevar o número ao quadrado, para isso precisamos de um **número**, que é a **quantidade de parâmetros**, também iremos utilizar a variável **n** para buscar essa informação, que é do tipo inteiro e enviar ela para a **função quadrado** e depois pegar o resultado e guardar em **r**.

```
caso 2
  escreval("Digite um numero: ")
  leia(n)
  r <- quadrado(n)
  escreval("O resultado é: ", r)
```

Caso 3 é a opção de ver uma mensagem de boas-vindas, a **função de boas-vindas** ela precisa de um **parâmetro nome do tipo caractere** para ser enviado.

Vamos enviar essa informação, chamando a função **escreverTela** e pedindo um nome, isso retornará um **caractere**, para isso, preciso criar uma variável para guardar essa informação e colocar resultado, **recebe a chamada de escrever na tela**.

```
caso 3
  escreval("Escreva seu nome para receber a mensagem")
  leia(nome)
  resultado <- escreverTela (nome)
  escreval(resultado)
```

E por fim o outro caso, caso não seja 1, nem 3, nem 3, só pode ser uma informação inválida, uma opção inválida.

Então vamos escrever Opção inválida e o fim escolha.

```
59      outrocaso
60          escreval("Opção Inválida")
61      fimescolha
```

Além de você poder fazer as suas próprias funções e utilizar elas da forma que quiser, também existem as funções prontas que vieram direto do Visualg.

FUNÇÕES NUMÉRICAS, ALGÉBRICAS E TRIGONOMÉTRICAS

Abs(expressão) - Retorna o valor absoluto de uma expressão do tipo inteiro ou real. Equivale a $| \text{expressão} |$ na álgebra.

ArcCos(expressão) - Retorna o ângulo (em radianos) cujo co-seno é representado por expressão.

ArcSen(expressão) - Retorna o ângulo (em radianos) cujo seno é representado por expressão.

ArcTan(expressão) - Retorna o ângulo (em radianos) cuja tangente é representada por expressão.

Cos(expressão) - Retorna o co-seno do ângulo (em radianos) representado por expressão.

CoTan(expressão) - Retorna a co-tangente do ângulo (em radianos) representado por expressão.

Exp(base, expoente) - Retorna o valor de base elevado a expoente, sendo ambos expressões do tipo real.

GraupRad(expressão) - Retorna o valor em radianos correspondente ao valor em graus representado por expressão.

Int(expressão) - Retorna a parte inteira do valor representado por expressão.

Log(expressão) - Retorna o logaritmo na base 10 do valor representado por expressão.

LogN(expressão) - Retorna o logaritmo neperiano (base e) do valor representado por expressão.

Pi - Retorna o valor 3.141592.

Quad(expressão) - Retorna quadrado do valor representado por expressão.

RadpGrau(expressão) - Retorna o valor em graus correspondente ao valor em radianos representado por expressão.

RaizQ(expressão) - Retorna a raiz quadrada do valor representado por expressão.

Rand - Retorna um número real gerado aleatoriamente, maior ou igual a zero e menor que um.

RandI(limite) - Retorna um número inteiro gerado aleatoriamente, maior ou igual a zero e menor que limite.

Sen(expressão) - Retorna o seno do ângulo (em radianos) representado por expressão.

Tan(expressão) - Retorna a tangente do ângulo (em radianos) representado por expressão.

FUNÇÕES PARA MANIPULAÇÃO DE CADEIAS DE CARACTERES (STRINGS)

Asc (s : character) : Retorna um inteiro com o código ASCII do primeiro caracter da expressão.

Carac (c : inteiro) : Retorna o caracter cujo código ASCII corresponde à expressão.

Caracpnum (c : character) : Retorna o inteiro ou real representado pela expressão. Corresponde a StrToInt() ou StrToFloat() do Delphi, Val() do Basic ou Clipper, etc.

Compr (c : character) : Retorna um inteiro contendo o comprimento (quantidade de caracteres) da expressão.

Copia (c : character ; p, n : inteiro) : Retorna um valor do tipo character contendo uma cópia parcial da expressão, a partir do caracter p, contendo n caracteres. Os caracteres são numerados da esquerda para a direita, começando de 1. Corresponde a Copy() do Delphi, Mid\$() do Basic ou Substr() do Clipper.

Maiusc (c : character) : Retorna um valor character contendo a expressão em maiúsculas.

Minusc (c : character) : Retorna um valor character contendo a expressão em minúsculas.

Numpcarac (n : inteiro ou real) : Retorna um valor caracter contendo a representação de n como uma cadeia de caracteres. Corresponde a IntToStr() ou FloatToStr() do Delphi, Str() do Basic ou Clipper.

Pos (subc, c : character) : Retorna um inteiro que indica a posição em que a cadeia subc se encontra em c, ou zero se subc não estiver contida em c. Corresponde funcionalmente a Pos() do Delphi, Instr() do Basic ou At() do Clipper, embora a ordem dos parâmetros possa ser diferente em algumas destas linguagens.

EXERCÍCIO

1. Crie uma função que receba um valor qualquer do teclado e imprima esse valor com reajuste de 10%.

Vetor (array ou matriz) - Um vetor é uma estrutura que armazena uma sequência de objetos do mesmo tipo, em posição consecutiva lógica e física, sendo possível alcançar qualquer item diretamente. Não é necessário passar pelo primeiro ou último elemento.

Duas das estruturas de dados mais básicas são **vetores** e **matrizes**.

O **vetor** é uma estrutura de dados com apenas uma dimensão (unidimensional).

Já uma **matriz** é uma estrutura de dados com duas (bidimensional) ou mais dimensões.

Observe o exemplo a seguir.

- a) O Vetor seria uma única prateleira capaz de guardar determinados objetos, um do lado do outro.
- b) A Matriz bidimensional seria uma grande estante com vários espaços, um do lado do outro e um abaixo do outro.



Figura 73 - Estrutura de dado unidimensional – Prateleira de Livros
Fonte: do Autor (2020)

Veja que, em uma estante de livros (Vetor de Livros), cada livro existente na coleção é posicionado um ao lado do outro, estando limitados a apenas uma dimensão, como se todos os objetos estivessem organizados em uma mesma “linha”, nesse caso, da esquerda para a direita sempre.

E no caso de uma matriz? Veja, na imagem, a seguir, uma forma de armazenamento bidimensional, que poderia, também, guardar uma coleção de livros.

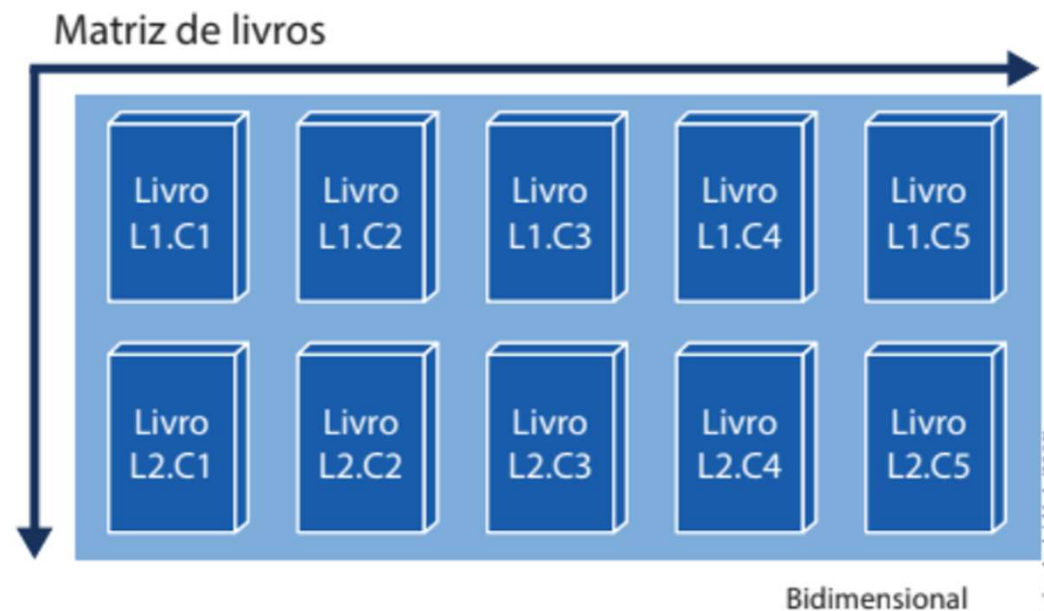


Figura 74 - Estrutura de Dado Bidimensional – Estante de Livros
Fonte: do Autor (2020)

Observe que, no caso da estante, agora há duas dimensões na estrutura de dados: uma dimensão horizontal (da esquerda para a direita) e uma vertical (cima para baixo). Note que é possível inclusive mapear os livros conforme essas dimensões:

- **Livro L1.C1:** Livro na Linha 1 e Coluna 1;
- **Livro L1.C2:** Livro na Linha 1 e Coluna 2;
- **Livro L1.C3:** Livro na Linha 1 e Coluna 3;
- **Livro L1.C4:** Livro na Linha 1 e Coluna 4;
- **Livro L1.C5:** Livro na Linha 1 e Coluna 5;
- **Livro L2.C1:** Livro na Linha 2 e Coluna 1;
- **Livro L2.C2:** Livro na Linha 2 e Coluna 2;
- **Livro L2.C3:** Livro na Linha 2 e Coluna 3;
- **Livro L2.C4:** Livro na Linha 2 e Coluna 4;
- **Livro L2.C5:** Livro na Linha 2 e Coluna 5.

As figuras, a seguir, serão utilizadas para exemplificar melhor a definição do que poderia ser encarado como um vetor (prateleira de livros) ou uma matriz (uma estante de livros).

Observe a prateleira de livros (vetor) e descubra como é utilizada a referência de índice para os livros alocados nessa estrutura de dados.

Índice	0	1	2	3	4
Prateleira de livros	Livro 1	Livro 2	Livro 3	Livro 4	Livro 5

Davi Leon (2020)

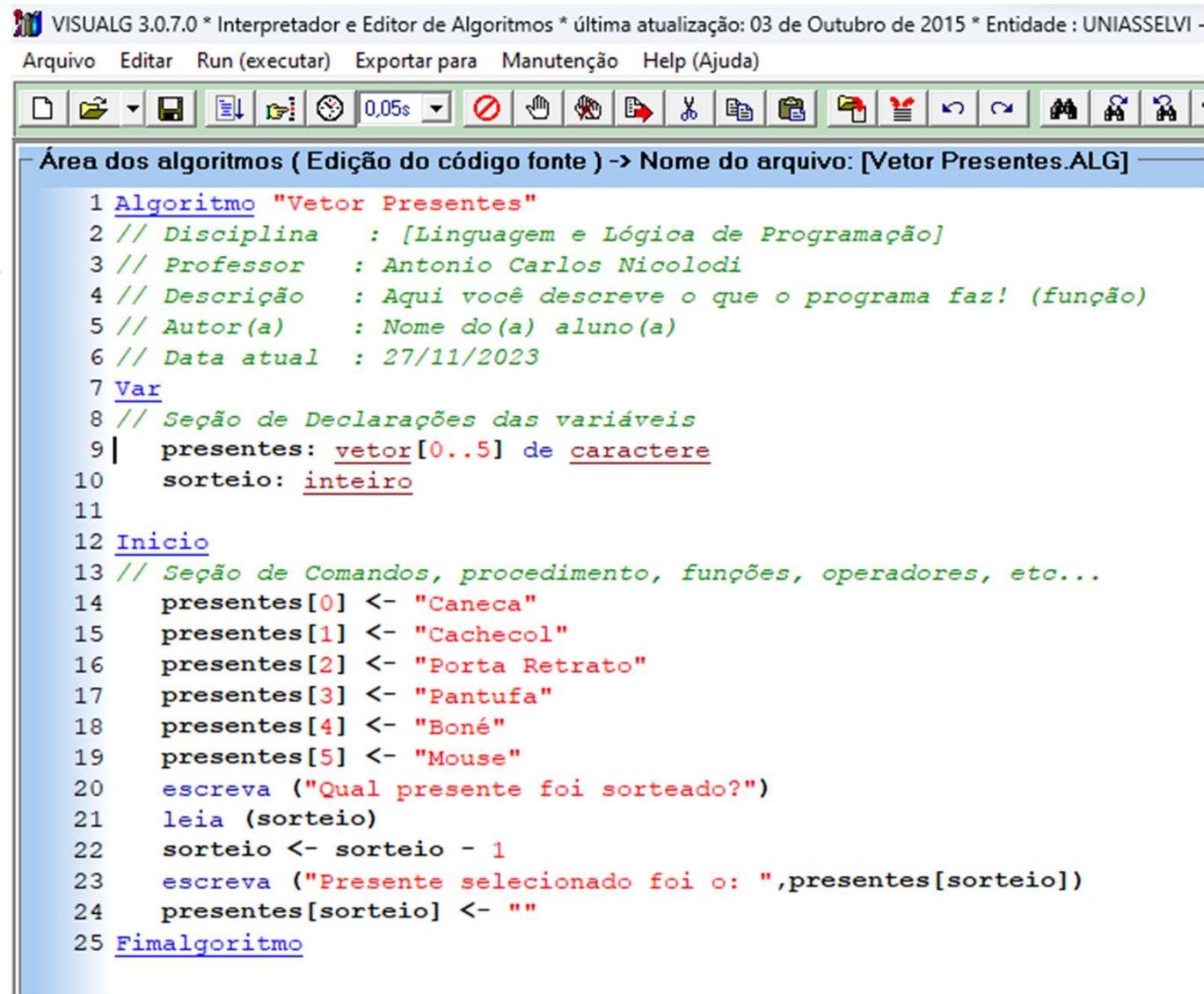
Figura 77 - Índice dos Livros no Vetor – Prateleira de Livros
Fonte: do Autor (2020)

Note que os livros possuem um índice que referencia sua localização a partir de um número que começa em zero (0) e vai se estendendo até o seu limite de capacidade de armazenamento, que, nesse exemplo, seria o índice quatro (4).

É muito importante que você grave que, na computação, os índices começam no número zero e não no número 1. Assim, se você tiver um vetor de tamanho 5, significará que o primeiro índice existente é o 0 e o último é o 4. Dessa forma, se você quisesse, por exemplo, pegar o livro 3 da prateleira de livros, teria que acessar o índice 2.

Veja a aplicação desses conceitos na prática, a partir de exemplos codificados em Portugol.

Para iniciar, utilizaremos o exemplo do Vetor de presentes, no qual há seis presentes que poderão ser sorteados e retirados por alguma pessoa, e que no exemplo foi o terceiro presente (índice 2).



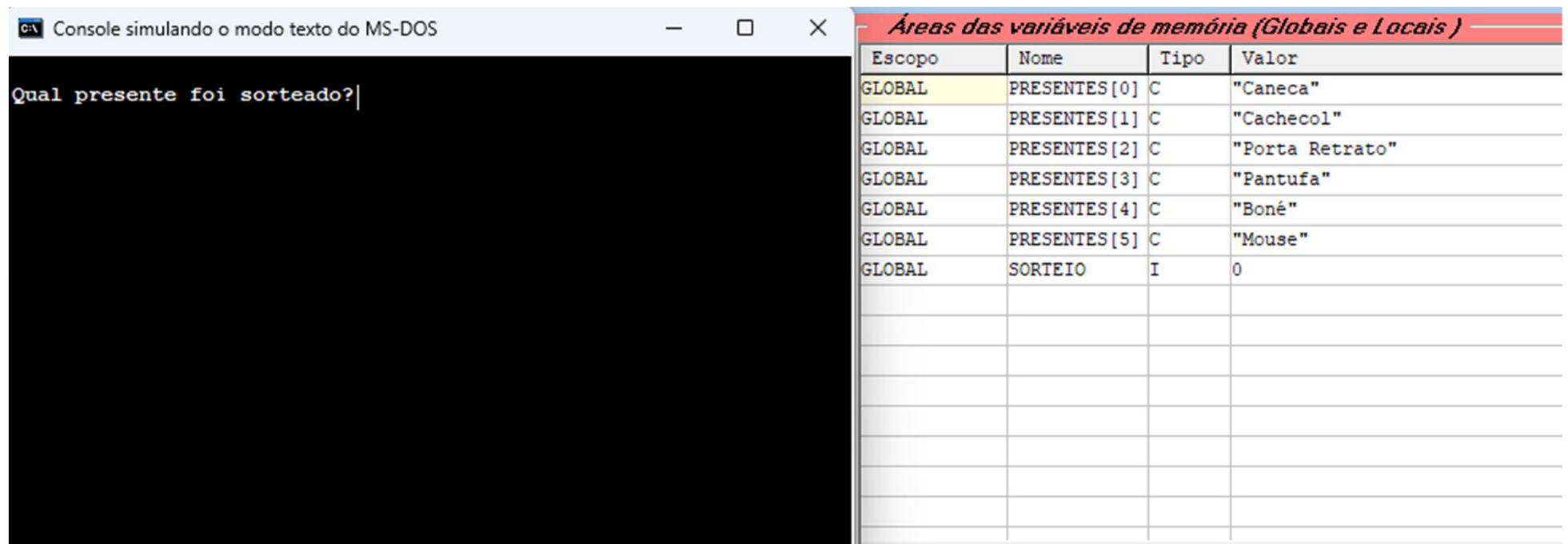
VISUALG 3.0.7.0 * Interpretador e Editor de Algoritmos * última atualização: 03 de Outubro de 2015 * Entidade : UNIasselvi

Arquivo Editar Run (executar) Exportar para Manutenção Help (Ajuda)

Área dos algoritmos (Edição do código fonte) -> Nome do arquivo: [Vetor Presentes.ALG]

```
1 Algoritmo "Vetor Presentes"
2 // Disciplina : [Linguagem e Lógica de Programação]
3 // Professor : Antonio Carlos Nicolodi
4 // Descrição : Aqui você descreve o que o programa faz! (função)
5 // Autor(a) : Nome do(a) aluno(a)
6 // Data atual : 27/11/2023
7 Var
8 // Seção de Declarações das variáveis
9 | presentes: vetor[0..5] de caractere
10 | sorteio: inteiro
11
12 Inicio
13 // Seção de Comandos, procedimento, funções, operadores, etc...
14 presentes[0] <- "Caneca"
15 presentes[1] <- "Cachecol"
16 presentes[2] <- "Porta Retrato"
17 presentes[3] <- "Pantufa"
18 presentes[4] <- "Boné"
19 presentes[5] <- "Mouse"
20 escreva ("Qual presente foi sorteado?")
21 leia (sorteio)
22 sorteio <- sorteio - 1
23 escreva ("Presente selecionado foi o: ", presentes[sorteio])
24 presentes[sorteio] <- ""
25 Fimalgoritmo
```

Ao executar esse programa, percebe-se que ele irá solicitar para que o usuário informe qual presente foi sorteado (linha 13) e a pessoa informará o número do presente (linha 14). Depois o valor informado (linha 15) é diminuído em 1, tendo em vista que os índices vão de 0 até 5 e os presentes de 1 a 6.



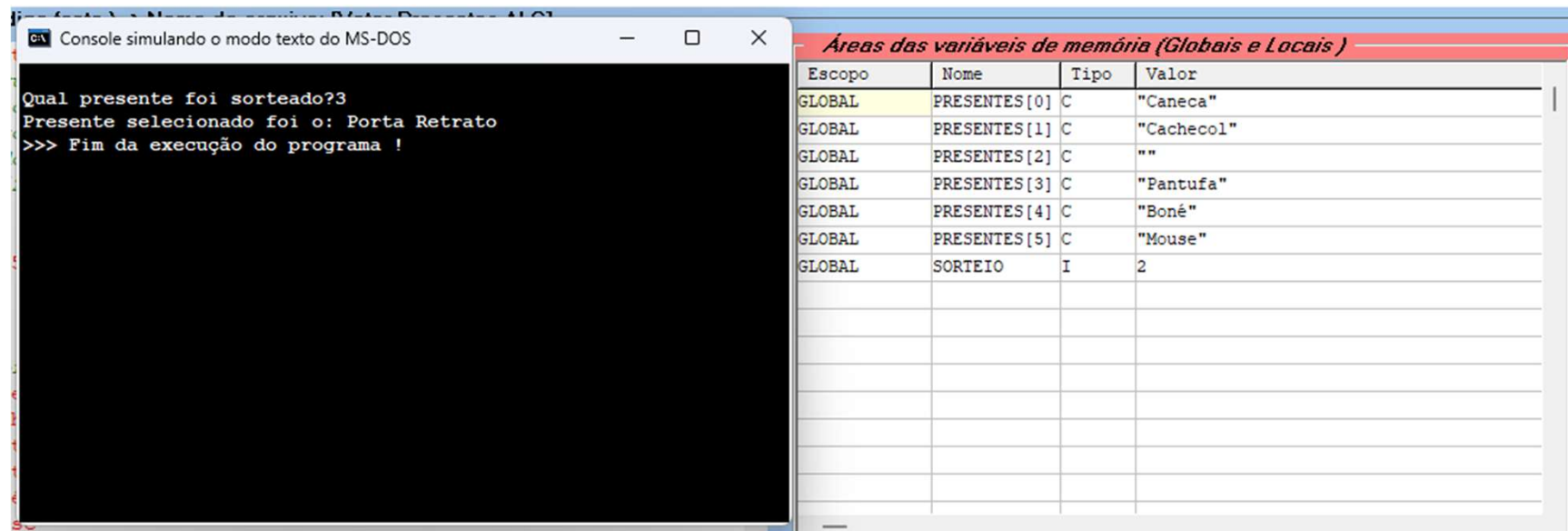
The image shows a DOS console window on the left and a memory variable table on the right. The console window has a title bar that reads "C:\> Console simulando o modo texto do MS-DOS". Inside the console, the text "Qual presente foi sorteado?" is displayed with a cursor at the end. The table on the right is titled "Áreas das variáveis de memória (Globais e Locais)". It has four columns: "Escopo", "Nome", "Tipo", and "Valor". The table contains seven rows of data, all with "GLOBAL" in the "Escopo" column. The "Nome" column lists "PRESENTES[0]" through "PRESENTES[5]" and "SORTEIO". The "Tipo" column lists "C" for the arrays and "I" for "SORTEIO". The "Valor" column lists "Caneca", "Cachecol", "Porta Retrato", "Pantufa", "Boné", "Mouse", and "0".

Escopo	Nome	Tipo	Valor
GLOBAL	PRESENTES[0]	C	"Caneca"
GLOBAL	PRESENTES[1]	C	"Cachecol"
GLOBAL	PRESENTES[2]	C	"Porta Retrato"
GLOBAL	PRESENTES[3]	C	"Pantufa"
GLOBAL	PRESENTES[4]	C	"Boné"
GLOBAL	PRESENTES[5]	C	"Mouse"
GLOBAL	SORTEIO	I	0

Note que o console é exibida na execução para solicitar qual o presente sorteado (quadro vermelho).

Observe também que nesse tempo de execução estão armazenados no vetor (quadro verde) os seis presentes.

Caso o presente 3 (índice 2) seja selecionado, aparecerá a mensagem de que o presente escolhido foi o “Porta-Retratos” (linha 16) e posteriormente esse item do presente vai ser eliminado da posição (linha 17), deixando-o vazio.



E no caso de matrizes? Nesta situação, é preciso compreender que o índice de um determinado elemento será determinado pelo número que define o encontro da linha com a coluna, fornecendo assim o ponto exato do elemento. Veja como ficariam os índices no exemplo da estante de livros.

Índices	0	1	2	3	4
0	Livro 1	Livro 2	Livro 3	Livro 4	Livro 5
1	Livro 6	Livro 7	Livro 8	Livro 9	Livro 10
2	Livro 11	Livro 12	Livro 13	Livro 14	Livro 15

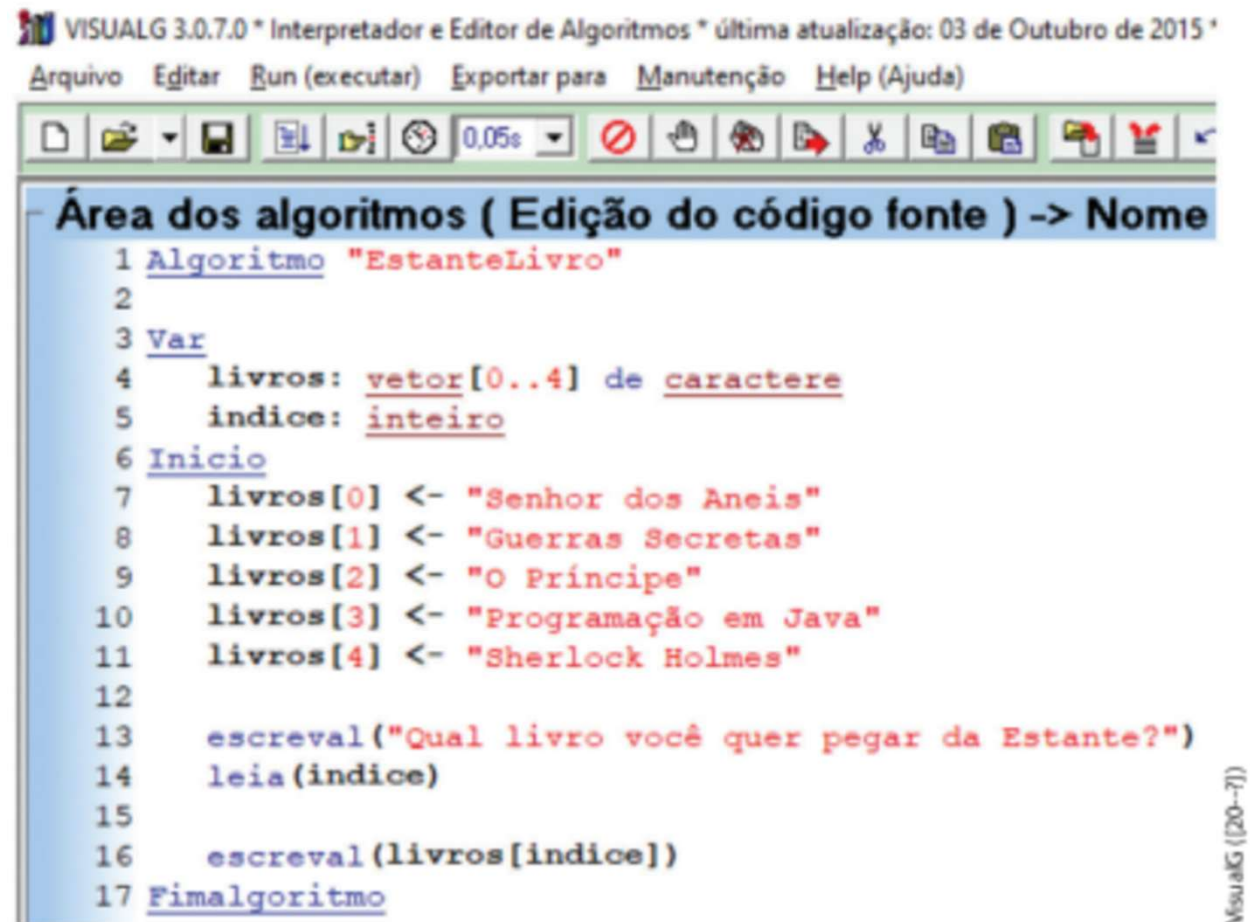
Davi Leon (2020)

Figura 78 - Índice dos Livros na Matriz – Estante de Livros
Fonte: do Autor (2020)

Note que agora o índice de cada elemento é formado pela referência da linha e da coluna que, em seu cruzamento, aponta exatamente a localização do dado. Se você quiser, por exemplo, resgatar o Livro 8, terá que acessar o seguinte índice: Linha 1 e Coluna 2.

Tal como nos vetores, as matrizes também iniciam os seus índices com o zero (0), ou seja, o primeiro elemento de uma matriz estará no índice da Linha 0 e da Coluna 0.

Veja agora como fica a estante de livros em Portugal, levando em consideração que a estante é capaz de armazenar até cinco livros.



```
1 Algoritmo "EstanteLivro"
2
3 Var
4   livros: vetor[0..4] de caractere
5   indice: inteiro
6 Inicio
7   livros[0] <- "Senhor dos Aneis"
8   livros[1] <- "Guerras Secretas"
9   livros[2] <- "O Príncipe"
10  livros[3] <- "Programação em Java"
11  livros[4] <- "Sherlock Holmes"
12
13  escreval("Qual livro você quer pegar da Estante?")
14  leia(indice)
15
16  escreval(livros[indice])
17 Fimalgoritmo
```

Figura 79 - Screenshot do VisualG – Estante de Livros em um Vetor
Fonte: do Autor (2020)

Acompanhe agora a análise linha após linha da declaração do programa proposto da Estante de Livros.

LINHA	ESTANTE DE LIVRO
4	Declarando a variável vetor chamada "livros", que terá tamanho cinco (0 até 4) e que armazenará apenas tipos de dados caractere.
5	Variável índice, que será usado para o usuário informar qual livro quer pegar da estante.
7 até 11	Armazenando quais os livros vão existir na estante, na posição zero até a última, que é o índice quatro.
13 e 14	Pergunta-se para o usuário qual o índice do livro que ele deseja pegar na estante. Ele irá informar um número, que será armazenado na variável índice.
16	O programa apresenta o livro que está na posição da variável índice dentro do vetor livros.

Tabela 33 - Interpretação das linhas de código em Java
Fonte: do Autor (2020)

Para compreender melhor a estrutura do vetor da linguagem Portugol, acompanhe os detalhes a seguir.

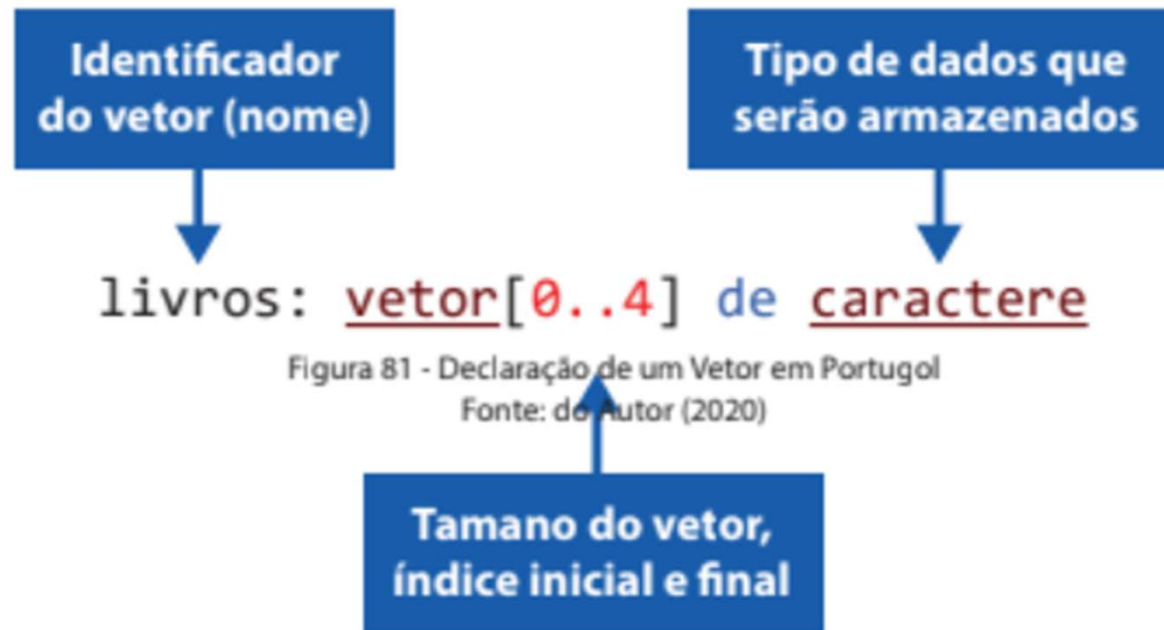


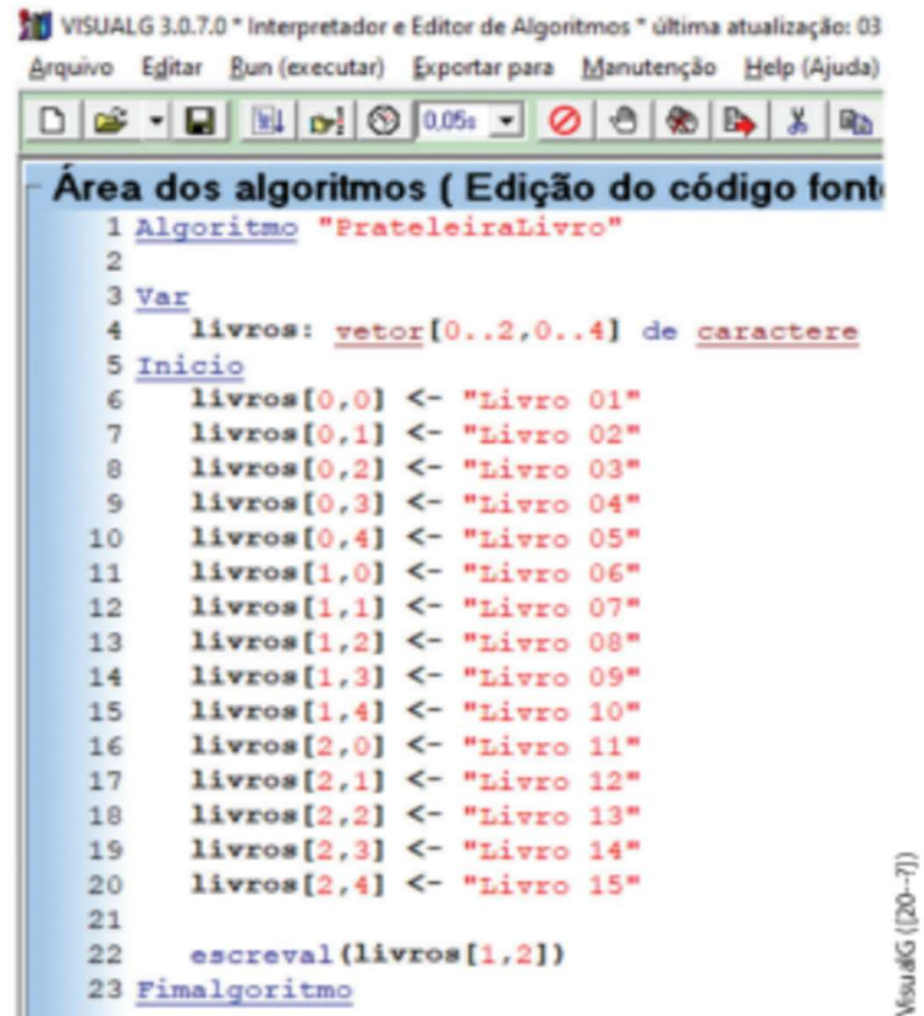
Figura 81 - Declaração de um Vetor em Portugol
Fonte: do Autor (2020)

David Leon (2020)

E quanto às Matrizes?

Como é possível representar e declarar uma matriz em Portugal?

Basicamente é preciso adicionar mais uma dimensão na sintaxe da declaração que anteriormente era do Vetor, transformando-a assim em uma estrutura com multidimensões.



The screenshot shows the VisualG 3.0.7.0 IDE interface. The title bar reads "VISUALG 3.0.7.0 * Interpretador e Editor de Algoritmos * última atualização: 03". The menu bar includes "Arquivo", "Editar", "Run (executar)", "Exportar para", "Manutenção", and "Help (Ajuda)". The toolbar contains icons for file operations, execution, and debugging. The main window, titled "Área dos algoritmos (Edição do código fonte)", displays the following code:

```
1 Algoritmo "PrateleiraLivro"
2
3 Var
4   livros: vetor[0..2,0..4] de caractere
5 Inicio
6   livros[0,0] <- "Livro 01"
7   livros[0,1] <- "Livro 02"
8   livros[0,2] <- "Livro 03"
9   livros[0,3] <- "Livro 04"
10  livros[0,4] <- "Livro 05"
11  livros[1,0] <- "Livro 06"
12  livros[1,1] <- "Livro 07"
13  livros[1,2] <- "Livro 08"
14  livros[1,3] <- "Livro 09"
15  livros[1,4] <- "Livro 10"
16  livros[2,0] <- "Livro 11"
17  livros[2,1] <- "Livro 12"
18  livros[2,2] <- "Livro 13"
19  livros[2,3] <- "Livro 14"
20  livros[2,4] <- "Livro 15"
21
22  escreval(livros[1,2])
23 Fimalgoritmo
```

On the right side of the IDE, a vertical status bar indicates "VisualG ([20--7])".

Figura 83 - Screenshot do VisualG – Prateleira de Livros
Fonte: do Autor (2020)

A proposta de código é condizente com a situação apresentada anteriormente em uma das figuras, onde havia uma prateleira de livros com 3 linhas e 5 colunas, perfazendo assim a possibilidade de alocar 15 livros. Veja que, no fim do algoritmo, apenas está sendo apresentado o Livro 08, que se encontra no seguinte índice: Linha 1 e Coluna 2.

Veja agora um pouco mais sobre a sintaxe de declaração de uma matriz em Portugol.

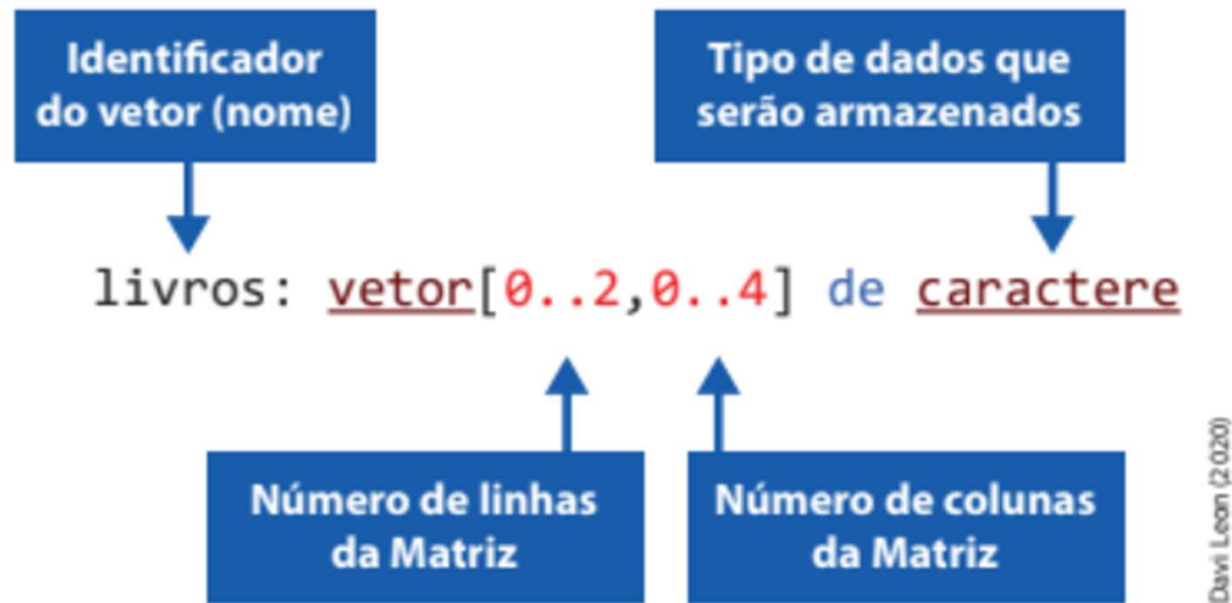


Figura 84 - Declaração de uma Matriz em Portugol
Fonte: do Autor (2020)

REGISTROS

Existem vários tipos de estruturas de dados. A princípio, quando se estuda matrizes e vetores, tem-se como objeto variáveis unidimensionais e multidimensionais homogêneas, ou seja, são estruturas preparadas para receber valores de somente um tipo de dados, no momento em que foram declaradas.

As variáveis possuem a função, além de armazenarem tipos de dados, de representar entidades identificadas no problema que será resolvido pelo programa que está sendo criado. Surge então a necessidade de armazenar, em uma mesma variável, diferentes tipos de dados.

Daí surgem os registros.

Registros são estruturas que conseguem agregar vários tipos de dados, não se limitando a utilizar os tipos de dados primitivos fornecidos pelas linguagens de programação. Cada dado contido em um registro é chamado de campo. Os campos podem ser de diferentes tipos primitivos, sendo, por esta razão, os registros serem conhecidos como variáveis compostas heterogêneas (ASCENCIO; CAMPOS, 2012).

REGISTROS

Vejamos no exemplo como é simples fazer um cadastro de usuário utilizando registros. O algoritmo deve ser capaz de armazenar os dados de 3 usuários. Em cada usuário, deverá ser solicitado os seguintes dados:

1. Nome Completo
2. Idade
3. Bairro
4. Cidade

No final o algoritmo deve ser capaz de retornar os dados da seguinte forma:
Usuario(a): **Nome Completo**, tem **Idade** anos, é do Bairro **Bairro** da Cidade **Cidade**.

```

1 Algoritmo "Registros"
2 |
3 Tipo
4 // Registro tem que ser declarado o tipo
5     usuario = registro
6         nome : caractere
7         idade : inteiro
8         bairro : caractere
9         cidade : caractere
10 fimregistro
11
12 Var
13 // Seção de Declarações das variáveis
14 cad_usuarios : vetor[1..3] de usuario // variavel mista, posso criar/inventa.
15 ind : inteiro
16

```

```
17 Inicio
18 // Seção de Comandos, procedimento, funções, operadores, etc...
19
20 escreval ("Novo Cadastro de Usuarios")
21
22 // laço de repetição - Inserção dos Dados
23 para ind de 1 ate 2 faca
24
25     escreva ("Digite o Nome Completo do Usuário:")
26     leia (cad_usuarios[ind].nome)
27
28     escreva ("Digite a Idade do Usuário:")
29     leia (cad_usuarios[ind].idade)
30
31     escreva ("Digite o Bairro do Usuário:")
32     leia (cad_usuarios[ind].bairro)
33
34     escreva ("Digite a cidade do Usuário:")
35     leia (cad_usuarios[ind].cidade)
36
37 fimpara
38
```



```

39 //Retorno dos dados do Cadastro do Usuário
40 escreval ("Usuarios Cadastrados!")
41 //Usuario(a): Nome Completo, tem Idade anos, é do Bairro Bairro da Cidade Ciudad
42 para ind de 1 ate 2 faca
43
44     escreva ("Usuário(a): ", cad_usuarios[ind].nome, ",")
45     escreva (" tem", cad_usuarios[ind].idade, " anos,")
46     escreva (" é Bairro ", cad_usuarios[ind].bairro)
47     escreval (" da cidade ", cad_usuarios[ind].cidade, ".")
48
49 fimpara
50
51 Fimalgoritmo

```

```

Novo Cadastro de Usuarios
Digite o Nome Completo do Usuário:Jose Pedro
Digite a Idade do Usuário:25
Digite o Bairro do Usuário:Poty
Digite a cidade do Usuário:Teresina
Digite o Nome Completo do Usuário:Joao Pedro
Digite a Idade do Usuário:22
Digite o Bairro do Usuário:Furnas
Digite a cidade do Usuário:Tanabi
Usuarios Cadastrados!
Usuário(a): Jose Pedro, tem 25 anos, é Bairro Poty da cidade Teresina.
Usuário(a): Joao Pedro, tem 22 anos, é Bairro Furnas da cidade Tanabi.

>>> Fim da execução do programa !

```

ESTRUTURA DE DADOS

Estruturas de dados definem a organização e suas operações (métodos de acesso) sobre um determinado conjunto de informações. Essa organização permite um melhor processamento e é usada para grandes volumes de dados.

Por exemplo: você deseja armazenar o nome de todos os alunos da sua turma.

O sistema deve definir a organização (como salvar) e, conseqüentemente, o acesso ao conjunto de operações sobre esses dados (inserir ou remover um aluno, editar um nome ou trocar a posição de um dado).

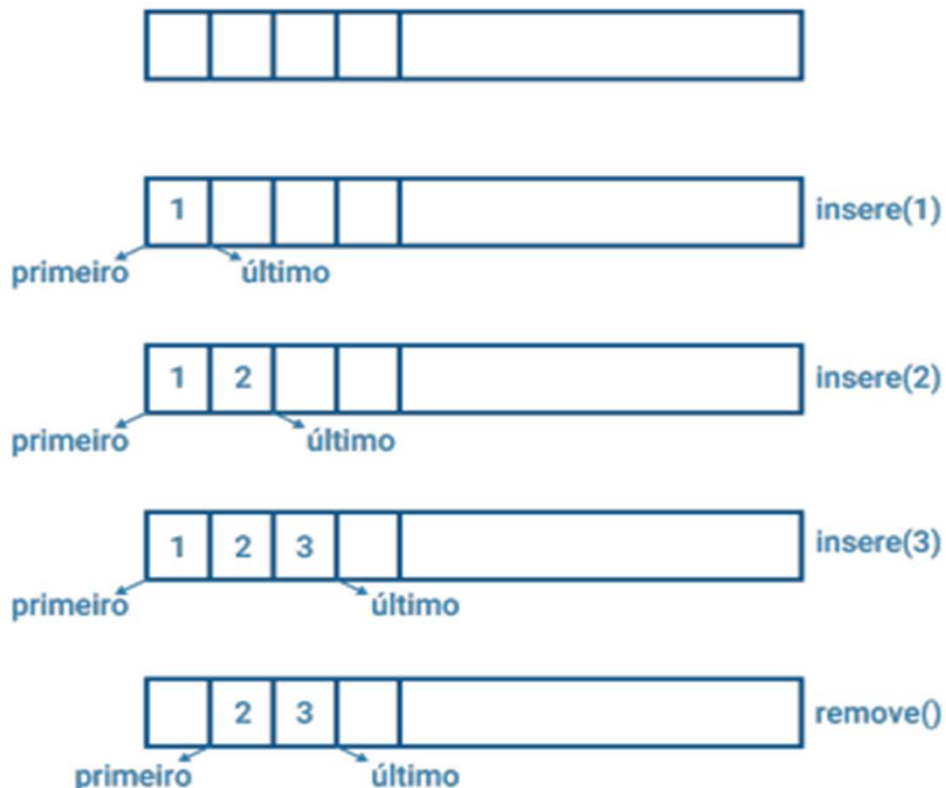
Há vários tipos de estrutura de dados.

Lista (list) – Uma lista armazena dados do mesmo tipo em sequência. Os dados não precisam estar necessariamente armazenados em sequência física na memória do computador, mas há uma sequência lógica entre eles. Cada elemento da lista é chamado de nó.

Em uma lista sequencial ou contígua, os nós estão em sequência lógica e física. Em uma lista encadeada, não há a necessidade da sequência física, apenas a sequência lógica deve ser mantida e o último nó é uma célula nula.

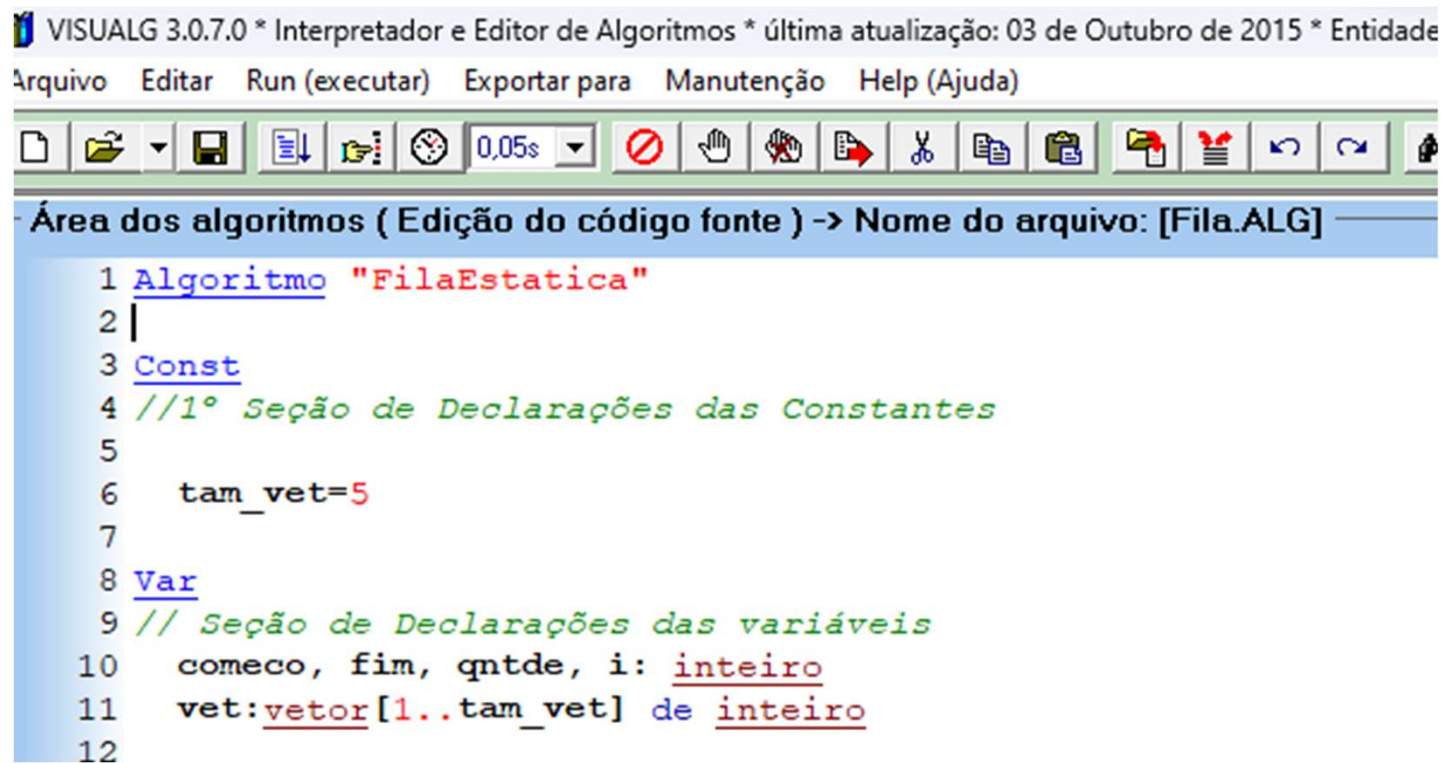
As listas são subdivididas em filas, pilhas e vetores, dependendo do acesso aos nós.

Fila (queue) - A fila é uma estrutura do tipo *first in first out* (FIFO), na qual o primeiro elemento a ser inserido será o primeiro a ser retirado. Assim como em uma fila de pessoas, o primeiro que chegou no balcão, será o primeiro a ser atendido, e assim por diante.



Agora vejamos como seria o funcionamento de Fila em português, iremos **inicializar, cadastrar, retirar, verificar se está vazia, verificar se esta cheia, imprimir seu conteúdo e o próximo na Fila.**

Primeiro passo é declarar uma variável constante a qual será o tamanho máximo de nossa Fila. Depois criar as variáveis de vetor, uma de incrementador, o começo, o fim e outra para a quantidade da Fila.



The screenshot shows the VISUALG 3.0.7.0 application window. The title bar reads "VISUALG 3.0.7.0 * Interpretador e Editor de Algoritmos * última atualização: 03 de Outubro de 2015 * Entidade". The menu bar includes "Arquivo", "Editar", "Run (executar)", "Exportar para", "Manutenção", and "Help (Ajuda)". The toolbar contains various icons for file operations and execution. The main area is titled "Área dos algoritmos (Edição do código fonte) -> Nome do arquivo: [Fila.ALG]". The code is as follows:

```
1 Algoritmo "FilaEstatica"
2 |
3 Const
4 //1º Seção de Declarações das Constantes
5
6     tam_vet=5
7
8 Var
9 // Seção de Declarações das variáveis
10     começo, fim, qntde, i: inteiro
11     vet:vetor[1..tam_vet] de inteiro
12
```

Segundo passo é criar as funções da Fila, onde precisamos saber se a Fila esta cheia ou vazia e acessar o próximo elemento da Fila.

```
13 //2° Criar funcoes da Fila
14 //Fila cheia
15 funcao esta_cheia():logico
16 inicio
17     se qntde=tam_vet entao
18         retorne verdadeiro
19     senao
20         retorne falso
21     fimse
22 fimfuncao
23
24 //Fila Vazia
25 funcao esta_vazia():logico
26 inicio
27     se qntde=0 entao
28         retorne verdadeiro
29     senao
30         retorne falso
31     fimse
32 fimfuncao
```

```
33
34 //Função Acessar Proximo da Fila
35 funcao mostrar_proximo():inteiro
36 inicio
37     se esta_vazia() entao
38         escreval("A Fila está Vazia")
39     senao
40         retorne vet[comeco]
41     fimse
42 fimfuncao
```

Terceiro passo é criar os procedimentos da Fila, onde iremos cadastrar, retirar, mostrar a Fila.

```
44 //3º Procedimento de Cadastro da Fila
45 procedimento cadastro(num:inteiro)
46 inicio
47     se esta_cheia() entao
48         escreval("A Fila está Cheia")
49     senao
50         fim<-fim+1
51         se fim>tam_vet entao
52             fim<-1
53         fimse
54         vet[fim]<-num
55         qntde<-qntde+1
56     fimse
57 fimprocedimento
```

```
59 // Procedimento Retirar da Fila
60 procedimento retirar()
61 inicio
62     se esta_vazia() entao
63         escreval("A Fila está Vazia")
64     senao
65         comeco<-comeco+1
66         se comeco>tam_vet entao
67             comeco<-1
68         fimse
69         qntde<-qntde-1
70     fimse
71 fimprocedimento
```

```

73 //Procedimento Mostrar a Fila
74 procedimento mostrar_fila()
75     inicio
76     escreval
77     escreval("-----Posições da Fila-----")
78     se qntde>0 entao
79         se comeco <=fim entao           //Começo ao fim ou fim ao começo
80             para i de comeco ate fim faca
81                 escreva(vet[i], " ")
82             fimpara
83         senao
84             para i de comeco ate tam_vet faca //começo ate o final
85                 escreva(vet[i], " ")
86             fimpara
87             para i de 1 ate fim faca
88                 escreva(vet[i], " ")      //1 ate fim
89             fimpara
90         fimse
91     fimse
92     escreval
93     escreval("-----")
94     escreval
95     escreval
96     fimprocedimento

```


E por fim, os comandos para Desempilhar, Empilhar, Mostrar a Pilha, Mostrar o Topo, Mostrar a Pilha cheia e a Pilha Vazia.

```
99 Inicio
100 // Seção de Comandos, procedimento, funções, operadores, etc...
101   comeco<-1  //iniciar as variaveis
102   fim<-0     //qnd cadastrar ele vai incrementar
103   escreval("===== Fila =====")
104   retirar()
105   cadastro(50)
106   cadastro(55)
107   cadastro(51)
108   cadastro(58)
109   cadastro(57)
110   cadastro(53)
111   mostrar_filas()
112   retirar()
113   mostrar_filas
114   cadastro(00)
115   mostrar_filas()
116   escreval("Proximo da Fila", mostrar_proximo())
117
118 Fimalgoritmo
```

```
===== Fila =====
A Fila está Vazia
A Fila está Cheia

-----Posições da Fila-----
50 55 51 58 57
-----

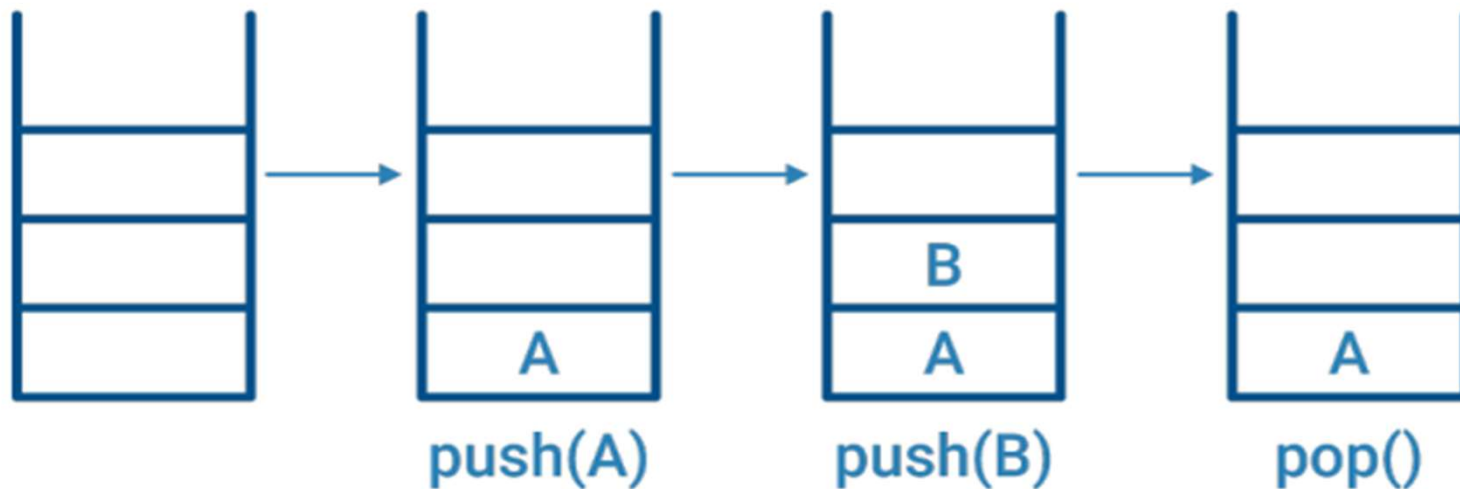
-----Posições da Fila-----
55 51 58 57
-----

-----Posições da Fila-----
55 51 58 57 88
-----

Proximo da Fila 55

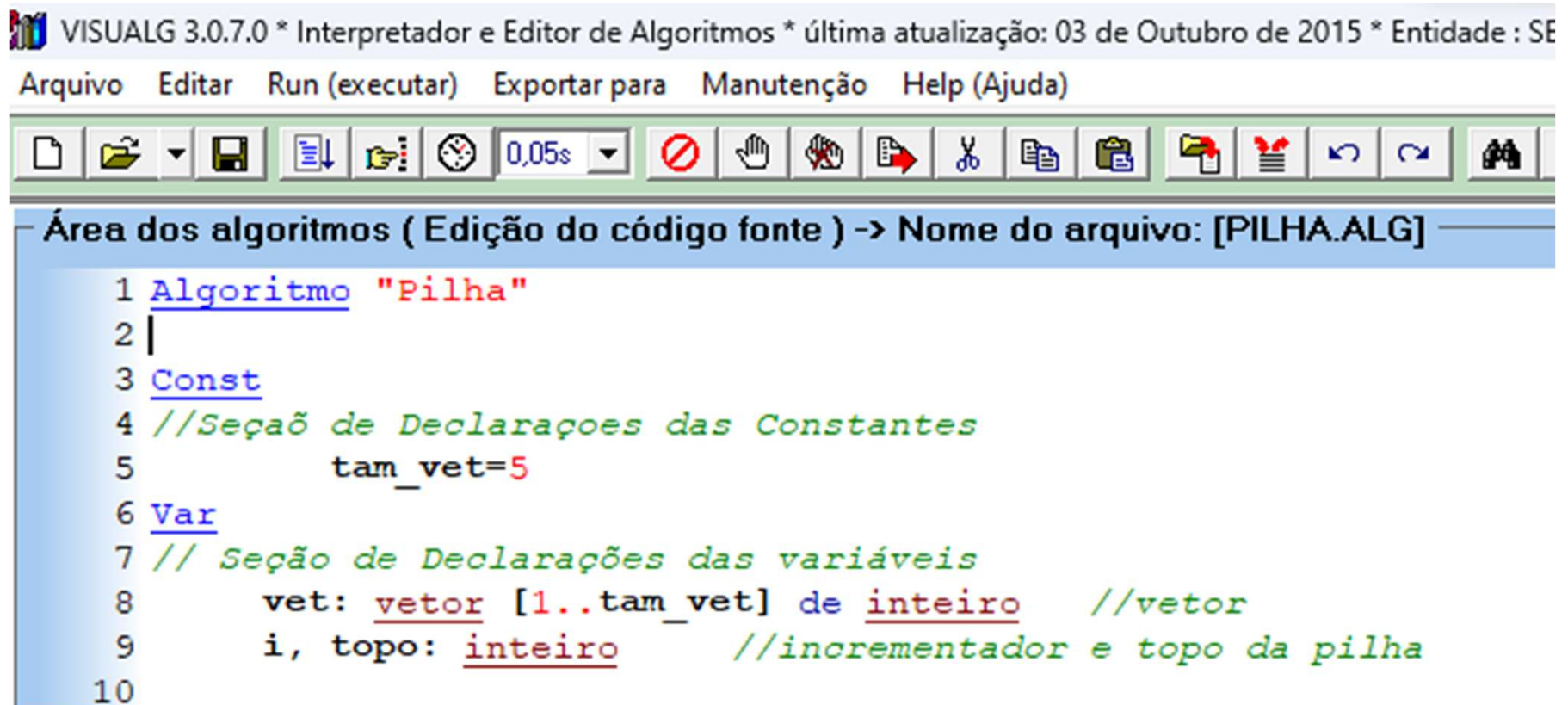
>>> Fim da execução do programa !
```


Pilha (stacks) - A pilha é uma estrutura do tipo *last in first out* (LIFO). O último elemento a ser inserido será o primeiro elemento a ser retirado. Podemos ter acesso ao topo dessa pilha, inserir um novo item, remover o item corrente. É como uma pilha de pratos: colocamos ou retiramos pratos no topo da pilha.



Agora vejamos como seria o funcionamento de pilha em português, iremos **inicializar**, **empilhar**, **desempilhar**, **verificar se está vazia**, **verificar se esta cheia** e **imprimir seu conteúdo**.

Primeiro passo é declarar uma variável constante a qual será o tamanho máximo de nossa Pilha. Depois criar as variáveis de vetor, uma de incrementador e outra para o topo da pilha.



VISUALG 3.0.7.0 * Interpretador e Editor de Algoritmos * última atualização: 03 de Outubro de 2015 * Entidade : SE

Arquivo Editar Run (executar) Exportar para Manutenção Help (Ajuda)

Área dos algoritmos (Edição do código fonte) -> Nome do arquivo: [PILHA.ALG]

```
1 Algoritmo "Pilha"
2 |
3 Const
4 //Seção de Declarações das Constantes
5     tam_vet=5
6 Var
7 // Seção de Declarações das variáveis
8     vet: vetor [1..tam_vet] de inteiro //vetor
9     i, topo: inteiro //incrementador e topo da pilha
10
```

Segundo passo é criar as funções da pilha, onde precisamos saber se a pilha esta cheia ou vazia.

```
13 //Empilhar - primeiro saber se a pilha esta cheia
14 procedimento empilhar(num: inteiro)
15     inicio
16         se esta_cheia() entao
17             escreval("A Pilha está cheia")
18         senao
19             topo <- topo + 1
20             vet[topo] <- num //vetor recebe numero que passaremos
21         fimse
22     fimprocedimento
23
24 // Desempilhar - primeiro saber se nao esta vazia
25
26 procedimento desempilhar()
27     inicio
28         se esta_vazia() entao
29             escreval("A Pilha está vazia")
30         senao
31             topo <- topo - 1
32         fimse
33     fimprocedimento
```

Terceiro passo é criar os procedimentos da pilha, onde iremos empilhar, desempilhar, mostrar o topo e a própria pilha.

```
35  //Mostrar topo
36
37  procedimento mostrar_topo()
38  inicio
39      escreval
40      escreval("---Topo da Pilha---")
41      escreval("-----", vet[topo], " -----")
42      escreval("-----")
43  fimprocedimento
44
45  //Mostrar Pilha
46
47  procedimento mostrar_pilha()
48  inicio
49      escreval
50      escreval("-----Pilha-----")
51      para i de 1 ate topo faca
52          escreva(vet[i], " ")
53      fimpara
54      escreval
55      escreval("-----")
56      escreval
57  fimprocedimento
```

```

59 //2° - Criar funções da pilha
60
61 //Função Pilha Cheia
62 função esta_cheia():logico
63 inicio
64     se topo=tam_vet entao
65         retorne verdadeiro
66     senao
67         retorne falso
68     fimse
69 fimfunção
70
71 //Função Pilha Vazia
72 função esta_vazia():logico
73 inicio
74     se topo=0 entao
75         retorne verdadeiro
76     senao
77         retorne falso
78     fimse
79 fimfunção

```

E por fim, os comandos para Desempilhar, Empilhar, Mostrar a Pilha, Mostrar o Topo, Mostrar a Pilha cheia e a Pilha Vazia.

```
80 Inicio
81 // Seção de Comandos, procedimento, funções, operadores, etc...
82
83 escreval("====Exemplo de Pilha====")
84 desempilhar()
85 empilhar(1)
86 empilhar(50)
87 empilhar(32)
88 empilhar(41)
89 empilhar(24)
90 mostrar_pilha()
91 empilhar(99)
92 mostrar_topo()
93 desempilhar()
94 mostrar_pilha()
95 Fimalgoritmo
```

```
====Exemplo de Pilha====
A Pilha está vazia

-----Pilha-----
1  50  32  41  24
-----

A Pilha está cheia

---Topo da Pilha---
----- 24 -----
-----

-----Pilha-----
1  50  32  41
-----

>>> Fim da execução do programa !
```